République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Jendouba
Institut Supérieur d'Informatique du Kef

Rapport de Projet de Fin d'Études

Présenté à

Institut Supérieur d'Informatique du Kef

En vue de l'obtention du

Diplôme de Mastère Professionnel

Option :

Applications Web Intelligentes

Par

Iheb El Ech

Conception et développement d'une plateforme intelligente d'aide au recrutement et de mise en correspondance entre CV et offres d'emploi

Soutenue le : 17/10/2026

Devant le jury composé de :

Président :	**Dr. Emna Khanfir**
Rapporteur :	**Dr. Ahmed Dridi**
Encadrant académique :	**Dr. Louati Aymen**
Encadrante professionnelle :	**Dr. Trabelsi Emna**

Année Universitaire : 2025-2026

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrante professionnelle, **Dr. Trabelsi Emna**

Signature et cachet

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **Dr. Louati Aymen**

Signature

Dédicaces

C'est avec une profonde gratitude et des mots sincères que je dédie ce modeste travail.

À mes chers parents, Abedlwaheb El Ech et Habiba Omri. *Votre soutien indéfectible et votre amour inconditionnel ont été une source inestimable de force et d'inspiration. Que ce travail soit le témoignage de ma profonde reconnaissance et de ma gratitude infinie. Que Dieu vous bénisse, vous accorde santé et longue vie.*

À ma petite sœur, Hayfa El Ech. *Merci pour ton soutien, ta présence et tes encouragements tout au long de ce parcours.*

À ma famille. *Je vous remercie pour votre affection, votre confiance et votre soutien constant.*

À mes amis. *Pour tous les instants inoubliables que nous avons passés ensemble, je vous remercie du fond du cœur. Que ce travail soit également le témoignage de mon amitié et de ma reconnaissance pour votre soutien.*

El Ech Iheb

Remerciements

Je remercie Dieu le Tout-Puissant de m'avoir donné la santé et la volonté d'entamer et de terminer ce rapport.

J'exprime également ma sincère gratitude à mon encadrant à l'ISI Kef, **Dr. Aymen Louati**, pour son aide et ses conseils concernant l'acheminement du projet et l'architecture du rapport. Je le remercie pour le temps qu'il m'a consacré, pour les informations qu'il m'a fournies avec compréhension, ainsi que pour sa patience et son amabilité.

J'adresse également mes remerciements à **Dr. Emna Trabelsi** pour sa pédagogie, sa confiance et le soutien qu'elle m'a accordé tout au long de ce projet au sein de CyberCloud Solutions.

Je tiens de même à remercier tous les enseignants de l'ISI Kef qui m'ont enseigné pendant ces trois années successives et qui, par leurs compétences, m'ont soutenu dans la poursuite de mes études.

Enfin, je présente mes profonds remerciements et mon respect aux membres du jury qui ont accepté de juger mon travail.

Table des matières

Dédicaces iii

Remerciements iv

Liste des abréviations x

Introduction générale 1

1 Contexte général et étude préalable 2

1.1 Présentation de l'organisme d'accueil . . . 3

1.1.1 Présentation de CyberCloud Solutions . . . 3

1.1.2 Activités et domaines d'expertise . . . 4

1.1.3 Cadre du stage . . . 4

1.2 Contexte du projet . . . 4

1.3 Étude de l'existant . . . 5

1.3.1 Processus manuel de présélection . . . 5

1.3.2 Approches lexicales et ATS traditionnels . . . 5

1.3.3 Solutions numériques étudiées . . . 6

1.3.4 Synthèse comparative . . . 6

1.4 Critique de l'existant . . . 6

1.5 Problématique . . . 7

1.6 Solution proposée : JobTest . . . 8

1.7 Objectifs du projet . . . 8

1.7.1 Objectif général . . . 8

1.7.2 Objectifs spécifiques . . . 8

1.8 Méthodologie de conduite du projet . . . 9

1.8.1 Justification du modèle en cascade . . . 9

1.8.2 Phases appliquées à JobTest . . . 10

2 Analyse et spécification des besoins 11

2.1 Périmètre fonctionnel de JobTest . . . 12

2.2 Identification des acteurs et parties prenantes . . . 12

2.2.1 Visiteur non authentifié . . . 12

2.2.2 Recruteur authentifié . . . 13

2.2.3 Candidat en tant qu'entité métier . . . 13
2.2.4 Partie prenante technique . . . 13
2.3 Spécification des besoins fonctionnels . . . 13
2.3.1 Authentification du recruteur . . . 14
2.3.2 Gestion des offres et des candidats . . . 14
2.3.3 Évaluation et restitution . . . 14
2.3.4 Historique, classement et réglages . . . 15
2.4 Spécification des besoins non fonctionnels . . . 15
2.5 Règles de gestion . . . 18
2.6 Diagramme global des cas d'utilisation . . . 18
2.7 Raffinement et description textuelle des cas d'utilisation . . . 19
2.7.1 UC-01 : s'authentifier . . . 20
2.7.2 UC-02 : gérer une offre d'emploi . . . 21
2.7.3 UC-03 : gérer un candidat . . . 22
2.7.4 UC-04 : évaluer un CV . . . 23
2.7.5 UC-05 : demander une analyse approfondie . . . 25
2.7.6 UC-06 : générer un rapport PDF . . . 26
2.7.7 UC-07 : classer plusieurs candidats . . . 27
2.7.8 UC-08 : consulter l'historique . . . 29
2.8 Matrice de traçabilité . . . 30
2.9 Contraintes et hypothèses . . . 31

3 Conception fonctionnelle et architecturale 33

Introduction . . . 34
3.1 Démarche de conception . . . 34
3.2 Vue fonctionnelle globale . . . 34
3.3 Architecture logique de JobTest . . . 35
3.4 Architecture applicative et responsabilités . . . 35
3.5 Conception statique – diagramme de classes global . . . 36
3.6 Conception dynamique – diagrammes de séquence . . . 37
3.6.1 Authentification du recruteur . . . 37
3.6.2 Création d'une offre d'emploi . . . 38
3.6.3 Évaluation d'un CV par rapport à une offre . . . 38
3.6.4 Analyse approfondie avec Ollama . . . 39
3.6.5 Génération du rapport PDF . . . 40

3.7 Conception des données . . . 40
3.8 Principes de conception retenus . . . 41
Conclusion . . . 41

4 Conception et évaluation du moteur intelligent 42

Introduction . . . 43
4.1 Objectifs et principes de conception . . . 43
4.2 Prétraitement et structuration des documents . . . 43
4.3 Similarité lexicale TF–IDF . . . 43
4.4 Similarité sémantique par embeddings . . . 44
4.5 Rapprochement structuré des compétences . . . 44
4.5.1 Contrôle des preuves avant le calcul . . . 45
4.6 Fusion hybride et décision . . . 45
4.7 Explicabilité et couche d'analyse locale . . . 46
4.7.1 Construction du contexte et format de réponse . . . 47
4.7.2 Garde-fou de cohérence et tentative d'injection . . . 47
4.8 Jeu de données et protocole expérimental . . . 48
4.8.1 Pourquoi un corpus synthétique . . . 48
4.8.2 Partitionnement et métriques . . . 50
4.9 Résultats et analyse comparative . . . 51
Verdicts affichés par la version finale . . . 52
Choix de la pondération . . . 54
Robustesse face aux CV manipulés . . . 55
4.10 Limites, risques et perspectives . . . 55
Conclusion . . . 56

5 Réalisation et présentation de la plateforme 57

5.1 Environnement matériel et logiciel . . . 58
5.1.1 Environnement matériel . . . 58
5.1.2 Environnement logiciel . . . 58
5.2 Choix des technologies . . . 59
5.2.1 Technologies frontend . . . 59
5.2.2 Technologies backend . . . 60
5.2.3 Technologies d'intelligence artificielle . . . 60
5.2.4 Technologies de données et d'infrastructure . . . 60
5.3 Organisation du code source . . . 61

5.4 Réalisation du backend . . . 62
5.5 Réalisation du frontend . . . 62
5.6 Intégration frontend–backend . . . 63
5.7 Présentation des interfaces . . . 64
5.7.1 Connexion et identité visuelle . . . 64
5.7.2 Consultation sur mobile . . . 65
5.7.3 Vue d'ensemble, offres et candidats . . . 66
5.7.4 Évaluation, résultat et analyse approfondie . . . 70
5.7.5 Classement, comparaison, qualité et paramètres . . . 74
5.8 Parcours fonctionnel de démonstration . . . 80
5.9 Difficultés de réalisation et solutions apportées . . . 80

6 Tests, sécurité, DevOps et déploiement Azure 82

6.1 Stratégie de validation . . . 83
6.2 Environnement et données de test . . . 85
6.3 Tests fonctionnels . . . 85
6.4 Validation du moteur intelligent . . . 86
6.5 Tests de l'API et de la persistance . . . 87
6.6 Validation du rapport PDF . . . 87
6.7 Tests de santé et stabilité des services . . . 87
6.8 Résultats de validation de bout en bout . . . 88
6.9 Validation sur données réelles . . . 88
6.10 Classement de dix-huit CV fictifs . . . 90
6.11 Sécurité applicative . . . 90
6.12 Sécurité réseau et infrastructure . . . 91
6.13 Analyse honnête des limites de sécurité . . . 92
6.14 Traitement des données à caractère personnel . . . 93
6.15 Conteneurisation . . . 94
6.16 Déploiement sur Microsoft Azure . . . 95
6.17 Architecture de déploiement . . . 98
6.18 Exploitation et maîtrise des coûts . . . 99
6.19 Procédures d'exploitation . . . 99
6.20 Bilan du déploiement . . . 99

Conclusion générale 101

Bibliographie 103

Résumé 106

Liste des figures

1.1 Logo de CyberCloud Solutions . . . 3
1.2 Cycle en cascade adapté à JobTest . . . 10

2.1 Diagramme global des cas d'utilisation . . . 19
2.2 Diagramme UC-01 : authentification . . . 20
2.3 Diagramme UC-02 : offres d'emploi . . . 22
2.4 Diagramme UC-03 : candidats . . . 23
2.5 Diagramme UC-04 : évaluation d'un CV . . . 25
2.6 Diagramme UC-05 : analyse approfondie . . . 26
2.7 Diagramme UC-06 : rapport PDF . . . 27
2.8 Diagramme UC-07 : classement . . . 29
2.9 Diagramme UC-08 : historique . . . 30

3.1 Vue fonctionnelle de JobTest . . . 34
3.2 Architecture logique de JobTest . . . 35
3.3 Diagramme de classes de JobTest . . . 37
3.4 Séquence d'authentification . . . 38
3.5 Séquence de création d'une offre . . . 38
3.6 Séquence d'évaluation d'un CV . . . 39
3.7 Séquence d'analyse approfondie . . . 40
3.8 Séquence de génération du rapport PDF . . . 40

4.1 Pipeline de rapprochement de JobTest . . . 43
4.2 Exemple de CV synthétique . . . 50
4.3 Courbes ROC des scores bruts . . . 52
4.4 Score brut de la fusion par classe . . . 53
4.5 Répartition des verdicts . . . 53
4.6 Matrice de confusion des verdicts . . . 54

5.1 Écran de connexion . . . 64
5.2 Écran d'inscription . . . 65
5.3 Affichage en largeur mobile . . . 66
5.4 Vue d'ensemble . . . 67
5.5 Recherche transversale . . . 67

5.6 Liste des offres d'emploi 68
5.7 Création d'une offre d'emploi 68
5.8 Suivi des candidats 69
5.9 Historique des évaluations 69
5.10 Saisie d'une évaluation 70
5.11 Score, verdict et profil de compétences 71
5.12 Sous-scores et adéquation d'expérience 71
5.13 Compétences reconnues et manquantes 72
5.14 Analyse approfondie locale 73
5.15 Verdict limité par le contrôle des preuves 74
5.16 Sélection de l'offre et des CV 75
5.17 Distribution des scores du classement 75
5.18 Podium du classement 76
5.19 Suite du classement 76
5.20 Comparaison des quatre méthodes 77
5.21 Réglage des seuils de décision 77
5.22 *AI Quality Center* : configuration 78
5.23 *AI Quality Center* : qualité des verdicts 79
5.24 *AI Quality Center* : robustesse 79

6.1 Sous-scores de mes CV réels 89
6.2 Architecture de déploiement 92
6.3 Machine virtuelle Azure 96
6.4 Abonnement Azure 97
6.5 Consommation Azure 97
6.6 Ressources Azure 97
6.7 Propriétés de la machine virtuelle 98
6.8 État de la machine virtuelle 98

Liste des tableaux

1.1 Comparaison fonctionnelle des solutions existantes 7

2.1 Acteurs de JobTest 13
2.2 Besoins non fonctionnels 16
2.3 Règles de gestion 18
2.4 UC-01 : s'authentifier 20
2.5 UC-02 : gérer une offre d'emploi 21
2.6 UC-03 : gérer un candidat 22
2.7 UC-04 : évaluer un CV 24
2.8 UC-05 : demander une analyse approfondie 25
2.9 UC-06 : générer un rapport PDF 26
2.10 UC-07 : classer plusieurs candidats 28
2.11 UC-08 : consulter l'historique 29
2.12 Matrice de traçabilité des besoins 30
2.13 Contraintes et hypothèses 32

3.1 Modules applicatifs et responsabilités 36

4.1 Ancrages de la calibration 46
4.2 Bandes de décision par défaut 46
4.3 Qualité du score brut (F1 binaire) 51
4.4 Qualité des verdicts affichés 52
4.5 Effet de la reformulation selon la pondération 54
4.6 Effet du contrôle des preuves 55
4.7 Synthèse des couches 56

5.1 Environnement matériel 58
5.2 Environnement de réalisation 59
5.3 Choix techniques 61
5.4 Difficultés et solutions 80

6.1 Niveaux de validation 84
6.2 Contrôles fonctionnels 85
6.3 Mes CV face à une offre réelle 89

6.4 Services conteneurisés 94
6.5 Configuration de la machine virtuelle 96
6.6 Synthèse des résultats et limites 100

Liste des abréviations

- **API** = Interface de programmation applicative
- **ASGI** = Interface asynchrone entre un serveur Web et une application Python
- **ATS** = Système de suivi des candidatures
- **AUC** = Aire sous la courbe
- **BERT** = Modèle de représentation contextuelle bidirectionnelle du texte
- **CPU** = Processeur central
- **CRUD** = Créer, lire, mettre à jour et supprimer
- **CSS** = Feuilles de style en cascade
- **CSV** = Fichier texte de valeurs séparées par des virgules
- **CV** = Curriculum vitae
- **DOCX** = Format de document Word fondé sur XML
- **DevOps** = Pratiques associant développement et exploitation
- **HTTP** = Protocole de transfert hypertexte
- **HTTPS** = Protocole de transfert hypertexte sécurisé par TLS
- **IA** = Intelligence artificielle
- **IP** = Protocole Internet, et par extension l'adresse qu'il attribue
- **JSON** = Format d'échange de données textuel
- **JWT** = JSON Web Token
- **LLM** = Grand modèle de langage
- **LTS** = Version bénéficiant d'un support à long terme
- **NSG** = Network Security Group
- **OAuth2** = Protocole de délégation d'autorisation, utilisé ici pour le formulaire de connexion
- **ORM** = Correspondance objet–relationnel

- **OWASP** = Open Worldwide Application Security Project
- **PDF** = Portable Document Format
- **RAM** = Mémoire vive
- **RH** = Ressources humaines
- **ROC** = Courbe caractéristique de fonctionnement du récepteur
- **SQL** = Langage de requête structuré pour bases relationnelles
- **SSD** = Disque à mémoire flash
- **SSH** = Protocole d'accès distant sécurisé à une machine
- **TCP** = Protocole de transport en mode connecté
- **TF–IDF** = Term Frequency–Inverse Document Frequency
- **TLS** = Protocole de chiffrement des échanges réseau
- **UFW** = Pare-feu en ligne de commande d'Ubuntu
- **UML** = Unified Modeling Language
- **URL** = Adresse d'une ressource sur le Web
- **VM** = Machine virtuelle
- **WAF** = Pare-feu applicatif Web
- **XDR** = Détection et réponse étendues aux incidents de sécurité
- **XML** = Langage de balisage extensible
- **XSS** = Injection de script dans une page Web

Introduction générale

Le recrutement exige d'examiner des CV nombreux et hétérogènes au regard d'offres dont les exigences sont souvent formulées de manière libre. Cette première lecture mobilise du temps, tout en devant rester justifiable et attentive au contexte de chaque candidature. Une assistance logicielle peut organiser les informations utiles, mais elle ne peut pas se substituer au jugement du recruteur ni produire seule une décision d'embauche.

Ce mémoire présente *JobTest*, réalisé dans le cadre du stage de PFE chez CyberCloud Solutions. Le projet répond au besoin d'une aide à la présélection qui rapproche un CV et une offre, rend visibles les éléments de correspondance et conserve les résultats d'évaluation. La plateforme permet au recruteur de gérer ses offres et ses candidats, de lancer des évaluations sur du texte ou des documents, puis de consulter un résultat composé d'un score, de sous-scores, de compétences détectées, d'écarts et de recommandations.

La solution associe une couche déterministe fondée sur TF–IDF, la similarité sémantique et les compétences, à une politique de décision explicite. Une analyse qualitative locale peut compléter la restitution, sans modifier le verdict officiel. L'application s'appuie sur un frontend Next.js, une API FastAPI, une persistance relationnelle et un export PDF. Les validations documentées couvrent les parcours principaux, un benchmark sur données synthétiques et un déploiement de démonstration conteneurisé sur Microsoft Azure.

Le premier chapitre présente le contexte, l'organisme d'accueil, la problématique et la démarche. Le deuxième précise les besoins et les règles de gestion. Le troisième expose la conception fonctionnelle et architecturale. Le quatrième décrit le moteur d'évaluation et son protocole expérimental. Le cinquième présente la réalisation de la plateforme, tandis que le sixième traite des tests, de la sécurité, de l'industrialisation et du déploiement. La conclusion générale synthétise les apports, les limites et les perspectives de JobTest.

Chapitre 1

Contexte général et étude préalable

Plan

1 Présentation de l'organisme d'accueil 3
2 Contexte du projet 4
3 Étude de l'existant 5
4 Critique de l'existant 6
5 Problématique 7
6 Solution proposée : JobTest 8
7 Objectifs du projet 8
8 Méthodologie de conduite du projet 9

Introduction

Le présent chapitre fixe le cadre organisationnel, fonctionnel et méthodologique du projet *JobTest*. Il situe d'abord le stage au sein de CyberCloud Solutions, puis examine le problème de présélection de CV et les réponses actuellement proposées par les outils numériques. Cette étude préalable sert à délimiter précisément la contribution de JobTest : une aide à l'analyse et à la comparaison de candidatures, et non un mécanisme autonome de recrutement. Le chapitre formule ensuite la problématique, décrit la solution retenue, ses objectifs ainsi que la démarche de conduite du projet.

1.1 Présentation de l'organisme d'accueil

1.1.1 Présentation de CyberCloud Solutions

CyberCloud Solutions est une entreprise de services et de conseil en informatique. Sa page publique la situe dans le Grand Tunis, indique un effectif de deux à dix personnes et présente une organisation tournée vers l'accompagnement technique, le conseil et la transformation numérique [1]. Dans ce contexte, l'entreprise peut réunir des besoins variés autour des environnements numériques sans que cela implique une spécialisation unique sur le recrutement.

Son positionnement associe prestation de services, assistance et conseil. La flexibilité du travail à distance ou hybride est cohérente avec le cadre du stage décrit dans ce mémoire : elle permet de structurer un travail individuel autour de livrables partagés et vérifiables, sans présumer d'une organisation interne, d'un rythme de réunion ou d'une répartition de services qui ne sont pas documentés publiquement.

La figure 1.1 reproduit le logo employé par l'entreprise sur sa page officielle. Il est utilisé ici uniquement pour identifier l'organisme d'accueil.

Figure 1.1: Logo de CyberCloud Solutions [1]

1.1.2 Activités et domaines d'expertise

Les activités publiées par CyberCloud Solutions peuvent être regroupées sans les confondre avec le périmètre de JobTest. Un premier ensemble concerne le conseil, l'assistance et la transformation numérique. Un deuxième porte sur l'infrastructure : équipements informatiques et bureautiques, serveurs, stockage, postes de travail, imprimantes et logiciels professionnels. La présentation publique mentionne également des solutions de sécurité Internet, de *network forensics* et d'interception légale [1].

Les spécialités affichées complètent ce panorama : cybersécurité, XDR, réseaux, serveurs, cloud, DevOps, logiciels et intelligence. Elles montrent un environnement technique pluridisciplinaire ; elles ne signifient pas que chacun de ces domaines a été directement mobilisé pour la plateforme. JobTest relève plus précisément de l'ingénierie d'une application d'aide au recrutement, à l'intersection du traitement de texte, du développement web, de la persistance des données et de l'évaluation de modèles.

1.1.3 Cadre du stage

J'ai réalisé ce projet dans le cadre du Mastère Professionnel en Applications Web Intelligentes de l'Institut Supérieur d'Informatique du Kef, relevant de l'Université de Jendouba. Le sujet, proposé par CyberCloud Solutions, est : *conception et développement d'une plateforme intelligente d'aide au recrutement et de mise en correspondance entre CV et offres d'emploi.*

Mon intervention est individuelle, en qualité de stagiaire PFE, dans un mode principalement distant ou hybride. L'encadrement professionnel est assuré par Dr. Trabelsi Emna. Ce cadre précise l'origine professionnelle du besoin tout en conservant le caractère académique du travail : les choix techniques, les résultats et leurs limites doivent pouvoir être explicités et contrôlés.

1.2 Contexte du projet

Une candidature condense des informations de natures différentes : expériences, formations, compétences déclarées, réalisations, certifications et éléments libres. Les offres d'emploi présentent elles aussi des exigences hétérogènes, parfois sous forme de listes, parfois dans des descriptions rédigées. La généralisation des candidatures en ligne a accru le volume à examiner et rend la présélection plus difficile à organiser manuellement [2].

L'intelligence artificielle désigne notamment des systèmes capables de produire des prédictions, recommandations ou décisions pour des objectifs définis par des humains [3]. Dans le cas du recrutement, cette capacité ne retire ni la responsabilité ni le jugement professionnel du recruteur. Les travaux sur les outils algorithmiques de ressources humaines soulignent à la fois leur intérêt pour traiter de nombreux dossiers et les risques de biais, d'opacité ou de mauvaise interprétation qui

justifient une supervision humaine [2], [4].

Une recherche par mots-clés est utile pour repérer des termes explicitement demandés, mais elle ne traite pas toujours les formulations proches, les synonymes ou les compétences décrites indirectement. La représentation contextuelle des textes a précisément été développée pour mieux exploiter le contexte linguistique [5]. Des représentations de phrases comparables par similarité cosinus permettent, par exemple, de rapprocher deux textes sur une base sémantique plutôt que de compter seulement des mots identiques [6]. Pour la présélection, l'enjeu n'est donc pas de remplacer le recruteur, mais de fournir un résultat reproductible, décomposé et intelligible : ce qui concorde, ce qui manque et ce qui a conduit à une recommandation.

1.3 Étude de l'existant

1.3.1 Processus manuel de présélection

Dans un processus manuel, le recruteur lit l'offre, définit ses critères, ouvre les CV, repère les expériences et compétences pertinentes, puis conserve ou écarte des candidatures avant les étapes suivantes. Cette séquence laisse place à une appréciation qualitative indispensable, notamment pour interpréter un parcours atypique ou contextualiser une expérience.

Elle devient cependant coûteuse à répéter lorsque les documents sont nombreux et de formats disparates. Elle rend également plus difficile la traçabilité du raisonnement appliqué à chaque dossier. La littérature consacrée aux décisions algorithmiques en ressources humaines rappelle que la recherche d'efficacité ne constitue pas, à elle seule, une garantie d'équité : la définition des critères, la qualité des données et le contrôle humain restent déterminants [4]. JobTest ne cherche donc pas à transformer une préconisation en décision d'embauche ; il structure l'examen initial et rend visibles ses éléments de calcul.

1.3.2 Approches lexicales et ATS traditionnels

Les systèmes de suivi de candidatures (*Applicant Tracking Systems*, ou ATS) organisent habituellement les offres, les profils candidats, les étapes de pipeline et les évaluations. Dans ce type d'environnement, la recherche peut s'appuyer sur des mots, des étiquettes, des filtres ou des champs structurés. Cette approche est efficace quand l'exigence est explicitement formulée et que la terminologie est stable. Elle devient moins robuste lorsqu'une compétence est exprimée par une paraphrase, lorsque le CV est multilingue ou lorsqu'une expérience pertinente ne reprend pas le vocabulaire exact de l'offre.

L'analyse textuelle par termes demeure ainsi un signal, non une preuve complète d'adéquation. Les revues sur le recrutement assisté indiquent que les outils de sélection vont aujourd'hui de la recherche de mots à des traitements cherchant des correspondances plus contextuelles ; elles invitent

également à distinguer le filtrage technique de la validité d'une décision de sélection [4]. La comparaison sémantique et l'analyse explicite des compétences constituent donc des compléments pertinents, à condition que leurs résultats restent contrôlables.

1.3.3 Solutions numériques étudiées

J'ai retenu trois solutions pour une comparaison ciblée, à partir de leur documentation officielle. LinkedIn Recruiter propose une recherche assistée par IA à partir d'une demande en langage naturel ; selon l'offre et la langue, elle peut produire des recommandations et une synthèse expliquant l'adéquation affichée sur la carte d'un candidat [7], [8]. La documentation étudiée ne permet pas de conclure à la présence d'une analyse détaillée des lacunes de compétences ni d'un export de rapport de correspondance CV/offre.

Workable centralise les profils, les CV et le pipeline. Sa vue de candidature permet notamment une recherche par mots-clés ou recherche avancée, l'affichage des compétences et le tri par meilleure évaluation ou meilleur appariement fondé sur les évaluations IA [9]. Sa documentation de sécurité mentionne des dispositifs de protection des données, dont le chiffrement en transit, des mécanismes liés aux droits des candidats et des contrôles de conformité [10]. Ces informations décrivent la plateforme Workable, pas le niveau de protection d'une installation particulière.

Greenhouse Recruiting documente une fonction *Talent Matching* qui compare les CV à une calibration définie et pondérée pour le poste. Il propose aussi des *scorecards* pour évaluer les candidats selon des critères préparés, ainsi que des rapports candidats exportables au format tableur [11], [12], [13]. Les sources consultées ne documentent pas un diagnostic autonome de lacunes de compétences comparable à celui visé par JobTest.

1.3.4 Synthèse comparative

Le tableau 1.1 met en regard des capacités directement documentées. Il ne mesure ni la qualité réelle de chaque solution, ni leur couverture complète : la mention *non documenté* signifie seulement que je n'ai trouvé aucune preuve explicite dans les pages officielles consultées. Les informations sur JobTest proviennent du code, des rapports techniques du dépôt et du déploiement vérifié du projet.

1.4 Critique de l'existant

Les plateformes étudiées couvrent des besoins opérationnels importants : recherche, pipeline, évaluation structurée, gestion de candidats et, selon les produits, fonctions d'IA. Elles ne sont donc pas des alternatives faibles à écarter ; elles répondent à des contextes organisationnels plus larges que le périmètre du PFE. La comparaison révèle surtout un besoin de cohérence entre l'analyse du texte,

Critère	LinkedIn Recruiter	Workable	Greenhouse Recruiting	JobTest
Correspondance CV/offre	Recherche IA et recommandations à partir d'une demande ; synthèse de fit selon disponibilité	Tri *best match* issu d'évaluations IA ; CV dans le pipeline	Talent Matching : CV comparé à une calibration pondérée	Analyse hybride : lexical, sémantique et compétences
Éléments explicatifs	Synthèse IA de fit pour certains accès ; granularité non documentée	Évaluation et score visibles ; décomposition d'IA non documentée	Scorecards structurées et critères d'entretien	Scores composantes, compétences, écarts et recommandations
Analyse des écarts de compétences	Non documenté	Non documenté	Non documenté	Compétences correspondantes, manquantes et critiques
Classement et export	Résultats de recherche ; export de rapport d'appariement non documenté	Tri de candidats ; export de rapport d'appariement non documenté	Rapport candidat exportable en XLS	Classement multi-CV, historique et rapport PDF
Contrôle et données	Modèle de déploiement/confidentialité non documenté dans les sources consultées	Documentation sécurité et confidentialité disponible	Modèle de déploiement/confidentialité non documenté dans les sources consultées	Persistance limitée au recruteur ; déploiement PFE sur VM Azure

Tableau 1.1: Comparaison fonctionnelle des solutions existantes [8], [9], [10], [11], [12], [13]

le contrôle de la règle de décision et l'explication fournie au recruteur.

JobTest adresse ce besoin par une chaîne unique : une analyse hybride associe un signal lexical, un signal sémantique et une analyse des compétences ; un moteur de décision central applique la même politique à toutes les interfaces ; les composantes, les compétences manquantes et des recommandations sont restituées avec le résultat. Les évaluations enregistrent également des métadonnées de reproductibilité, notamment la configuration de décision et les poids utilisés. Une analyse qualitative facultative, exécutée localement via Ollama, intervient après le score statistique et ne peut ni le modifier ni annuler la recommandation.

Cette proposition comporte des limites qu'il faut exposer dès maintenant. L'évaluation quantitative repose sur un corpus synthétique ; les essais que j'ai menés avec mes propres CV face à des offres réelles sont fonctionnels et ne disposent pas de jugements de référence. Ni l'une ni les autres ne constituent une preuve de performance sur des décisions d'embauche réelles. L'inférence sémantique et la petite couche LLM locale sont exécutées sur CPU, ce qui introduit une latence perceptible pour l'analyse approfondie. Le déploiement Azure vérifié est adapté à une démonstration de PFE : il s'agit d'une instance unique, servie en HTTPS, sans dispositif de haute disponibilité. Enfin, le petit modèle local peut produire une analyse superficielle ou inexacte ; son résultat est strictement consultatif. Ces limites renforcent la nécessité d'un recruteur décisionnaire.

1.5 Problématique

La problématique du projet peut être formulée ainsi :

> *Comment fournir au recruteur, à partir d'un CV et d'une offre d'emploi, une évaluation hybride de la proximité sémantique et des compétences qui soit explicable, reproductible et*

persistante, tout en préservant une analyse qualitative consciente de ses limites et sans déléguer la décision d'embauche à l'outil ?

Cette question associe quatre exigences. Il faut traiter le contenu hétérogène des documents, rendre la recommandation vérifiable par ses composantes, conserver le contexte nécessaire à la reproductibilité et encadrer l'usage d'une analyse générative par une approche respectueuse des données et du rôle du recruteur.

1.6 Solution proposée : JobTest

JobTest est une plateforme web d'aide au recrutement destinée au recruteur ou spécialiste RH. Elle permet de créer et gérer des offres, de saisir le texte d'un CV ou de téléverser un document PDF ou DOCX, puis de lancer une évaluation contre une offre sélectionnée. Le résultat comprend une recommandation `HIRE`, `CONSIDER` ou `REJECT`, un score global, une décomposition des composantes et une présentation des compétences détectées, correspondantes ou absentes.

La plateforme complète l'analyse individuelle par la gestion des candidats et de l'historique, un classement de plusieurs CV pour une même offre, ainsi qu'un rapport PDF. Elle met aussi à disposition une comparaison de modèles et un *AI Quality Center* qui expose un benchmark hors ligne avec sa provenance. Ces éléments distinguent une aide au recrutement instrumentée d'une simple recherche textuelle.

Une analyse approfondie optionnelle est disponible via un modèle local exécuté par Ollama. Elle produit une lecture qualitative, des points d'attention, des recommandations et des questions d'entretien. Elle est appelée après la couche statistique ; elle est donc additionnelle. La décision et le score officiels restent issus du moteur déterministe de la plateforme. Les données métier sont isolées par recruteur, et les résultats d'analyse sont conservés afin de pouvoir les retrouver dans le workflow.

1.7 Objectifs du projet

1.7.1 Objectif général

L'objectif général est de concevoir et développer une plateforme d'aide à la présélection qui aide un recruteur à comparer des CV à une offre de façon structurée, explicable et reproductible, sans automatiser la décision d'embauche.

1.7.2 Objectifs spécifiques

Les objectifs spécifiques retenus sont les suivants :

- mettre en oeuvre un workflow de recrutement comprenant l'authentification, la gestion des

offres et la gestion des candidats ;

- accepter un CV sous forme de texte saisi ou de document PDF/DOCX ;
- associer des signaux lexicaux, sémantiques et orientés compétences dans une même évaluation ;
- appliquer une politique de décision cohérente pour les recommandations `HIRE`, `CONSIDER` et `REJECT` ;
- exposer les composantes du score, les compétences correspondantes, les écarts et des recommandations d'amélioration ;
- enregistrer les évaluations, leurs paramètres utiles et l'historique associé au recruteur ;
- générer un rapport PDF et proposer un classement de plusieurs candidatures pour une offre ;
- comparer les méthodes disponibles et afficher les résultats d'une évaluation hors ligne avec leur provenance ;
- intégrer des mesures de sécurité applicative, notamment l'authentification et le cloisonnement des données par recruteur ;
- déployer une version démontrable de la solution avec le frontend, l'API, la base de données et la couche d'analyse locale.

1.8 Méthodologie de conduite du projet

1.8.1 Justification du modèle en cascade

Le modèle en cascade organise un projet logiciel en séquences identifiables, depuis l'expression du besoin jusqu'à la validation. Dans son article fondateur, Royce insiste sur la documentation et sur la nécessité de retours contrôlés lorsque les résultats d'une phase révèlent un problème [14]. Il ne doit donc pas être interprété comme une interdiction absolue de corriger l'implémentation.

Pour JobTest, ce cadre convient à un PFE dont les livrables attendus peuvent être stabilisés : besoins, conception, réalisation, tests et déploiement. Il fournit un fil de lecture clair pour le mémoire et la démonstration. Des corrections techniques et des retours de validation ont néanmoins été réalisés à l'intérieur des phases d'implémentation et de validation, notamment sur la cohérence de la décision, l'analyse des compétences, la qualité du benchmark et la sécurisation des accès. Le modèle en cascade décrit ici la progression principale ; il ne masque pas ces boucles de correction.

1.8.2 Phases appliquées à JobTest

La figure 1.2 présente la déclinaison de cette démarche pour JobTest. Chaque étape produit un résultat qui alimente la suivante, tandis que la phase de validation peut conduire à corriger le livrable concerné.

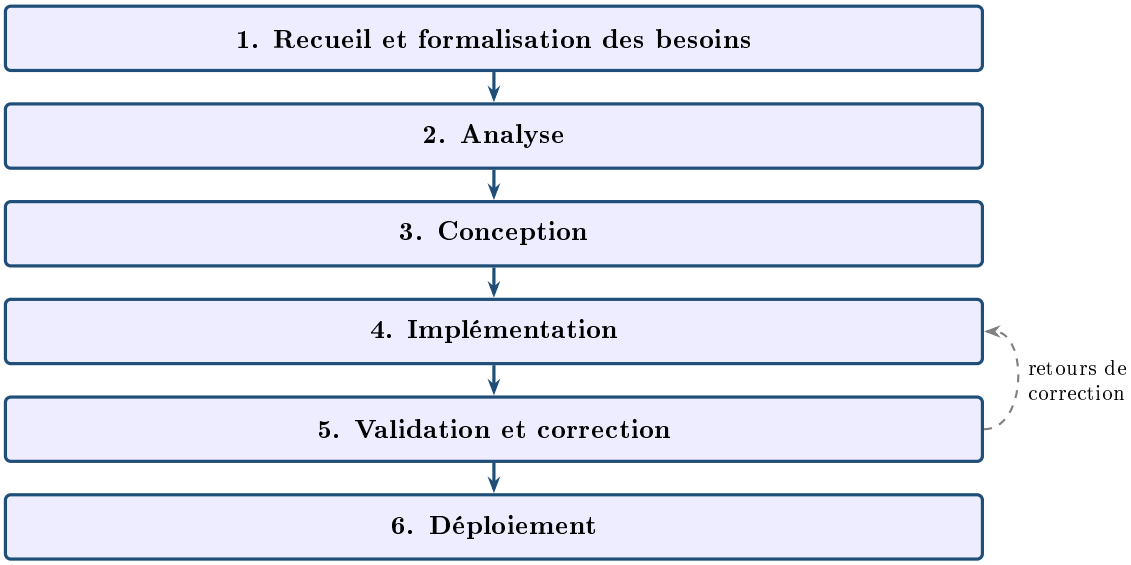


Figure 1.2: Cycle en cascade adapté à JobTest

Le recueil et la formalisation des besoins ont permis de préciser le rôle du recruteur, les entrées CV/offre et les attentes d'explication. L'analyse a ensuite structuré les cas d'usage, les données manipulées et les contraintes de confidentialité. La conception a défini la séparation entre interface web, API, moteur d'appariement, décision, persistance et analyse qualitative optionnelle.

L'implémentation a produit l'application web, les services de traitement des CV, la gestion des offres et des candidats, l'export PDF et les écrans d'évaluation. La validation a porté sur les parcours fonctionnels, le classement, les rapports, les résultats d'évaluation et les corrections du moteur. Enfin, le déploiement a rassemblé Next.js, FastAPI, PostgreSQL, Ollama et un proxy inverse HTTPS dans une configuration Docker Compose sur une machine virtuelle Azure. Cette dernière étape reste une livraison de démonstration, avec les limites d'exploitation déjà indiquées.

Conclusion

Ce chapitre a présenté CyberCloud Solutions, le cadre individuel du stage et les contraintes du problème de présélection. L'étude de l'existant a montré l'intérêt des ATS et de l'assistance par IA, tout en mettant en évidence l'importance de l'explicabilité, de la supervision humaine et de la prudence face aux résultats automatisés. JobTest répond à ce besoin par une analyse hybride, une décision cohérente et une restitution orientée recruteur, tout en assumant les limites de ses données, de ses modèles et de son déploiement. Le chapitre suivant, *Analyse et spécification des besoins*, détaillera les acteurs, les besoins fonctionnels et non fonctionnels ainsi que la spécification du système.

Chapitre 2

Analyse et spécification des besoins

Plan

1 Périmètre fonctionnel de JobTest 12

2 Identification des acteurs et parties prenantes 12

3 Spécification des besoins fonctionnels 13

4 Spécification des besoins non fonctionnels 15

5 Règles de gestion 18

6 Diagramme global des cas d'utilisation 18

7 Raffinement et description textuelle des cas d'utilisation 19

8 Matrice de traçabilité 30

9 Contraintes et hypothèses 31

Introduction

L'analyse des besoins transforme le périmètre présenté au chapitre précédent en attentes vérifiables, cas d'utilisation et règles de gestion. Elle s'appuie ici sur le comportement du dépôt JobTest : interface Next.js, routes FastAPI effectivement sollicitées, persistance et moteur de décision. L'objectif n'est pas de décrire une plateforme RH générique, mais de fixer le contrat fonctionnel de l'application démontrable. Après la délimitation du système et l'identification de ses parties prenantes, le chapitre spécifie les besoins, les contraintes de qualité, les règles de gestion, les cas d'utilisation et leur traçabilité.

2.1 Périmètre fonctionnel de JobTest

JobTest est une application d'aide à la présélection destinée au recruteur. Elle organise des offres et des dossiers candidats, puis fournit une évaluation hybride entre un CV et une offre. Le résultat associe une recommandation officielle, ses composantes, les compétences détectées ou manquantes et des éléments d'explication. L'analyse qualitative produite par le LLM local complète cette restitution sans participer à la décision officielle.

Le périmètre couvert comprend l'inscription et la session du recruteur, la gestion des offres et des candidats, l'évaluation d'un texte ou d'un document PDF/DOCX, la consultation des résultats et de l'historique, le classement multi-CV, l'export PDF, la comparaison ponctuelle des méthodes, le centre de qualité hors ligne et le réglage de seuils pour la session. Les données opérationnelles et les résultats persistés sont isolés par recruteur.

Sont hors périmètre une décision d'embauche autonome, un portail candidat, la conduite des entretiens et un apprentissage sur des résultats réels d'embauche. JobTest ne remplace donc pas l'appréciation du recruteur ni les étapes humaines de sélection. Son benchmark repose sur des données synthétiques. Enfin, le déploiement Azure validé est une démonstration PFE sur une instance sans haute disponibilité permanente ; il ne constitue pas une offre de production 24/7.

2.2 Identification des acteurs et parties prenantes

2.2.1 Visiteur non authentifié

Avant l'authentification, le visiteur peut créer un compte recruteur ou se connecter. Je n'ai intégré à l'application aucun parcours public de consultation d'offres, de dépôt de candidature, de réinitialisation de mot de passe, de vérification d'e-mail ou de connexion sociale.

2.2.2 Recruteur authentifié

Le recruteur est l'acteur opérationnel principal. Une fois la session JWT établie, il consulte le tableau de bord et gère ses propres offres, candidats et résultats. Il peut évaluer un CV, consulter le résultat explicable, demander l'analyse approfondie facultative, exporter un rapport, classer plusieurs CV, consulter l'historique, comparer les méthodes, visualiser la qualité hors ligne et modifier les seuils actifs de sa session. Les contrôles d'appartenance empêchent l'accès aux objets d'un autre recruteur.

2.2.3 Candidat en tant qu'entité métier

Le candidat est le sujet d'une évaluation et une entité persistée : identité facultative, coordonnées, statut de pipeline, CV et résultats associés. Il n'est pas un acteur logiciel interactif. La version étudiée ne fournit ni compte candidat ni écran candidat ni soumission publique de CV. Cette séparation évite de confondre l'objet métier évalué avec l'utilisateur autorisé à utiliser l'application.

2.2.4 Partie prenante technique

Une personne chargée de maintenance ou d'assurance qualité peut exécuter le benchmark hors ligne, exporter ses figures ou intervenir sur l'environnement. Cette partie prenante est technique et secondaire : aucun portail administrateur, rôle d'administration distinct ni parcours d'authentification administrateur n'est prouvé dans l'interface opérationnelle. Elle ne figure donc pas comme acteur du diagramme global.

Le tableau 2.1 récapitule ces acteurs et situe chacun d'eux par rapport au parcours métier.

Élément	**Type**	**Rôle**	**Accès au système**
Visiteur non authentifié	Acteur	Crée un compte ou amorce une session	Écran d'accès, sans données métier
Recruteur authentifié	Acteur	Gère et évalue ses dossiers	Toutes les fonctions métier après JWT
Candidat	Entité métier	Porte le CV, le pipeline et les résultats	Aucun accès interactif
Maintenance / QA	Partie prenante technique	Produit ou contrôle le benchmark hors ligne	Hors parcours métier courant

Tableau 2.1: Acteurs de JobTest

2.3 Spécification des besoins fonctionnels

Les besoins sont présentés par domaine afin de décrire le parcours réellement disponible, sans confondre les fonctions du recruteur avec une décision autonome de recrutement. Chaque liste énonce les capacités attendues ; le paragraphe qui la suit précise les règles et critères permettant d'en constater la réalisation.

2.3.1 Authentification du recruteur

- Créer un compte recruteur avec nom, e-mail et mot de passe, en refusant un e-mail déjà utilisé.
- Se connecter, consulter son profil courant et fermer la session côté interface.

L'inscription crée uniquement un recruteur. La connexion utilise le formulaire OAuth2 de l'API, qui retourne un jeton bearer. L'interface conserve ce jeton pour appeler `/auth/me` et les routes protégées ; la déconnexion l'efface côté client. Aucun besoin de réinitialisation de mot de passe ou d'activation n'est spécifié car ces parcours ne sont pas présents.

2.3.2 Gestion des offres et des candidats

- Créer, consulter, modifier, clôturer ou rouvrir ses propres offres ; le code est attribué automatiquement.
- Supprimer une offre seulement lorsqu'aucune évaluation ne lui est liée ; une offre évaluée peut être clôturée.
- Consulter les candidats du recruteur, mettre à jour leurs informations et leur statut de pipeline, puis supprimer un dossier sans résultat associé.

Les offres sont créées avec un titre, une description, des compétences requises facultatives et un statut. Le code est calculé automatiquement. Toutes les opérations vérifient le recruteur propriétaire. Une offre liée à des évaluations ne peut pas être supprimée, afin de préserver la cohérence de l'historique. L'évaluation crée ou réutilise le dossier candidat lorsque les informations disponibles le permettent ; la page candidats sert ensuite à la consultation et à la mise à jour. Les fonctions exposées dans l'interface courante sont la modification et la suppression conditionnelle. L'API historique comporte une anonymisation, mais elle n'est pas présentée comme un parcours du frontend Next.js faute d'action correspondante vérifiée.

2.3.3 Évaluation et restitution

- Évaluer un texte CV ou un document PDF/DOCX par rapport à une offre possédée.
- Consulter le verdict, le score, les composantes, les compétences, les écarts et les recommandations enregistrés.
- Demander séparément une analyse approfondie locale ; son indisponibilité ne modifie pas le verdict officiel.
- Générer et télécharger le rapport PDF d'une évaluation.

Le recruteur sélectionne une offre qu'il possède, saisit éventuellement identité et e-mail, puis transmet du texte ou un fichier. La couche statistique lance l'appariement choisi, le *DecisionEngine* applique les seuils actifs et le service persiste un résultat. Le résultat visible comprend le score global, les sous-scores, la décision, l'explication, les compétences et les écarts. L'analyse approfondie est une seconde demande sur une évaluation déjà persistée. Si Ollama est désactivé, indisponible ou en erreur, l'état est retourné sans bloquer ni altérer le verdict statistique. Seule une enveloppe avec état `ok` est sauvegardée. Le rapport PDF, enfin, réutilise les données déjà calculées et n'ouvre pas un nouveau calcul de correspondance.

2.3.4 Historique, classement et réglages

- Consulter l'historique et le détail des évaluations appartenant au recruteur.
- Classer plusieurs CV pour une offre, puis exporter le classement courant au format CSV.
- Comparer ponctuellement les méthodes, consulter les résultats du benchmark hors ligne et régler les seuils actifs de la session.

Le classement accepte plusieurs fichiers pour une seule offre et lance l'évaluation de chaque fichier avant de les trier par score décroissant ; l'export CSV est généré par le frontend. L'historique expose les résultats persistés du recruteur et leur détail. *AI Insights* exécute une comparaison directe non persistante, tandis que *AI Quality Center* présente un benchmark hors ligne : il ne doit pas être interprété comme une mesure d'exactitude de production en temps réel. Les seuils Strong Fit et Potential Fit sont configurés dans le navigateur et accompagnent les évaluations ultérieures de cette session ; ils doivent respecter `potential < strong`, et les résultats antérieurs gardent les seuils enregistrés.

2.4 Spécification des besoins non fonctionnels

Les qualités attendues sont structurées à partir du modèle de qualité produit ISO/IEC 25010, utilisé comme grille pour spécifier et évaluer des caractéristiques de logiciel [15]. Les indicateurs suivants expriment des critères vérifiables sur la version étudiée, sans prétendre démontrer une certification.

Le détail figure dans le tableau 2.2, où chaque exigence est accompagnée de l'élément observable qui permet d'en juger.

Tableau 2.2: Besoins non fonctionnels

Identifiant	**Catégorie**	**Exigence**	**Indicateur ou critère de vérification**
BNF-SEC-01	Sécurité	Les opérations métier exigent un utilisateur authentifié et respectent l'appartenance recruteur.	Les routes `/ui` appellent `get_current_user` et filtrent par `recruiter_id`.
BNF-SEC-02	Sécurité	Les mots de passe ne sont pas persistés en clair et la session a une durée limitée.	Le compte stocke `hashed_password` ; la création de JWT applique l'expiration configurée.
BNF-SEC-03	Sécurité	Les services internes ne sont pas directement publiés dans la topologie Docker validée.	Seul le proxy inverse (Caddy) expose les ports HTTP et HTTPS ; API, base et Ollama restent sur le réseau Compose.
BNF-PRIV-01	Confidentialité	Les dossiers et résultats sont isolés par recruteur.	Les listes, détails et vérifications d'objets incluent l'identifiant du recruteur.
BNF-PRIV-02	Confidentialité	La suppression est encadrée pour préserver l'historique ; l'analyse LLM est locale au service Ollama.	Les gardes 409 empêchent la suppression avec résultat ; le LLM est joint sur réseau interne.
BNF-EXPL-01	Explicabilité	Le verdict affiché et persisté provient de la même politique de décision.	`DecisionEngine` produit la décision ; les composantes, écarts et recommandations sont affichés avec le résultat.
BNF-REPRO-01	Reproductibilité	Une évaluation conserve les paramètres utiles à son interprétation.	`MatchResult` porte méthode, version, poids, seuils et métadonnées de décision lorsqu'ils sont fournis.

Identifiant	**Catégorie**	**Exigence**	**Indicateur ou critère de vérification**
BNF-PERF-01	Performance	La couche statistique reste le flux de verdict ; la couche LLM ne le bloque pas.	La Deep Analysis est appelée après l'affichage de Layer 1. Son exécution CPU sur la VM Azure a été observée autour de 54 secondes.
BNF-RES-01	Fiabilité	L'indisponibilité d'Ollama ne corrompt ni ne retarde le verdict Layer 1.	La route renvoie un état `disabled`, `unavailable` ou `error` et ne modifie le résultat officiel.
BNF-ERG-01	Ergonomie	L'interface doit fournir retours de chargement, d'absence et d'erreur, avec une mise en page réactive.	Les composants d'états et les vues responsive sont présents ; les animations consultent `prefers-reduced-motion`.
BNF-PORT-01	Portabilité	La démonstration doit pouvoir être exécutée avec des services séparés et configurables.	Docker Compose assemble Next.js, FastAPI, PostgreSQL, Ollama et le proxy inverse ; les paramètres viennent de l'environnement.
BNF-MAINT-01	Maintenabilité	Les responsabilités d'interface, API, appariement, décision et persistance restent séparées.	Les routes `/ui`, `matching.service`, `decisioning` et les modèles ORM sont des modules distincts.

La sécurité et la confidentialité décrites sont des exigences vérifiées localement, non une affirmation de sécurité complète : aucune conformité légale ni audit externe n'est déduite. La limite de latence de la couche LLM et le caractère démonstratif du déploiement doivent rester visibles lors de l'utilisation de la plateforme.

2.5 Règles de gestion

Le tableau 2.3 énonce les règles que toute évolution ultérieure de la plateforme devra continuer de respecter.

Tableau 2.3: Règles de gestion

ID	Règle de gestion	Justification / effet
RG-01	Un recruteur accède uniquement à ses offres, candidats et évaluations.	Cloisonne les données métier et évite l'accès inter-recruteur.
RG-02	Une offre possédant des évaluations ne peut pas être supprimée définitivement.	Préserve les résultats ; l'offre doit être clôturée.
RG-03	Un candidat possédant des évaluations ne peut pas être supprimé définitivement.	Préserve l'historique ; le statut de pipeline reste modifiable.
RG-04	Les bandes de décision utilisent les seuils Strong Fit et Potential Fit actifs.	La recommandation est cohérente avec la configuration transmise.
RG-05	Le seuil Potential Fit doit être strictement inférieur au seuil Strong Fit.	Évite des bandes de décision ambiguës ou inversées.
RG-06	La couche LLM ne modifie jamais le score ni la décision officielle Layer 1.	L'analyse approfondie demeure consultative et additive.
RG-07	La comparaison *AI Insights* ne persiste pas de `MatchResult`.	Une comparaison exploratoire ne pollue pas l'historique métier.
RG-08	Une Deep Analysis est persistée uniquement si son état est `ok`.	Les états d'indisponibilité ou d'erreur ne sont pas assimilés à une analyse valide.
RG-09	Les résultats historiques conservent leurs métadonnées de score et de seuil.	Les modifications futures de session ne réécrivent pas les évaluations passées.
RG-10	Les téléversements d'évaluation acceptent seulement PDF et DOCX ; le texte extrait doit être exploitable.	Les formats non pris en charge ou sans texte lisible sont rejetés.

2.6 Diagramme global des cas d'utilisation

La notation UML est employée pour séparer les acteurs externes de la frontière du système et représenter les objectifs utilisateur ; UML 2.5.1 est la spécification normative retenue pour ce vocabulaire [16]. La figure 2.1 ne représente donc ni PostgreSQL, ni Ollama, ni FastAPI ou Next.js comme acteurs : ce sont des composants internes.

Les deux acteurs sont placés hors de la frontière du système. Le visiteur reste limité à la

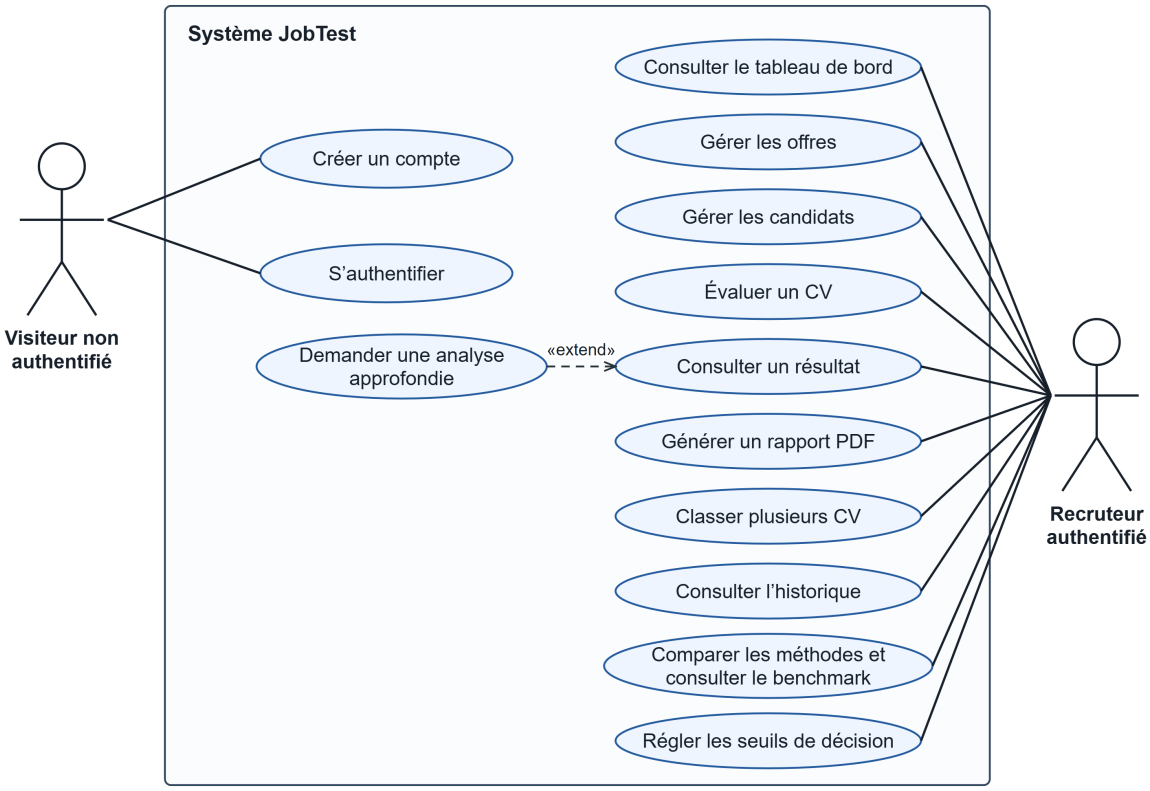


Figure 2.1: Diagramme global des cas d'utilisation

création de compte et à l'authentification. Le recruteur porte tous les cas opérationnels, y compris la comparaison des méthodes, la consultation du benchmark et le réglage des seuils de décision. L'analyse approfondie est représentée comme une extension facultative de la consultation d'un résultat : la flèche *extend* part du cas qui étend et pointe vers le cas de base. L'export PDF suppose l'existence d'un résultat. L'authentification est une précondition aux autres cas et non un lien *include* répété dans tout le diagramme.

2.7 Raffinement et description textuelle des cas d'utilisation

Les huit parcours significatifs sont maintenant détaillés un par un, sans transformer les opérations internes en cas d'utilisation. Pour chacun, un tableau donne la description textuelle champ par champ, puis le diagramme raffiné correspondant en montre les relations. Trois relations y sont employées. Une relation *include* exprime un comportement toujours exécuté par le cas de base ; la flèche part du cas de base. Une relation *extend* exprime un comportement facultatif ou conditionnel ; la flèche part du cas qui étend et pointe vers le cas de base. Enfin, une généralisation, dessinée avec un triangle creux, relie un cas abstrait (en italique) à des cas spécialisés qui sont des alternatives : le recruteur en réalise un à la fois.

2.7.1 UC-01 : s'authentifier

Le tableau 2.4 décrit l'authentification et l'ouverture de session ; le diagramme correspondant est donné par la figure 2.2.

Tableau 2.4: UC-01 : s'authentifier

Champ	Description
Identifiant	UC-01
Titre	S'authentifier
Acteur principal	Visiteur non authentifié
Objectif	Ouvrir une session recruteur donnant accès aux données possédées.
Déclencheur	Le visiteur valide le formulaire de connexion.
Préconditions	Un compte recruteur existe.
Postconditions	Le JWT est stocké côté client et le profil courant est récupérable.
Scénario nominal	1. Le visiteur saisit e-mail et mot de passe. 2. Le système vérifie les identifiants. 3. Le système émet le JWT bearer. 4. L'interface charge `/auth/me` puis redirige vers l'espace opérationnel.
Scénarios alternatifs	Un nouveau visiteur choisit la création de compte, puis revient à la connexion.
Exceptions	Identifiants invalides, compte absent ou API indisponible : message d'erreur, aucune session créée.
Règles associées	RG-01.

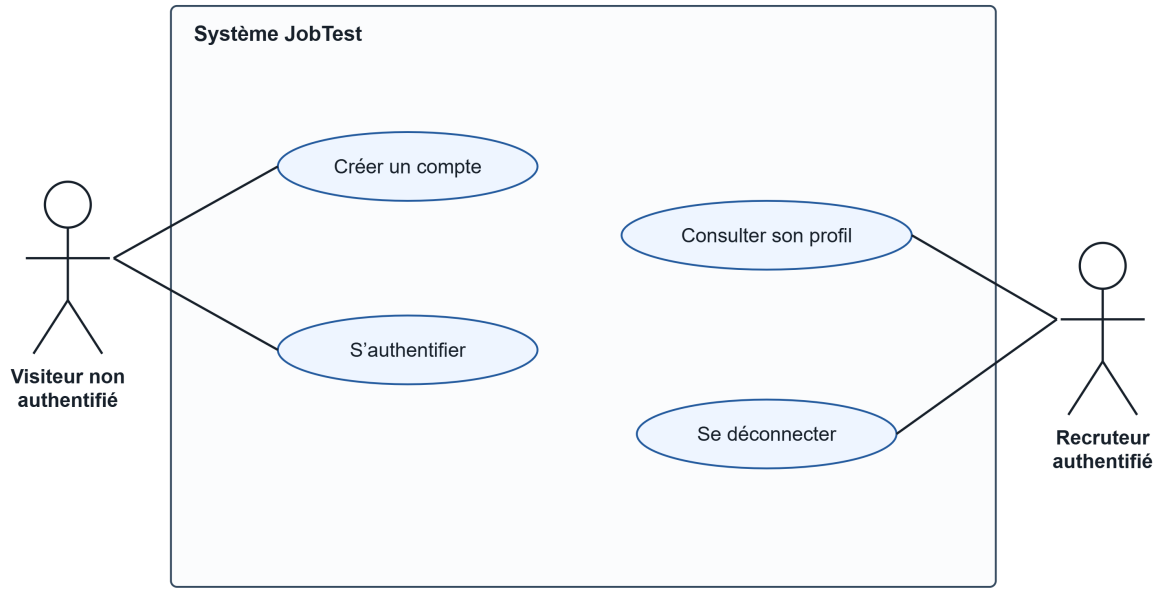


Figure 2.2: Diagramme UC-01 : authentification

2.7.2 UC-02 : gérer une offre d'emploi

La gestion des offres est décrite dans le tableau 2.5, puis représentée par la figure 2.3.

Tableau 2.5: UC-02 : gérer une offre d'emploi

Champ	**Description**
Identifiant	UC-02
Titre	Gérer une offre d'emploi
Acteur principal	Recruteur authentifié
Objectif	Créer, consulter, modifier, clôturer ou supprimer une offre possédée.
Déclencheur	Le recruteur ouvre la page des offres ou son formulaire.
Préconditions	Session valide.
Postconditions	L'offre est créée ou mise à jour ; l'historique associé reste cohérent.
Scénario nominal	1. Le recruteur saisit titre, description, compétences et statut. 2. Le système valide titre et description. 3. À la création, il attribue un code `JOB-YYYY-NNN`. 4. Il enregistre l'offre sous le recruteur courant. 5. L'interface actualise la liste.
Scénarios alternatifs	Le recruteur modifie une offre ou utilise l'action de bascule `OPEN`/`CLOSED`.
Exceptions	Offre non possédée ou champs obligatoires absents : opération refusée. Suppression avec évaluations : réponse 409 et proposition de clôture.
Règles associées	RG-01, RG-02.

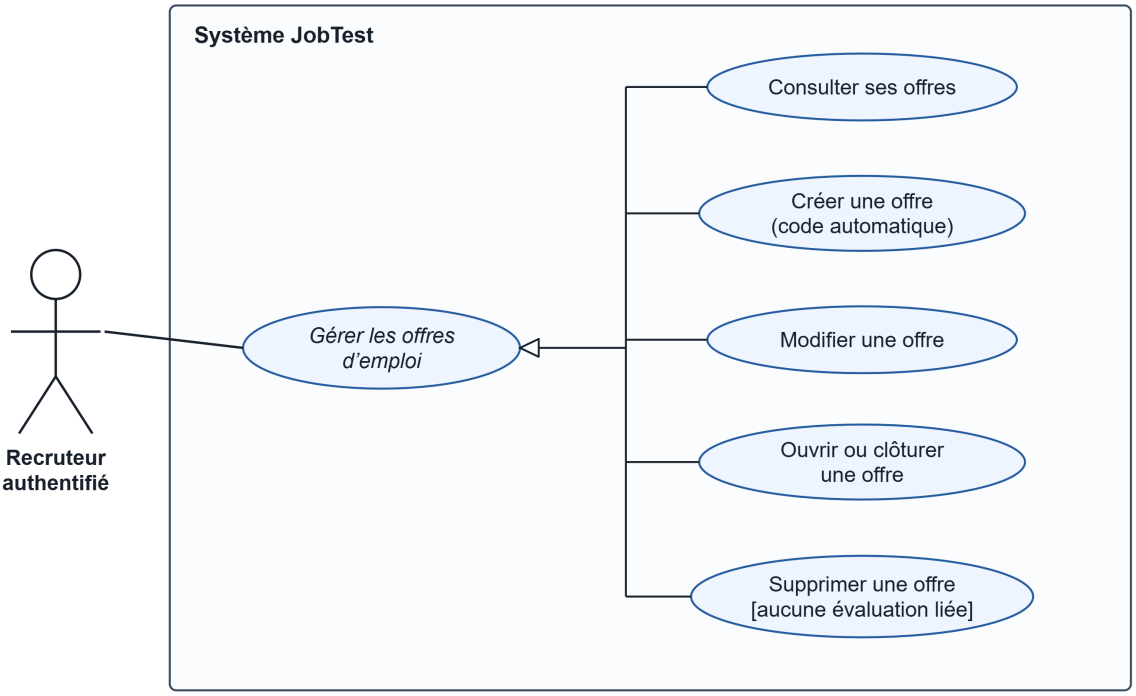


Figure 2.3: Diagramme UC-02 : offres d'emploi

Ce diagramme utilise la généralisation plutôt que l'inclusion : créer, consulter, modifier, changer le statut ou supprimer sont des opérations alternatives, et un lien *include* aurait laissé croire qu'elles s'exécutent toutes à chaque gestion. La suppression porte sa condition entre crochets, car l'application la refuse lorsqu'une évaluation est liée.

2.7.3 UC-03 : gérer un candidat

Le tableau 2.6 poursuit avec le suivi des candidats, dont la figure 2.4 donne le diagramme.

Tableau 2.6: UC-03 : gérer un candidat

Champ	**Description**
Identifiant	UC-03
Titre	Gérer un candidat
Acteur principal	Recruteur authentifié
Objectif	Mettre à jour et suivre les dossiers candidats issus des évaluations.
Déclencheur	Le recruteur consulte la page candidats et choisit un dossier.
Préconditions	Session valide ; candidat possédé par le recruteur.
Postconditions	Les coordonnées ou le statut de pipeline sont mis à jour, ou le dossier non évalué est supprimé.

Champ	Description
Scénario nominal	1. Le système liste les candidats du recruteur. 2. Le recruteur ouvre l'édition. 3. Il modifie nom, e-mail, téléphone ou statut. 4. Le système valide puis enregistre. 5. L'interface affiche les données actualisées.
Scénarios alternatifs	Le recruteur consulte l'activité et le meilleur score du dossier.
Exceptions	Nom vide, e-mail déjà utilisé ou candidat hors périmètre : mise à jour refusée. Candidat évalué : suppression bloquée.
Règles associées	RG-01, RG-03.

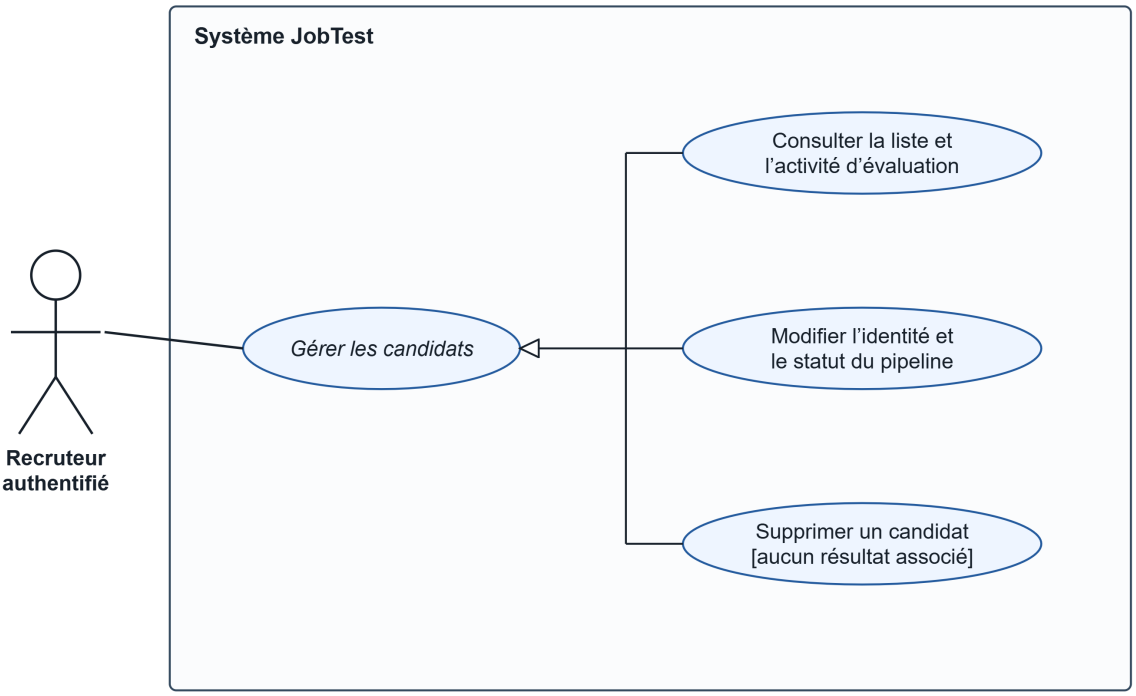


Figure 2.4: Diagramme UC-03 : candidats

La même généralisation s'applique ici : consulter, modifier ou supprimer un dossier sont des opérations alternatives, et la suppression reste conditionnée à l'absence de résultat associé.

2.7.4 UC-04 : évaluer un CV

Vient ensuite l'évaluation d'un CV, dont le tableau 2.7 détaille le déroulement et les exceptions, et la figure 2.5 les relations.

Tableau 2.7: UC-04 : évaluer un CV

Champ	**Description**
Identifiant	UC-04
Titre	Évaluer un CV
Acteur principal	Recruteur authentifié
Objectif	Obtenir et conserver une évaluation explicable d'un CV contre une offre.
Déclencheur	Le recruteur lance l'évaluation depuis la page d'analyse.
Préconditions	Session valide ; offre possédée, ouverte ou en attente ; texte de CV ou fichier PDF/DOCX disponible ; seuils valides.
Postconditions	Un `MatchResult` est persisté avec score, décision, explication et métadonnées.
Scénario nominal	1. Le recruteur choisit une offre. 2. Il fournit identité facultative et texte CV, ou un PDF/DOCX. 3. Le système valide l'entrée et extrait le texte si nécessaire. 4. Il exécute l'appariement statistique choisi. 5. Le *DecisionEngine* détermine le verdict. 6. Le système persiste le résultat. 7. L'interface affiche score, sous-scores, compétences, écarts et recommandations.
Scénarios alternatifs	Le recruteur remplace la méthode hybride, proposée par défaut, par l'une des autres méthodes admises : embedding, TF-IDF ou compétences.
Exceptions	Offre manquante ; texte inférieur à cinq mots ; format non admis ; document sans texte lisible ; seuils invalides ; erreur de persistance : aucune évaluation valide n'est affichée comme réussie.
Règles associées	RG-01, RG-04, RG-05, RG-09, RG-10.

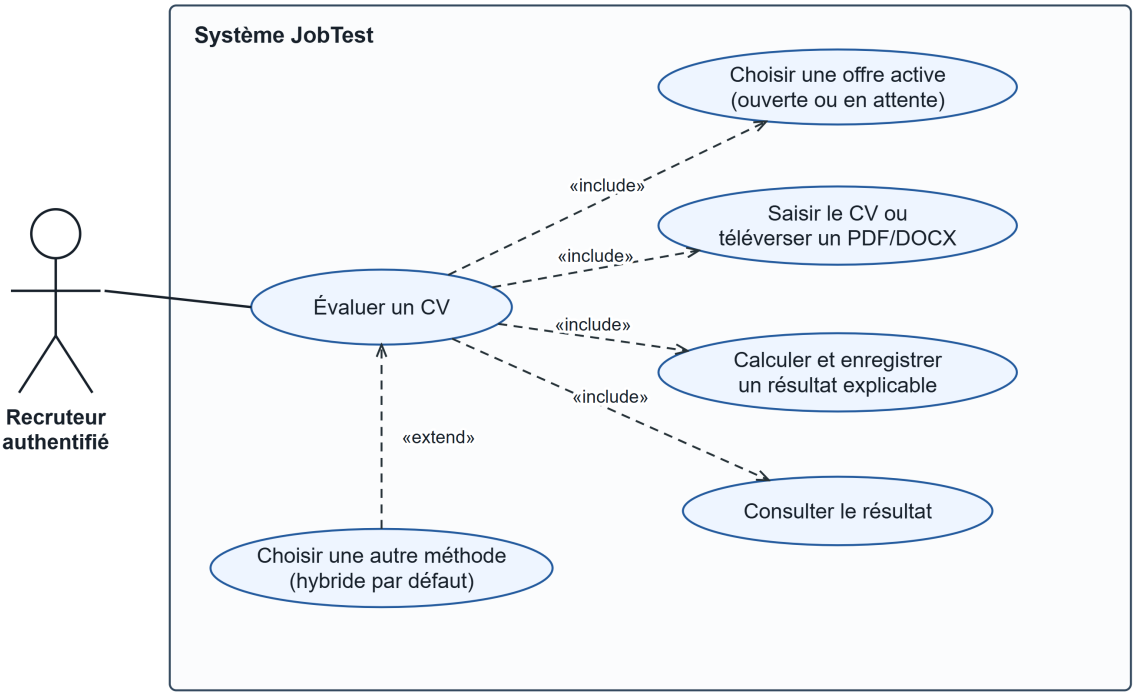


Figure 2.5: Diagramme UC-04 : évaluation d'un CV

Le choix d'une autre méthode y est une extension, puisque la méthode hybride est proposée par défaut.

2.7.5 UC-05 : demander une analyse approfondie

Le tableau 2.8 traite la demande d'analyse approfondie et son caractère facultatif ; la figure 2.6 en donne le diagramme.

Tableau 2.8: UC-05 : demander une analyse approfondie

Champ	**Description**
Identifiant	UC-05
Titre	Demander une analyse approfondie
Acteur principal	Recruteur authentifié
Objectif	Compléter un résultat persistant par une lecture qualitative locale.
Déclencheur	Le résultat Layer 1 est déjà affiché et le recruteur demande l'analyse.
Préconditions	Session valide ; `MatchResult` possédé par le recruteur.
Postconditions	Une enveloppe LLM est affichée ; elle est sauvegardée seulement si son état est `ok`.

Champ	Description
Scénario nominal	1. Le recruteur demande l'analyse. 2. Le système reconstruit le contexte depuis le résultat existant. 3. Il sollicite Ollama. 4. Il vérifie la réponse. 5. Il affiche la synthèse et persiste l'enveloppe réussie.
Scénarios alternatifs	Après un état indisponible ou en erreur, le recruteur relance la demande depuis le même résultat.
Exceptions	LLM désactivé, indisponible, sortie malformée ou incohérente : l'état est retourné sans modifier score ni verdict.
Règles associées	RG-01, RG-06, RG-08.

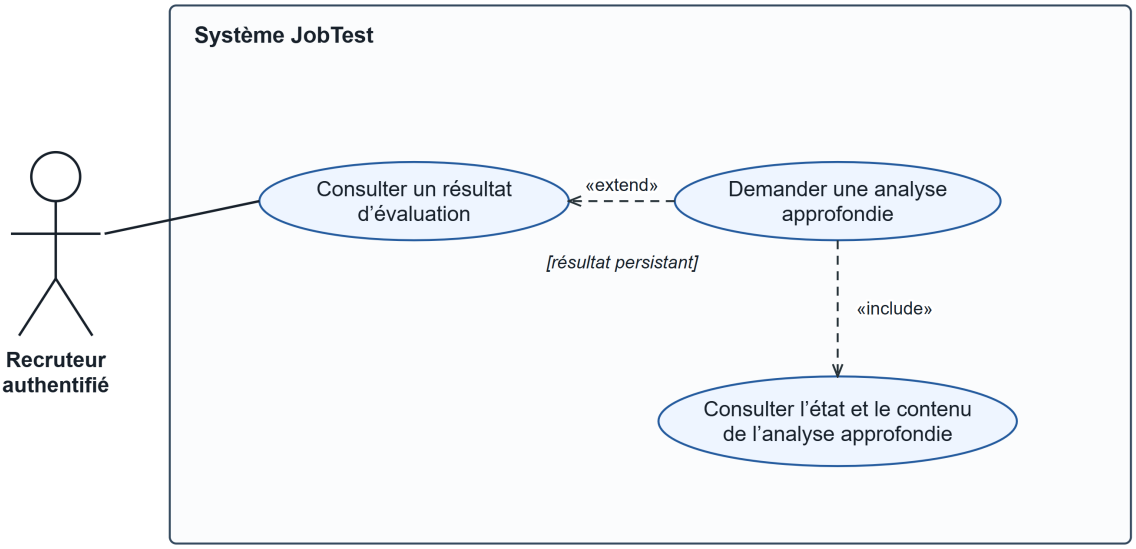


Figure 2.6: Diagramme UC-05 : analyse approfondie

Ce diagramme isole le complément LLM du flux obligatoire d'évaluation décrit par la figure 2.5 : il n'est possible que sur un résultat déjà persisté.

2.7.6 UC-06 : générer un rapport PDF

La génération du rapport, déclenchée depuis un résultat existant, est décrite par le tableau 2.9 et représentée par la figure 2.7.

Tableau 2.9: UC-06 : générer un rapport PDF

Champ	Description
Identifiant	UC-06
Titre	Générer un rapport PDF

Champ	Description
Acteur principal	Recruteur authentifié
Objectif	Télécharger une restitution portable d'une évaluation calculée.
Déclencheur	Le recruteur active l'export depuis le résultat.
Préconditions	Un résultat et son explication sont disponibles.
Postconditions	Le navigateur reçoit le PDF ; aucun nouveau score n'est calculé.
Scénario nominal	1. Le recruteur sélectionne l'export. 2. L'interface transmet le résultat, l'explication, les recommandations et les libellés disponibles. 3. Le système produit le document. 4. Le navigateur déclenche le téléchargement.
Scénarios alternatifs	Le recruteur revient au résultat sans exporter.
Exceptions	Données de résultat incomplètes ou échec de génération : l'interface signale l'échec.
Règles associées	RG-01, RG-09.

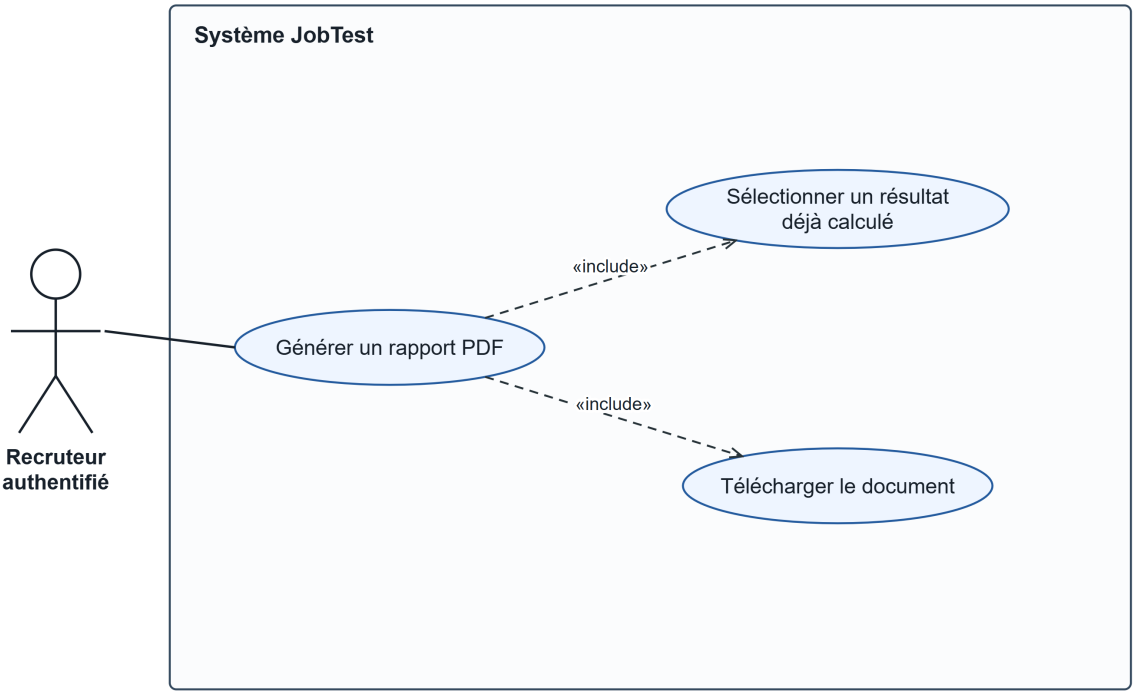


Figure 2.7: Diagramme UC-06 : rapport PDF

2.7.7 UC-07 : classer plusieurs candidats

Le tableau 2.10 formalise le classement de plusieurs candidats pour une offre unique ; la figure 2.8 en donne le diagramme.

Tableau 2.10: UC-07 : classer plusieurs candidats

Champ	**Description**
Identifiant	UC-07
Titre	Classer plusieurs candidats
Acteur principal	Recruteur authentifié
Objectif	Comparer plusieurs CV contre une seule offre.
Déclencheur	Le recruteur sélectionne une offre et plusieurs fichiers.
Préconditions	Session valide ; offre possédée, ouverte ou en attente ; fichiers PDF/DOCX lisibles.
Postconditions	Chaque évaluation réussie est persistée et l'interface affiche le classement décroissant.
Scénario nominal	1. Le recruteur choisit une offre. 2. Il ajoute plusieurs CV PDF/DOCX. 3. L'interface soumet chaque fichier au flux d'évaluation. 4. Elle collecte les résultats réussis. 5. Elle les trie par score décroissant. 6. Elle permet l'export CSV du classement courant.
Scénarios alternatifs	Le recruteur retire un fichier avant le lancement.
Exceptions	Aucun poste, fichier non lisible ou évaluation en échec : le fichier concerné est signalé ; les autres résultats valides restent consultables.
Règles associées	RG-01, RG-04, RG-05, RG-10.

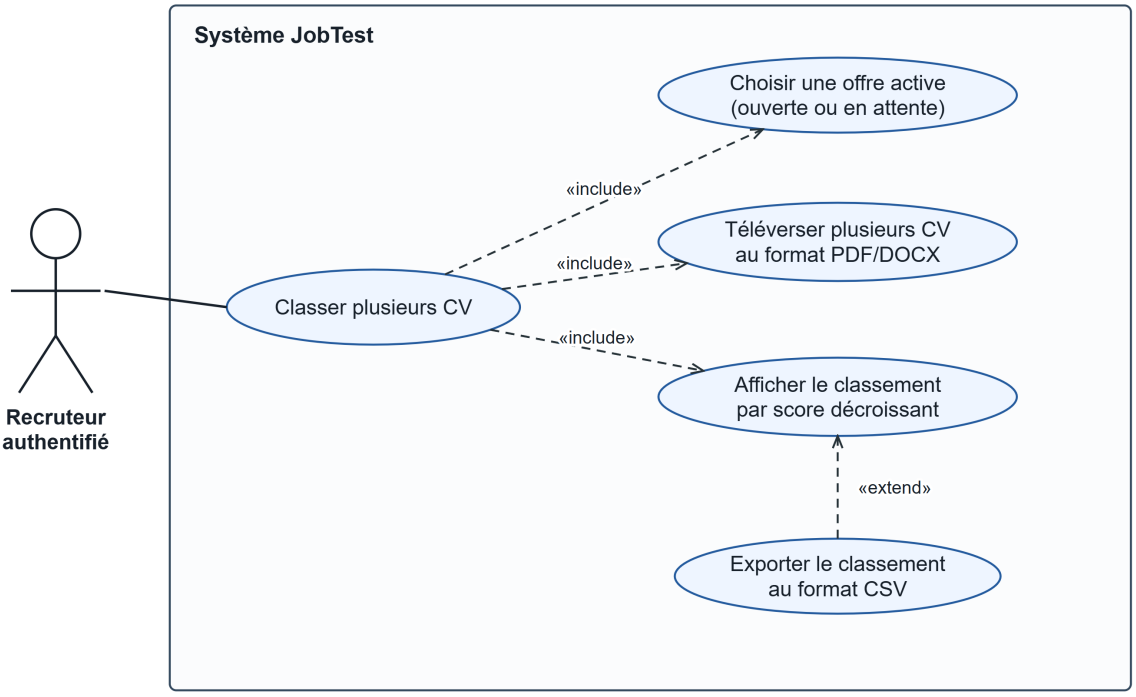


Figure 2.8: Diagramme UC-07 : classement

L'export CSV du classement y est une action facultative et est donc modélisé par *extend*.

2.7.8 UC-08 : consulter l'historique

La consultation de l'historique clôt la série avec le tableau 2.11 et la figure 2.9.

Tableau 2.11: UC-08 : consulter l'historique

Champ	**Description**
Identifiant	UC-08
Titre	Consulter l'historique et les résultats
Acteur principal	Recruteur authentifié
Objectif	Retrouver les évaluations persistées et leur détail.
Déclencheur	Le recruteur ouvre la page historique.
Préconditions	Session valide.
Postconditions	Une liste filtrée ou le détail d'un résultat possédé est affiché.
Scénario nominal	1. Le système liste les évaluations du recruteur par date décroissante. 2. Le recruteur filtre par code, offre ou e-mail. 3. Il sélectionne une ligne. 4. Le système vérifie l'appartenance et retourne le détail. 5. L'interface affiche les métadonnées et données associées.
Scénarios alternatifs	Le recruteur consulte l'historique sans filtre ou revient à la liste.

Champ	Description
Exceptions	Identifiant absent ou résultat d'un autre recruteur : accès refusé ; erreur réseau : état d'erreur affiché.
Règles associées	RG-01, RG-09.

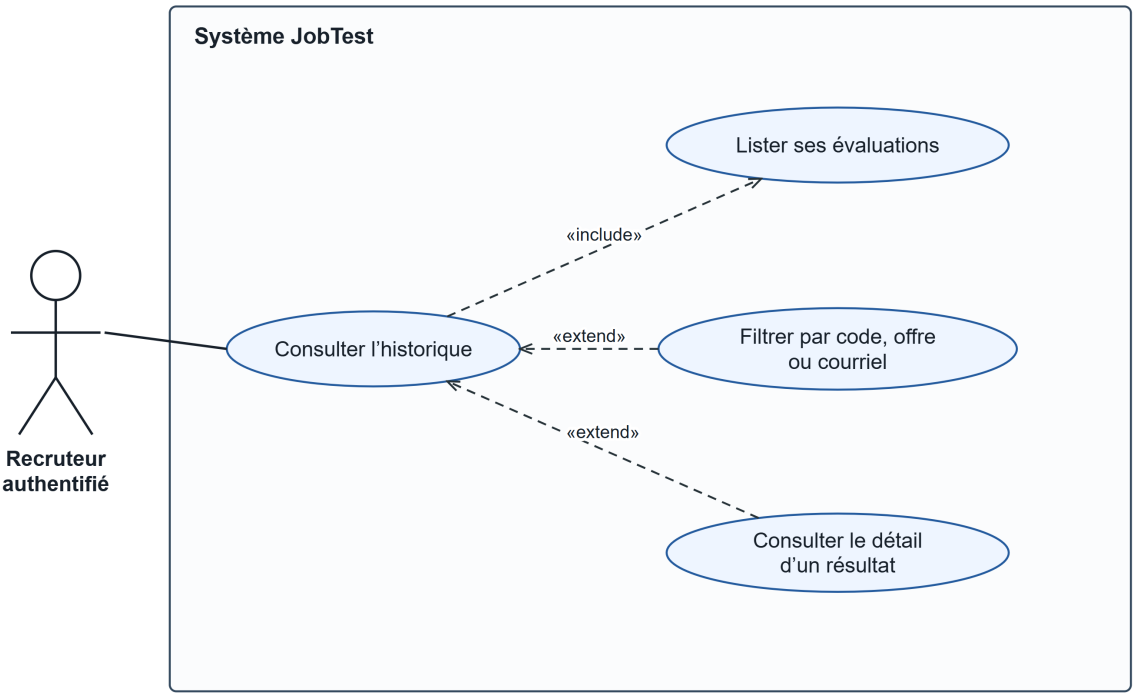


Figure 2.9: Diagramme UC-08 : historique

Le filtrage et la consultation du détail d'un résultat sont facultatifs et sont donc modélisés par *extend*.

2.8 Matrice de traçabilité

La matrice relie les domaines de besoins définis plus haut aux cas d'utilisation et à des groupes de routes ou composants observés. Elle ne remplace pas une documentation API exhaustive ; elle constitue une preuve de correspondance entre l'analyse et la version implémentée.

Le tableau 2.12 en présente le contenu.

Tableau 2.12: Matrice de traçabilité des besoins

Domaine de besoins	Cas d'utilisation	Composant ou route vérifiée	Critère de validation
Authentification du recruteur	UC-01	`/auth/register`, `/auth/login`, `/auth/me`, client API	Compte créé, jeton reçu, profil chargé et jeton effacé à la déconnexion.

Domaine de besoins	**Cas d'utilisation**	**Composant ou route vérifiée**	**Critère de validation**
Gestion des offres	UC-02	`/ui/job-offers`, page `/jobs`	Création codée, mise à jour, statuts et garde 409 vérifiables.
Gestion des candidats	UC-03	`/ui/candidates`, page `/candidates`	Liste propriétaire, édition de pipeline et suppression conditionnelle.
Évaluation d'un texte ou d'un fichier	UC-04	`/ui/evaluate`, `/ui/evaluate/file`, `evaluate_and_persist`	Résultat persistant et explication après texte ou fichier valide.
Analyse approfondie facultative	UC-05	`/ui/evaluate/deep-analysis`, `llm_reasoning`	États LLM rendus sans changement du verdict.
Rapport PDF	UC-06	`/report/pdf`, client `reportPdf`	Réponse PDF téléchargeable depuis le résultat.
Classement et export CSV	UC-07	page `/leaderboard`, `evaluateFile`	Tri décroissant des résultats et CSV côté client.
Historique des évaluations	UC-08	`/ui/matches`, `/ui/matches/{id}`, page `/history`	Liste filtrée et détail soumis à l'appartenance.
Comparaison et benchmark	Diagramme global	`/ui/compare`, `/ui/quality`	Comparaison sans persistance et benchmark file-backed distingués.
Réglage des seuils	Diagramme global	`decision-config.tsx`, page `/settings`	Seuils validés, envoyés aux nouvelles évaluations et mémorisés avec elles.

2.9 Contraintes et hypothèses

Le tableau 2.13 rassemble ces contraintes et hypothèses ainsi que leur effet sur l'analyse.

Nature	Contrainte ou hypothèse	Effet sur l'analyse
Fonctionnelle	Aucun portail candidat ni décision autonome d'embauche.	Le recruteur reste l'acteur opérationnel et décideur final.
Fonctionnelle	Les paramètres de seuil sont locaux à la session du navigateur.	Ils ne constituent pas une politique serveur globale et n'écrasent pas l'historique.
Technique	Textes de CV et d'offres possibles en français ou anglais.	La qualité dépend du contenu extrait et des modèles disponibles.
Technique	PDF/DOCX seulement ; extraction parfois impossible ou incomplète.	Les fichiers non lisibles sont rejetés avant évaluation.
Technique	Ollama local, CPU/RAM limités et latence notable.	La Deep Analysis est optionnelle, séparée du verdict et susceptible d'indisponibilité.
Évaluation	Benchmark quantitatif synthétique ; essais fonctionnels avec mes propres CV, sans jugement de référence.	Les indicateurs hors ligne ne prouvent pas une performance RH réelle.
Déploiement	VM Azure PFE, HTTPS, instance unique et pas de haute disponibilité.	La solution est démontrable mais non qualifiée comme hébergement permanent.
Historique	Discontinuité de score pour des résultats antérieurs à une correction de pondérations.	La comparaison temporelle doit tenir compte de la méthode et des métadonnées conservées.

Tableau 2.13: Contraintes et hypothèses

Conclusion

Ce chapitre a fixé les limites de JobTest, identifié le recruteur comme acteur fonctionnel principal et distingué le candidat comme entité métier. Les besoins couvrent le parcours implémenté, depuis la session et les données de recrutement jusqu'à l'évaluation explicable, son historique et ses compléments facultatifs. Les règles de gestion maintiennent la propriété des données, la cohérence de l'historique et la séparation stricte entre décision statistique et analyse LLM. Les cas d'utilisation et la matrice de traçabilité constituent la base de la conception fonctionnelle et architecturale présentée au chapitre suivant.

Chapitre 3

Conception fonctionnelle et architecturale

Plan

Introduction 34

1 Démarche de conception 34

2 Vue fonctionnelle globale 34

3 Architecture logique de JobTest 35

4 Architecture applicative et responsabilités 35

5 Conception statique – diagramme de classes global 36

6 Conception dynamique – diagrammes de séquence 37

7 Conception des données 40

8 Principes de conception retenus 41

Conclusion 41

Introduction

Le chapitre précédent a défini le périmètre de JobTest et les besoins du recruteur. Le présent chapitre traduit ces besoins en une conception vérifiable dans l'application : responsabilités, données, interactions et déploiement logique. La conception sépare volontairement le calcul déterministe de correspondance de l'analyse générative. Ainsi, le recruteur conserve la décision finale ; le système restitue un résultat de correspondance et des éléments d'interprétation, sans automatiser l'embauche.

3.1 Démarche de conception

Dans le cycle en cascade retenu, la conception établit le passage des exigences du chapitre 2 vers des composants implémentables, avant leur réalisation et leur validation. Elle fixe les responsabilités et les échanges qui doivent demeurer stables lorsque l'interface ou l'infrastructure évoluent.

La notation UML 2.5.1 est employée de manière ciblée : le diagramme de classes représente les objets persistants et leurs liens, tandis que les diagrammes de séquence montrent l'ordre des appels et les alternatives importantes. La spécification UML de l'OMG sert de référence de notation [16]. Le diagramme global de cas d'utilisation reste celui de la figure 2.1; il n'est donc pas reproduit ici.

3.2 Vue fonctionnelle globale

La vue complémentaire de la figure 3.1 organise les fonctions par enchaînement plutôt que par acteur. L'authentification ouvre une session de recruteur. Celui-ci gère ses offres et ses candidats, lance l'évaluation puis consulte un résultat persistant. Le résultat constitue le point d'entrée du rapport PDF, de l'historique et de l'analyse approfondie. Cette dernière est une couche complémentaire : elle reçoit des informations déjà calculées et ne revient jamais modifier le score ni le verdict déterministe.

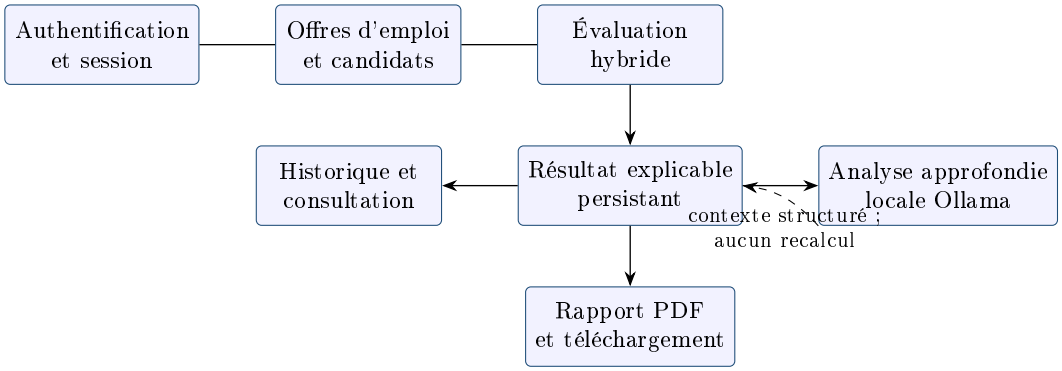


Figure 3.1: Vue fonctionnelle de JobTest

3.3 Architecture logique de JobTest

La solution déployée est un monolithe modulaire conteneurisé, et non un ensemble de microservices métier autonomes. Le navigateur ne joint publiquement que le proxy inverse, en HTTPS sur le port 443 (le port 80 redirige vers HTTPS). Ce proxy relaie le préfixe `/api/v1/` vers FastAPI et les autres chemins vers Next.js. Les ports 3000, 8000, 5432 et 11434 sont internes au réseau Docker : ils ne sont pas publiés sur Internet.

La figure 3.2 représente cette organisation ainsi que les frontières réseau qui la délimitent.

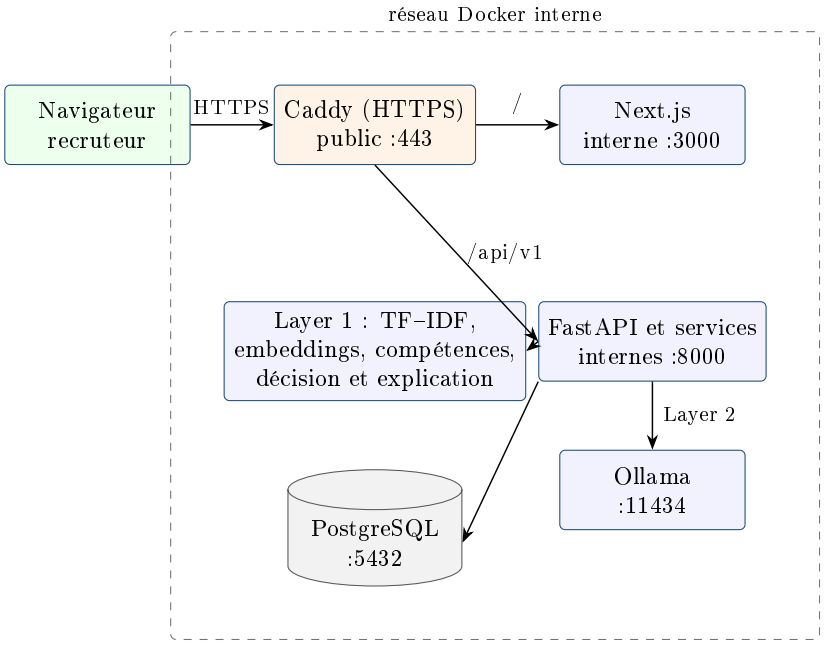


Figure 3.2: Architecture logique de JobTest

La couche de présentation est une application Next.js. Son client API conserve le jeton bearer dans le stockage local du navigateur et utilise le même origin `/api/v1`. La couche FastAPI porte l'authentification, les routes HTTP, les contrôles d'appartenance et l'orchestration des services. Elle appelle en processus la Layer 1, composée des modèles TF–IDF, embeddings et compétences, du moteur de décision et du générateur d'explications.

La Layer 2 appelle Ollama à l'intérieur du réseau Docker avec un contexte structuré issu de l'évaluation déjà persistée. PostgreSQL est la persistance du déploiement; l'application conserve néanmoins SQLite comme valeur par défaut de développement à travers `DATABASE_URL`. Le chapitre 6 détaillera les choix d'exploitation Azure, sans les confondre avec cette vue logique.

3.4 Architecture applicative et responsabilités

Le tableau 3.1 relie les responsabilités de conception aux modules réellement présents. Les routes `/ui` sont un routeur de parité ajoutant une façade HTTP pour l'interface Next.js : elles réutilisent les mêmes services de calcul, modèles SQLAlchemy et moteur de décision.

Tableau 3.1: Modules applicatifs et responsabilités

Couche ou module	Éléments vérifiés	Responsabilité
Interface utilisateur	`web/app`, composants d'analyse, `web/lib/api/client.ts`	Écrans Next.js, saisie des offres et CV, restitution des résultats, appel des routes et téléchargement du PDF.
Exposition HTTP	`src/api/routers/auth.py`, `src/api/routers/ui.py`, `src/api/main.py`	JWT, validation Pydantic, contrôle du recruteur connecté, routes métiers et transformation des réponses JSON.
Services métier	`src/matching/service.py`, `src/decisioning/decision_engine.py`	Orchestration de l'évaluation, décision canonique, persistance atomique et métadonnées de reproductibilité.
Extraction et modèles	`src/parsing/parser.py`, `src/models`, `src/fusion/hybrid_scorer.py`	Extraction de texte PDF/DOCX, scores TF–IDF, sémantique et compétences, puis agrégation hybride.
Explicabilité	`src/explainability/explainer.py`	Composantes de score, analyse des écarts, compétences et recommandations cohérentes avec le résultat déterministe.
Analyse approfondie	`src/ai/llm_reasoning.py`, `DeepAnalysis.tsx`	Construction du contexte, appel Ollama local, validation JSON, contrôle de cohérence et restitution d'un état contrôlé.
Rapport PDF	`src/reporting/pdf_generator.py`, `/report/pdf`	Composition ReportLab du rapport à partir du résultat, de l'explication et des recommandations déjà calculés.
Persistance	`src/models_db.py`, `src/database.py`	Modèles SQLAlchemy, relations, transactions et adaptation SQLite/PostgreSQL par configuration.

3.5 Conception statique – diagramme de classes global

La figure 3.3 reprend les classes persistées définies dans `src/models_db.py`, avec leurs attributs principaux ; les attributs techniques secondaires et les opérations des classes de service ne sont pas représentés. Le recruteur n'est pas une sous-classe : c'est un `User` dont le rôle vaut `recruiter`, et le candidat est une entité métier sans compte. Une analyse LLM n'est pas une table distincte : son enveloppe JSON est conservée dans `MatchResult.llm_analysis_json` uniquement lorsqu'elle est valide. De même, le rapport PDF est généré à la demande et ne possède pas d'entité persistée propre.

Un utilisateur recruteur possède des offres et des candidats. Un candidat peut regrouper plusieurs CV, qui n'existent pas sans lui (composition), et plusieurs résultats de correspondance ; chaque résultat se rattache, lorsqu'ils sont connus, au CV et à l'offre qui ont servi à son calcul. Les multiplicités 0..1 côté résultat traduisent des clés étrangères facultatives dans le schéma. Les

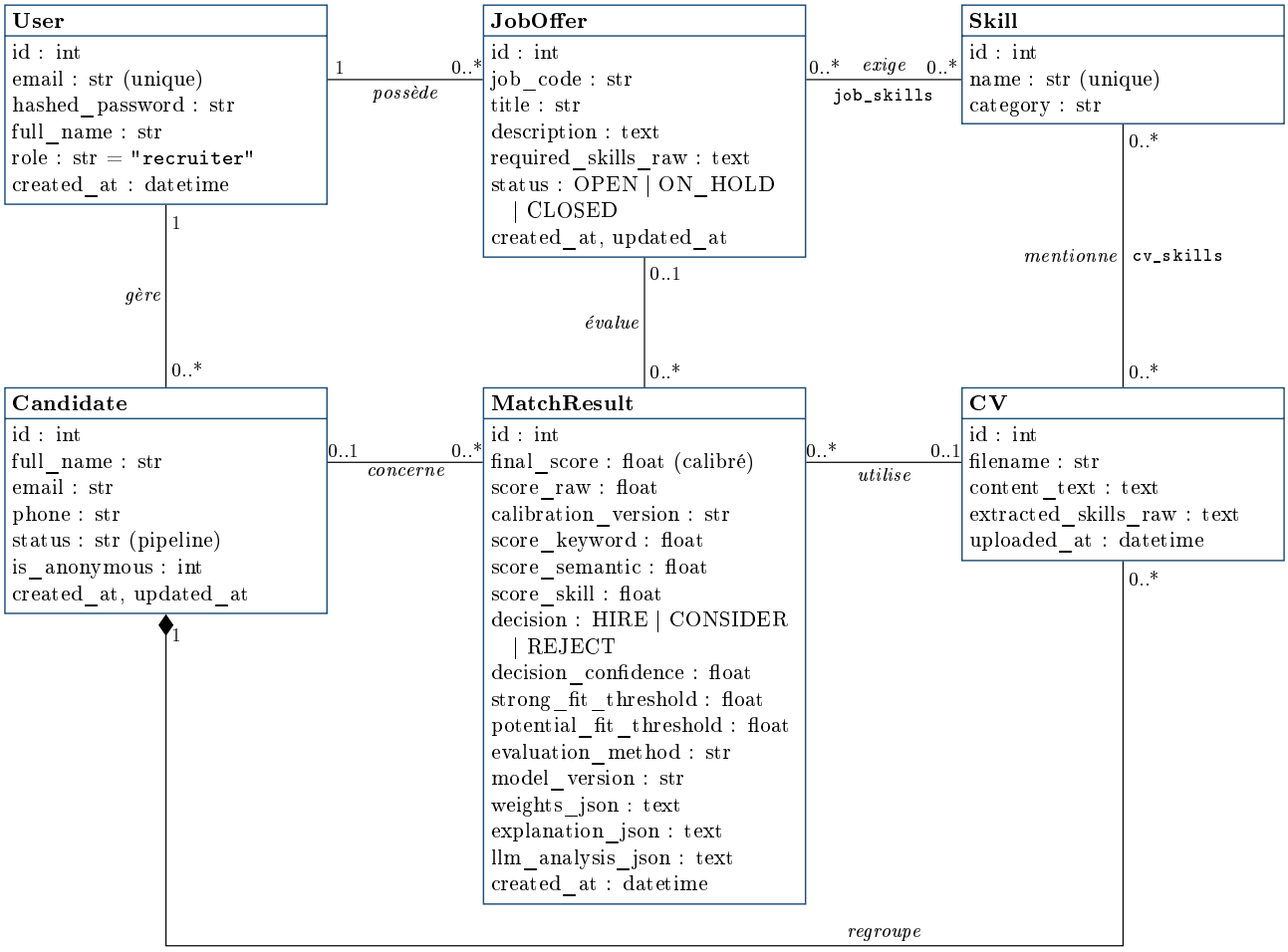


Figure 3.3: Diagramme de classes de JobTest

compétences sont normalisées dans `Skill` et reliées aux CV et aux offres par les tables d'association `cv_skills` et `job_skills`. Les chemins actuels d'évaluation conservent également le texte et les informations d'explication dans le résultat pour en permettre la consultation ultérieure.

3.6 Conception dynamique – diagrammes de séquence

3.6.1 Authentification du recruteur

La connexion utilise le formulaire OAuth2 de FastAPI. Après vérification du mot de passe haché, l'API émet un JWT bearer. L'interface le stocke localement puis appelle `/auth/me` afin de charger le profil de session.

La figure 3.4 détaille cet échange, depuis la soumission du formulaire jusqu'au chargement du profil.

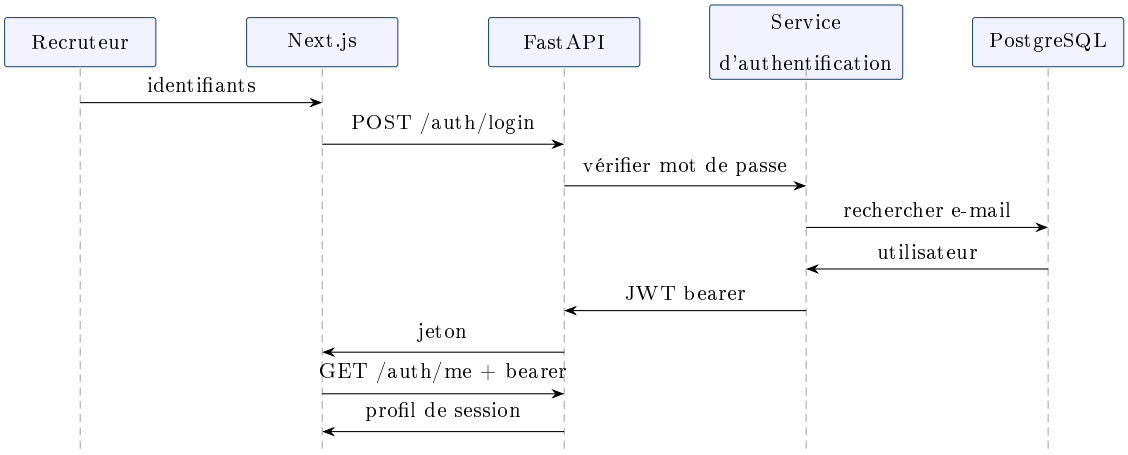


Figure 3.4: Séquence d'authentification

3.6.2 Création d'une offre d'emploi

L'écran des offres transmet le titre, la description, les compétences requises et l'état demandé. La route `/ui/job-offers` vérifie la présence du titre et de la description, associe l'offre au recruteur connecté et construit le code `JOB-AAAA-NNN` avant l'écriture en base.

La figure 3.5 en retrace le déroulement.

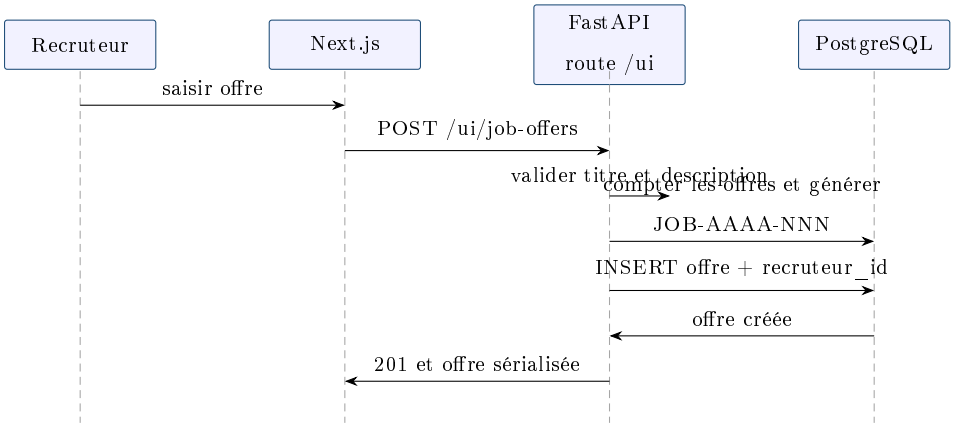


Figure 3.5: Séquence de création d'une offre

3.6.3 Évaluation d'un CV par rapport à une offre

La figure 3.6 représente le chemin canonique. Le CV est fourni comme texte ou par téléversement; dans ce dernier cas, le routeur extrait le texte du PDF ou du DOCX avant l'évaluation. L'offre doit appartenir au recruteur. Le service contrôle d'abord les preuves contenues dans le CV (texte recopié de l'offre, mentions niées, compétences déclarées ou démontrées), exécute les trois composantes, les agrège, calibre le score, applique les seuils et persiste un `MatchResult`.

Pour le modèle hybride par défaut, les scores élémentaires sont compris entre 0 et 1 et la formule effective est :

$$S_h = 0{,}2S_{\text{TF-IDF}} + 0{,}4S_{\text{emb}} + 0{,}4S_{\text{comp}}.$$

Le score brut S_h n'est pas affiché tel quel : une calibration versionnée le convertit en un score sur 100 (chapitre 4). Avec la configuration par défaut, le `DecisionEngine` produit `HIRE` si ce score calibré est supérieur ou égal à 75, `CONSIDER` s'il est compris entre 55 et strictement inférieur à 75, et `REJECT` en dessous de 55. `HIRE` est remplacé par `CONSIDER` lorsque les compétences correspondantes reposent surtout sur des déclarations ou sur du texte recopié. Les seuils de session peuvent être transmis par l'interface, et la configuration utilisée est enregistrée avec le résultat. Ce verdict est une recommandation algorithmique et non une décision de recrutement.

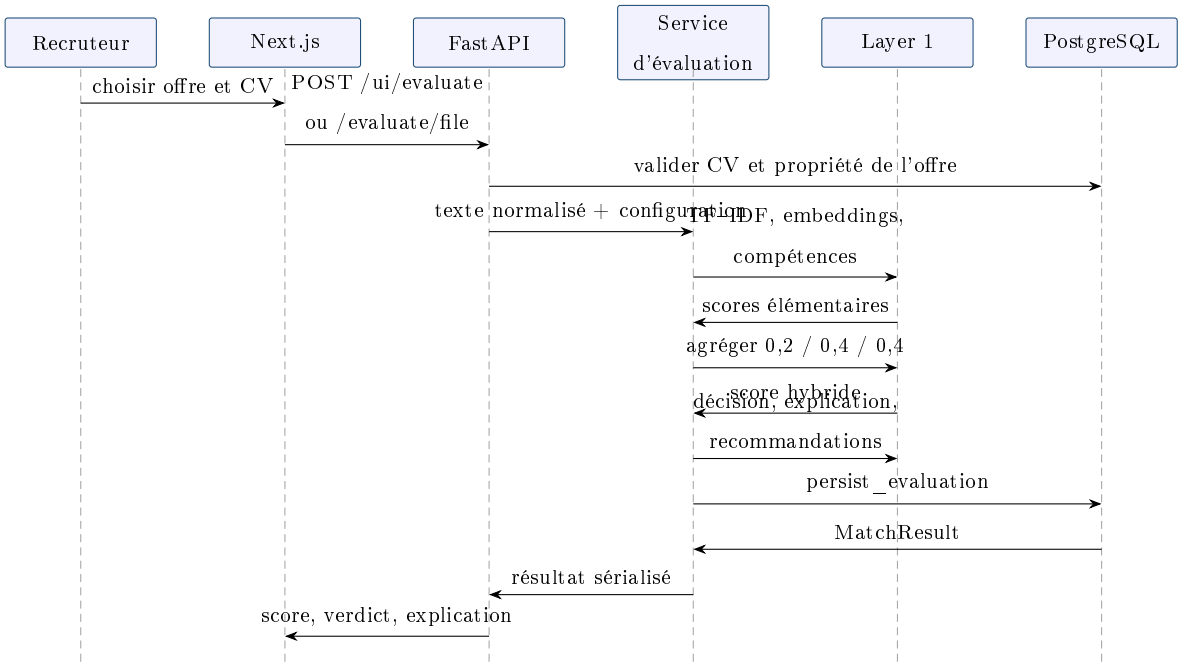


Figure 3.6: Séquence d'évaluation d'un CV

3.6.4 Analyse approfondie avec Ollama

Après le rendu de l'évaluation, le composant `DeepAnalysis` déclenche automatiquement un appel séparé pour l'identifiant du résultat; il peut aussi être relancé depuis l'interface. L'API reconstruit le contexte à partir du `MatchResult` sans recalculer la Layer 1. Ollama reçoit ce contexte structuré et doit fournir un JSON conforme. Une analyse réussie est ajoutée au résultat; sinon l'interface reçoit un état `disabled`, `unavailable`, `error` ou `inconsistent`. Dans tous les cas, le score et la décision de la Layer 1 restent inchangés.

L'enchaînement correspondant apparaît sur la figure 3.7, où l'on voit que l'appel part d'un résultat déjà enregistré.

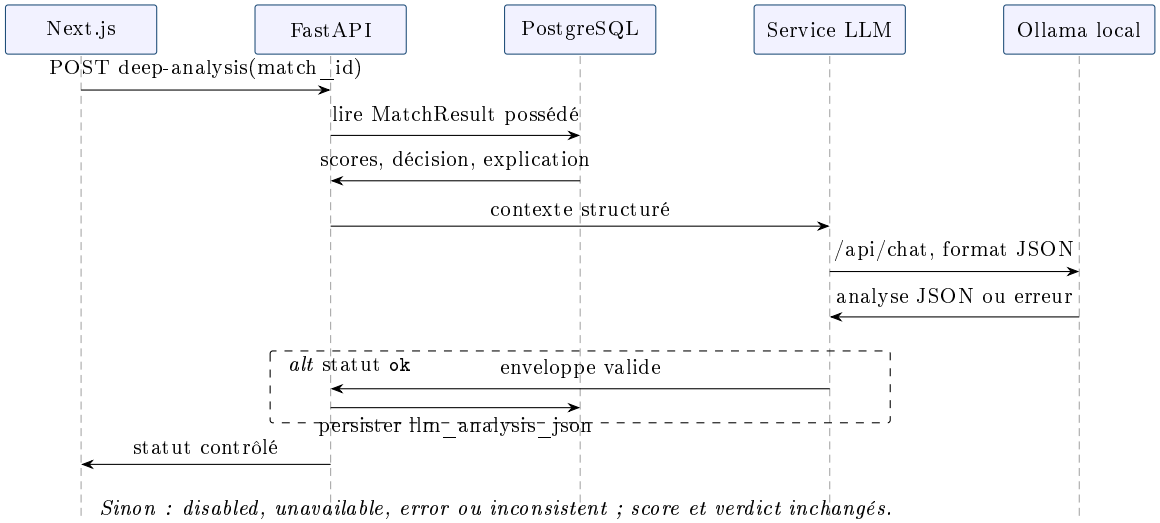


Figure 3.7: Séquence d'analyse approfondie

3.6.5 Génération du rapport PDF

Le téléchargement demande au serveur un document à partir du résultat, de l'explication et des recommandations déjà affichés. FastAPI délègue la mise en page au générateur ReportLab et retourne les octets PDF avec une en-tête de pièce jointe. Cette opération ne crée pas de nouvelle évaluation et ne modifie pas le résultat de correspondance.

La figure 3.8 montre la séquence suivie jusqu'au téléchargement du document.

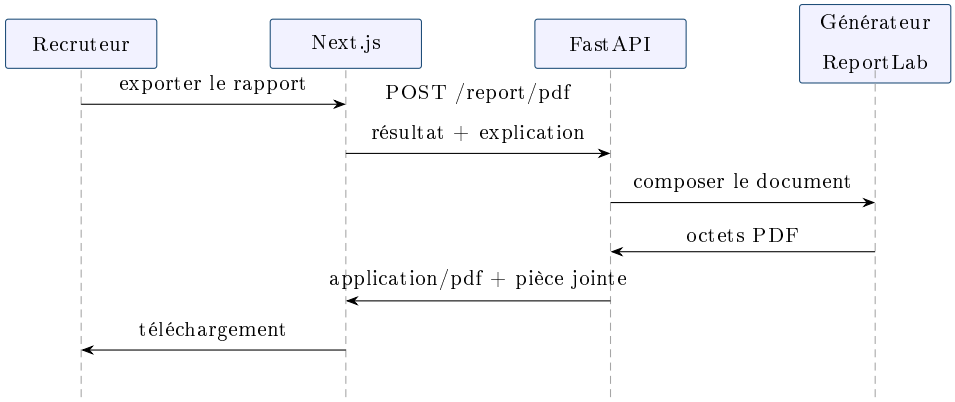


Figure 3.8: Séquence de génération du rapport PDF

3.7 Conception des données

Le modèle relationnel est porté par SQLAlchemy. `users.email`, `skills.name` et `job_offers.job_code` sont déclarés uniques. Les clés étrangères relient candidats et offres au recruteur, les CV aux candidats, puis les résultats à leur candidat, CV et offre. Les suppressions en cascade sont configurées entre candidat et CV, ainsi qu'entre candidat ou offre et résultats de correspondance.

Le résultat de correspondance est l'élément de traçabilité principal : il conserve le score calibré, le score brut, les scores sémantique, lexical et compétences, la décision, les seuils utilisés, la version de calibration, la version du modèle, la méthode et les poids. Un résultat dépourvu de

version de calibration a été calculé sur l'ancienne échelle : il est conservé tel quel, signalé comme tel dans l'historique et exclu des moyennes. L'explication et les recommandations sont sérialisées dans `explanation_json`; l'analyse approfondie valide est ajoutée dans `llm_analysis_json`. Les comparaisons de modèles et le benchmark de qualité ne créent pas de résultats persistants.

En développement, la valeur par défaut de `DATABASE_URL` pointe vers SQLite et l'application active les clés étrangères SQLite. Dans le déploiement conteneurisé, cette variable cible PostgreSQL 16. Les mêmes modèles SQLAlchemy sont employés; la différence porte sur le moteur de base de données, non sur le modèle métier présenté.

3.8 Principes de conception retenus

- **Séparation des responsabilités :** Next.js présente et collecte, FastAPI orchestre, les services calculent, SQLAlchemy persiste et Ollama interprète qualitativement.
- **Traçabilité et reproductibilité :** le résultat conserve les composantes, poids, version de calibration, version du modèle, méthode et seuils actifs lors de son calcul.
- **Isolation :** la pile Docker place PostgreSQL, FastAPI, Next.js et Ollama sur un réseau interne; seul le proxy inverse expose les ports 80 et 443.
- **Dégradation contrôlée :** l'indisponibilité, l'erreur ou l'incohérence de la Layer 2 est représentée par un état explicite, sans bloquer l'évaluation déterministe.
- **Responsabilité humaine :** la décision `HIRE`/`CONSIDER`/`REJECT` facilite l'examen d'un dossier; le recruteur demeure responsable de la décision finale.

Conclusion

La conception de JobTest articule une interface Next.js, une API FastAPI, une persistance relationnelle et deux couches d'intelligence aux rôles distincts. Les diagrammes ont montré la conservation d'un résultat traçable, depuis l'authentification jusqu'au rapport et à l'analyse approfondie. Le chapitre suivant détaillera la conception et l'évaluation du moteur intelligent, notamment ses composantes de correspondance et ses résultats de validation.

Chapitre 4

Conception et évaluation du moteur intelligent

Plan

Introduction 43

1 Objectifs et principes de conception 43

2 Prétraitement et structuration des documents 43

3 Similarité lexicale TF–IDF 43

4 Similarité sémantique par embeddings 44

5 Rapprochement structuré des compétences 44

6 Fusion hybride et décision 45

7 Explicabilité et couche d'analyse locale 46

8 Jeu de données et protocole expérimental 48

9 Résultats et analyse comparative 51

10 Limites, risques et perspectives 55

Conclusion 56

Introduction

Le moteur de rapprochement de JobTest relie un CV et une offre à une recommandation exploitable par un recruteur. Il ne prédit ni une embauche ni une décision autonome : il mesure une compatibilité textuelle et métier, la calibre, puis applique des règles explicites. Ce chapitre décrit les composants réellement implémentés et leur évaluation quantitative reproductible sur le corpus synthétique du projet.

4.1 Objectifs et principes de conception

Le moteur doit accepter des CV hétérogènes, confronter vocabulaire et sens général, signaler les compétences manquantes et rendre le résultat vérifiable. Le choix retenu est une fusion tardive : chaque composant conserve son score avant agrégation. Deux couches sont séparées. La couche 1 produit le score, la décision et les justifications déterministes. La couche 2, optionnelle, enrichit seulement l'affichage par une analyse locale ; elle ne modifie ni score ni décision.

La figure 4.1 résume cet enchaînement et la place qu'y occupe la couche optionnelle.

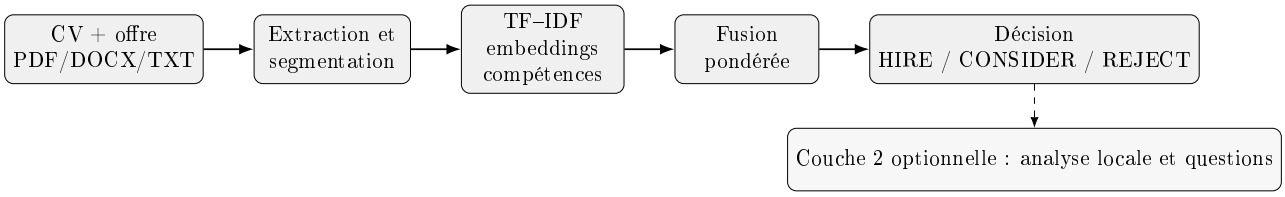


Figure 4.1: Pipeline de rapprochement de JobTest

4.2 Prétraitement et structuration des documents

L'extraction privilégie `pdfplumber` pour les PDF et bascule sur `PyPDF2` en cas d'indisponibilité ; les fichiers DOCX sont lus avec `python-docx`, avec un repli XML. Le texte est nettoyé des URL, courriels, numéros de téléphone, retours superflus et artefacts de césure. La normalisation disponible applique, lorsque les modèles spaCy sont installés, tokenisation, lemmatisation et filtrage des mots-outils ; un repli par expressions régulières et lemmatisation légère maintient le traitement sinon.

Le CV est ensuite segmenté par intitulés français et anglais : profil, expérience, compétences, formation, langues, certifications et projets. Cette structure n'est pas une interprétation humaine du dossier ; elle permet notamment de pondérer l'expérience distinctement des compétences.

4.3 Similarité lexicale TF–IDF

Le premier composant vectorise CV et offre avec `TfidfVectorizer`. La configuration implémentée utilise des n-grammes de 1 à 3 termes, au plus 10 000 caractéristiques et une fréquence de terme

sous-linéaire. TF–IDF valorise les termes discriminants ; la similarité cosinus compare les vecteurs issus des deux textes [17], [18] :

$$\mathrm{tfidf}(t,d) = \mathrm{tf}(t,d)\log\left(\frac{N+1}{\mathrm{df}(t)+1}\right)+1, \qquad S_{\mathrm{TFIDF}} = \frac{\vec{v}_{CV}\cdot\vec{v}_{offre}}{\|\vec{v}_{CV}\|\|\vec{v}_{offre}\|}.$$

Cette composante est lisible lorsque les mêmes technologies sont citées, mais elle reste sensible aux synonymes et aux reformulations.

4.4 Similarité sémantique par embeddings

Le deuxième composant encode les textes avec Sentence-Transformers lorsqu'il est disponible. Le modèle anglais configuré par défaut est `all-mpnet-base-v2`. Le modèle multilingue est identifié dans la configuration par :

`paraphrase-multilingual-mpnet-base-v2`

`HashingVectorizer` constitue le repli local en cas de chargement impossible. Les représentations de phrases et leur comparaison cosinus s'appuient sur le principe de Sentence-BERT [6].

Les similitudes anglaise et multilingue sont moyennées. Une similitude par sections reçoit les poids expérience 0,40 ; compétences 0,30 ; résumé 0,15 ; formation 0,10 ; langues 0,05. Le composant retourne donc :

$$S_{\mathrm{emb}} = 0{,}7\,(0{,}5s_{en} + 0{,}5s_{multi}) + 0{,}3s_{sections}.$$

La pondération rend l'expérience et les compétences plus influentes sans prétendre représenter une adéquation humaine complète.

4.5 Rapprochement structuré des compétences

Le troisième composant exploite le dictionnaire versionné du projet. Il détecte les compétences techniques et comportementales, les certifications, les variantes et des n-grammes normalisés. Les termes de nécessité, tels que *required*, *must* ou *obligatoire*, signalent des compétences critiques.

Soient H et S les recouvrements techniques et comportementaux. La base répartit les poids 0,70 et 0,30 entre les catégories présentes. Elle reçoit un bonus de domaine de 0,05, un bonus de certification plafonné à 0,10, puis une pénalité plafonnée à 0,20 pour les compétences critiques absentes :

$$S_{\mathrm{skill}} = \mathrm{clip}\left(S_{base} + 0{,}05I_{domaine} + \min(0{,}10, 0{,}03n_{cert}) - 0{,}20\frac{n_{critiques\ absentes}}{n_{critiques}}, 0, 1\right).$$

Il restitue aussi les compétences trouvées, manquantes et critiques : ces éléments constituent la justification métier principale.

4.5.1 Contrôle des preuves avant le calcul

Un CV peut citer les compétences d'une offre sans démontrer qu'elles ont été pratiquées. Avant le calcul, la version finale applique donc trois contrôles explicables, dont le résultat accompagne chaque évaluation.

- **Texte recopié de l'offre.** Toute séquence d'au moins huit mots consécutifs également présente dans l'offre est retirée du CV avant le calcul des trois composantes. Sur les 12 000 paires du corpus, la plus longue séquence commune légitime compte sept mots : aucune paire authentique n'est modifiée.
- **Mentions niées.** Une compétence citée dans une proposition négative (« aucune expérience en Kubernetes », *no experience with Docker*) n'est pas comptée, et la proposition correspondante est retirée du texte utilisé pour TF–IDF et les embeddings.
- **Compétences déclarées ou démontrées.** Une compétence qui n'apparaît que dans une liste (section « Compétences », « Technologies » ou ligne « Skills : ... ») est dite déclarée ; elle compte pour moitié dans la couverture. Une compétence citée dans l'expérience, les projets, le profil ou une certification est dite démontrée. Lorsque le CV ne comporte aucun intitulé de section reconnaissable, la distinction est impossible et toutes les mentions sont traitées comme démontrées.

Ces contrôles ne modifient aucun score des 3 600 paires authentiques de validation et de test ; ils agissent sur les CV qui répètent l'offre ou qui reposent sur des listes déclaratives.

4.6 Fusion hybride et décision

La configuration documentée n'apprend pas les poids de fusion ; elle les fixe ainsi :

$$S_{hybride} = 0{,}2S_{\text{TFIDF}} + 0{,}4S_{\text{emb}} + 0{,}4S_{\text{skill}}.$$

Elle donne le même poids à la proximité sémantique et à la couverture explicite des compétences. Le score brut $S_{hybride}$ reste toutefois une combinaison de similarités qui n'ont pas d'échelle commune : la similarité cosinus de deux documents différents dépasse rarement 0,8, le recouvrement TF–IDF reste en général sous 0,5, et la fusion n'atteint pas 0,75 même lorsque chaque composante est à son maximum observé. Lu directement en pourcentage contre des bornes à 55 et 75, ce score ne produirait presque que des verdicts `REJECT`.

La version finale interpose donc une calibration versionnée entre le score brut et le score affiché. Il s'agit d'une transformation monotone, linéaire par morceaux, définie par trois points d'ancrage

appris sur la partition de validation (Tableau 4.1). Elle préserve l'ordre des scores bruts jusqu'au plafond. Au-delà de 0,6126, en revanche, tous les scores affichés valent 100 : deux scores bruts distincts peuvent donc recevoir le même score affiché, et l'arrondi au dixième produit le même effet à plus petite échelle. Les AUC rapportées dans ce chapitre sont pour cette raison calculées sur les scores bruts, et le classement multi-CV départage les égalités par le score brut. La calibration laisse les sous-scores bruts inchangés dans l'explication. Chaque résultat enregistre le score brut, le score affiché, la version de calibration et les seuils appliqués.

Tableau 4.1: Ancrages de la calibration

Point d'ancrage	Score brut $S_{hybride}$	Score affiché
Origine	0	0
Frontière `REJECT`/`CONSIDER`	0,2633	55
Frontière `CONSIDER`/`HIRE`	0,3638	75
Plafond (maximum observé en validation)	0,6126	100

Les deux frontières sont choisies sur la validation en maximisant le macro-F1 du verdict à trois classes, comparé aux trois catégories du générateur : paires sans rapport, profils transférables et adéquations fortes. Pour l'exemple $(S_{\text{TFIDF}}, S_{\text{emb}}, S_{\text{skill}}) = (0{,}10; 0{,}55; 0{,}20)$, le score brut vaut 0,32 et le score affiché 66,3. Les méthodes isolées, sélectionnables dans l'interface, disposent chacune de leurs propres ancrages.

Le `DecisionEngine` applique ensuite au score affiché les bandes du Tableau 4.2, réglables par le recruteur. Une règle de prudence complète ces bandes : lorsque le texte recopie l'offre, ou lorsque moins de la moitié des compétences correspondantes sont démontrées, le verdict `HIRE` est remplacé par `CONSIDER`, avec une justification invitant à vérifier l'expérience. Le score affiché n'est pas modifié.

Tableau 4.2: Bandes de décision par défaut

Score affiché	Recommandation	Interprétation opérationnelle
$[75, 100]$	`HIRE`	adéquation forte à examiner en priorité
$[55, 75[$	`CONSIDER`	profil à examiner avec réserves explicites
$[0, 55[$	`REJECT`	adéquation insuffisante selon les signaux disponibles

Ces ancrages sont appris sur des étiquettes synthétiques. Ils alignent les bandes affichées sur les classes du banc d'essai ; ils ne constituent pas une politique d'embauche validée. La confiance présentée par le moteur est une distance au seuil le plus proche, et non une probabilité d'embauche.

4.7 Explicabilité et couche d'analyse locale

La couche 1 conserve les sous-scores, poids, décision, compétences trouvées ou absentes, compétences critiques et expérience détectée. L'interface peut donc formuler une justification contrôlable, par exemple l'absence de Docker et Kubernetes parmi les prérequis.

Après ce résultat, l'interface peut demander une analyse approfondie. Le service transmet alors le texte source et ce contexte déterministe à un modèle local configuré par défaut comme `qwen2.5:1.5b`, exécuté via Ollama. La réponse structurée propose trajectoire, alertes, justification, recommandations et questions d'entretien. Les états `disabled`, `unavailable`, `error` et `inconsistent` sont contrôlés sans bloquer la couche 1 ; une incohérence générative ne révise donc jamais la recommandation.

4.7.1 Construction du contexte et format de réponse

L'appel au modèle local n'est pas une requête libre. Le service assemble un contexte à partir du résultat déjà persistant : score calibré global, sous-scores des trois composantes, décision retenue et libellé de sa bande, compétences trouvées, compétences absentes et compétences critiques manquantes. Le modèle reçoit donc le verdict comme une donnée d'entrée à commenter, et non comme une question à trancher. Le texte du CV et celui de l'offre accompagnent ce contexte pour que l'analyse puisse s'appuyer sur le contenu réel du dossier.

La réponse attendue est un objet structuré comportant cinq parties : une lecture de la trajectoire professionnelle, les points d'attention, une justification du positionnement, des recommandations et des questions d'entretien. Cette contrainte de format sert deux objectifs. Elle rend la réponse exploitable par l'interface sans analyse de texte libre, et elle interdit au modèle de produire un score concurrent : aucun champ ne prévoit de note chiffrée ni de verdict. Une réponse qui ne se conforme pas au schéma attendu est rejetée plutôt que corrigée, et l'état correspondant est remonté à l'interface.

4.7.2 Garde-fou de cohérence et tentative d'injection

La contrainte de format ne suffit pas. Un petit modèle exécuté sur processeur peut respecter la structure demandée tout en énonçant une appréciation qui contredit la décision statistique : décrire comme un excellent profil un dossier que le moteur a classé en `REJECT` avec des compétences critiques absentes. Une telle réponse n'est pas seulement inexacte, elle est trompeuse, puisqu'elle apparaît à côté du verdict qu'elle dément.

Le service applique donc un contrôle de cohérence sur les cas de faible adéquation. Le texte produit est examiné à la recherche d'affirmations positives incompatibles avec le verdict, en tenant compte des négations pour éviter de sanctionner une formulation telle que « ce n'est pas un profil adapté ». Lorsqu'une contradiction est détectée, le service relance une fois le modèle avec un rappel explicite du verdict à respecter. Si la seconde réponse contredit encore la décision, l'analyse est abandonnée et l'état `inconsistent` est retourné : le système préfère ne rien afficher plutôt qu'afficher un commentaire qui affaiblirait la recommandation. C'est un comportement de défaillance fermée, cohérent avec le principe selon lequel la couche 2 ne peut jamais réviser la couche 1.

Ce garde-fou joue également un rôle de sécurité. Le texte d'un CV est fourni par un tiers et se

retrouve intégré au message envoyé au modèle ; un candidat averti pourrait y insérer des instructions destinées à l'influencer, par exemple une consigne affirmant qu'il s'agit d'un profil parfait et qu'aucune réserve ne doit être émise. L'audit de sécurité du projet a retenu ce scénario comme un risque réel. Deux propriétés le contiennent. D'une part, le score et la décision sont calculés par le moteur déterministe avant tout appel au modèle : une injection réussie ne peut pas modifier le verdict, seulement le commentaire qui l'accompagne. D'autre part, c'est précisément sur les dossiers faiblement classés qu'une injection aurait un intérêt, et c'est là que le contrôle de cohérence est actif : un texte trop favorable sur un dossier rejeté conduit au rejet de l'analyse. La protection n'est pas absolue, mais la surface exposée se limite à du texte consultatif, jamais à la recommandation elle-même.

4.8 Jeu de données et protocole expérimental

L'évaluation utilise `cv_job_dataset.csv`, généré localement avec la graine 42. Il contient 12 000 paires CV–offre synthétiques, plusieurs domaines et niveaux d'expérience, des textes français et anglais, des abréviations, fautes de frappe, compétences obsolètes et offres contradictoires. Les étiquettes positives regroupent les cas de forte et de transférable adéquation générés ; elles ne sont pas des décisions de recrutement observées.

4.8.1 Pourquoi un corpus synthétique

Le choix d'évaluer le moteur sur des documents générés plutôt que sur de vraies candidatures mérite d'être justifié, car il conditionne la portée de tous les chiffres qui suivent. Cinq raisons l'ont motivé.

La première est l'absence de corpus disponible. Aucun ensemble de CV réels appariés à des offres, en nombre suffisant et en accès autorisé, n'a pu être réuni dans le cadre de ce stage. Les quelques dossiers accessibles se comptaient en dizaines, ce qui n'aurait permis ni un partitionnement en trois sous-ensembles, ni une mesure stable : sur un échantillon aussi réduit, un écart de quelques paires suffit à déplacer un F1 de plusieurs points, et la comparaison entre quatre méthodes n'aurait plus été interprétable.

La deuxième raison est juridique et éthique. Un CV réel est une donnée à caractère personnel, et son usage dans un travail académique suppose une base légale, une information des personnes et une durée de conservation définie, comme le rappelle la section consacrée à ce sujet au chapitre 6. Construire un corpus de test à partir de candidatures authentiques aurait demandé des démarches que le cadre du projet ne permettait pas, et aurait introduit dans le dépôt des données qu'il aurait fallu ensuite protéger, anonymiser puis supprimer.

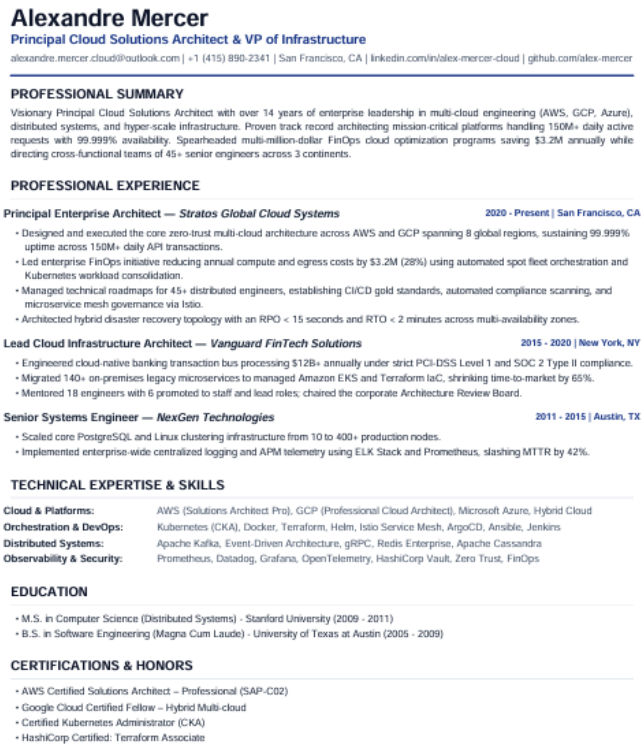
La troisième tient à la qualité d'extraction. Les CV réels circulent dans des formats hétérogènes, parfois exportés depuis un traitement de texte, parfois scannés, souvent mis en page sur plusieurs

colonnes ou dans des tableaux. L'extraction de texte y échoue partiellement : ordre de lecture inversé, colonnes fusionnées, caractères manquants, sections illisibles. Un échec de ce type se répercute sur les trois composantes du score. En travaillant sur des documents dont le contenu textuel est maîtrisé, l'évaluation mesure le rapprochement lui-même et non la qualité variable d'une chaîne d'extraction, dont le comportement est vérifié séparément par les tests de `parsing`.

La quatrième raison est l'absence de vérité de terrain exploitable. Même avec des CV réels, il aurait fallu disposer, pour chaque paire, d'un jugement fiable indiquant si le profil convient à l'offre. Ce jugement n'existe pas dans un dossier de candidature : une embauche dépend d'entretiens, de disponibilités et d'arbitrages qui ne figurent nulle part dans le document. Il aurait donc fallu faire annoter des milliers de paires par des recruteurs, ce qui dépassait largement le périmètre du stage. Le générateur, lui, connaît par construction la raison pour laquelle une paire est compatible ou non, ce qui fournit une étiquette cohérente avec le critère évalué.

La cinquième est la reproductibilité. Le corpus est produit par un script du dépôt avec une graine fixée à 42 : n'importe quel lecteur peut le régénérer à l'identique, relancer le banc d'essai et retrouver les valeurs du Tableau 4.3. Un jeu de CV réels, même anonymisé, n'aurait pas pu être distribué avec le code.

J'ai écrit le générateur pour ne pas produire des documents trop faciles. Chaque CV est assemblé à partir du dictionnaire de compétences du projet, avec un domaine et un niveau d'expérience, puis dégradé volontairement : abréviations, fautes de frappe, technologies obsolètes, formulations en français ou en anglais. Des paires contradictoires sont également générées, où le vocabulaire se recoupe alors que le profil ne correspond pas, précisément pour empêcher qu'un simple recouvrement de mots suffise. La figure 4.2 montre l'un de ces documents.



Alexandre Mercer
Principal Cloud Solutions Architect & VP of Infrastructure
alexandre.mercer.cloud@outlook.com | +1 (415) 890-2341 | San Francisco, CA | linkedin.com/in/alex-mercer-cloud | github.com/alex-mercer

PROFESSIONAL SUMMARY

Visionary Principal Cloud Solutions Architect with over 14 years of enterprise leadership in multi-cloud engineering (AWS, GCP, Azure), distributed systems, and hyper-scale infrastructure. Proven track record architecting mission-critical platforms handling 150M+ daily active requests with 99.999% availability. Spearheaded multi-million-dollar FinOps cloud optimization programs saving $3.2M annually while directing cross-functional teams of 45+ senior engineers across 3 continents.

PROFESSIONAL EXPERIENCE

Principal Enterprise Architect — *Stratos Global Cloud Systems* 2020 - Present | San Francisco, CA

- Designed and executed the core zero-trust multi-cloud architecture across AWS and GCP spanning 8 global regions, sustaining 99.999% uptime across 150M+ daily API transactions.
- Led enterprise FinOps initiative reducing annual compute and egress costs by $3.2M (28%) using automated spot fleet orchestration and Kubernetes workload consolidation.
- Managed technical roadmaps for 45+ distributed engineers, establishing CI/CD gold standards, automated compliance scanning, and microservice mesh governance via Istio.
- Architected hybrid disaster recovery topology with an RPO < 15 seconds and RTO < 2 minutes across multi-availability zones.

Lead Cloud Infrastructure Architect — *Vanguard FinTech Solutions* 2015 - 2020 | New York, NY

- Engineered cloud-native banking transaction bus processing $12B+ annually under strict PCI-DSS Level 1 and SOC 2 Type II compliance.
- Migrated 140+ on-premises legacy microservices to managed Amazon EKS and Terraform IaC, shrinking time-to-market by 65%.
- Mentored 18 engineers with 6 promoted to staff and lead roles; chaired the corporate Architecture Review Board.

Senior Systems Engineer — *NexGen Technologies* 2011 - 2015 | Austin, TX

- Scaled core PostgreSQL and Linux clustering infrastructure from 10 to 400+ production nodes.
- Implemented enterprise-wide centralized logging and APM telemetry using ELK Stack and Prometheus, slashing MTTR by 42%.

TECHNICAL EXPERTISE & SKILLS

Cloud & Platforms:	AWS (Solutions Architect Pro), GCP (Professional Cloud Architect), Microsoft Azure, Hybrid Cloud
Orchestration & DevOps:	Kubernetes (CKA), Docker, Terraform, Helm, Istio Service Mesh, ArgoCD, Ansible, Jenkins
Distributed Systems:	Apache Kafka, Event-Driven Architecture, gRPC, Redis Enterprise, Apache Cassandra
Observability & Security:	Prometheus, Datadog, Grafana, OpenTelemetry, HashiCorp Vault, Zero Trust, FinOps

EDUCATION

- M.S. in Computer Science (Distributed Systems) - Stanford University (2009 - 2011)
- B.S. in Software Engineering (Magna Cum Laude) - University of Texas at Austin (2005 - 2009)

CERTIFICATIONS & HONORS

- AWS Certified Solutions Architect – Professional (SAP-C02)
- Google Cloud Certified Fellow – Hybrid Multi-cloud
- Certified Kubernetes Administrator (CKA)
- HashiCorp Certified: Terraform Associate

Figure 4.2: Exemple de CV synthétique

Il faut néanmoins nommer ce que ce choix coûte, et le chapitre y revient dans sa section consacrée aux limites. Un document généré reste plus régulier qu'une candidature authentique : la longueur varie peu, la structure est constante, le vocabulaire provient d'un dictionnaire fini et aucune trajectoire atypique n'y figure. Les résultats obtenus mesurent donc la capacité du moteur à distinguer des profils sur un matériau contrôlé ; ils ne préjugent pas de son comportement sur des CV réels, dont la validation reste la première étape à mener avant tout usage opérationnel.

4.8.2 Partitionnement et métriques

La séparation est stratifiée par étiquette et domaine, puis vérifiée par groupe de paire CV–offre normalisée : 8 400 paires pour l'apprentissage, 1 800 pour la validation et 1 800 pour le test final. Les poids de fusion sont fixés avant l'évaluation, le vocabulaire TF–IDF est appris sur l'apprentissage seul, les seuils binaires de chaque méthode sont sélectionnés sur la validation en maximisant le F1, et les ancrages de calibration le sont en maximisant le macro-F1 à trois classes. Les métriques sont ensuite calculées sur le test isolé. Je n'ai choisi aucun poids, seuil ni ancrage sur le test. J'ai toutefois consulté ce test à plusieurs reprises au cours du projet : les chiffres finaux n'ont pas été réglés sur lui, mais il n'est plus strictement neuf, ce qui laisse un risque résiduel d'optimisme. Une alerte automatique signale toute méthode qui présenterait une AUC élevée et un F1 quasi nul, symptôme d'un seuil mal placé. Le partitionnement réduit la fuite de paires identiques, sans remplacer une validation externe réelle.

Les métriques sont l'exactitude, la précision, le rappel, le F1, l'AUC-ROC, l'aire précision–rappel, le kappa de Cohen et la matrice de confusion, selon les définitions de scikit-learn [19]. Deux familles sont distinguées : le F1 binaire, qui oppose les paires positives aux autres à un seuil donné, et le macro-F1 à trois classes, moyenne des F1 de `REJECT`, `CONSIDER` et `HIRE`, qui évalue les verdicts affichés. Ces deux mesures ne sont pas comparables entre elles. Le F1 équilibre précision et rappel :

$$F1 = 2\frac{\text{precision} \times \text{rappel}}{\text{precision} + \text{rappel}}.$$

4.9 Résultats et analyse comparative

Le Tableau 4.3 reproduit les résultats enregistrés sur les 1 800 paires de test. Les seuils affichés sont des points de fonctionnement expérimentaux et non les seuils `HIRE`/`CONSIDER`/`REJECT` de l'interface.

Tableau 4.3: Qualité du score brut (F1 binaire)

Méthode	Seuil	Exact.	Préc.	Rappel	F1	AUC-ROC
TF–IDF	0,04	0,93	0,93	0,97	0,95	0,98
Embeddings	0,54	0,78	0,81	0,89	0,85	0,81
Compétences	0,00	0,89	0,89	0,97	0,93	0,95
Hybride	0,26	0,91	0,94	0,93	0,93	0,96

La figure 4.3 présente les courbes ROC des scores bruts sur les mêmes paires. Elles mesurent la qualité du classement indépendamment de tout seuil : l'AUC de la fusion (0,964) reste inférieure à celle de TF–IDF (0,979), et les embeddings seuls classent nettement moins bien (0,811). Ces courbes ne disent rien des verdicts affichés, évalués plus loin.

La figure 4.4 relie le score brut de la fusion aux trois classes synthétiques et aux deux ancrages de la calibration, qui deviennent 55 et 75 sur l'échelle affichée. Les paires sans rapport se concentrent sous le premier ancrage et la plupart des adéquations fortes au-dessus du second. Les profils transférables chevauchent les deux autres classes, ce qui annonce la difficulté de la classe intermédiaire.

TF–IDF obtient le meilleur F1 dans ce protocole : $F1 = 0{,}95$ (intervalle bootstrap à 95 % : 0,941–0,959), contre $F1 = 0{,}93$ (0,924–0,944) et $AUC = 0{,}96$ pour la fusion. Sur les mêmes 1 800 paires, chacune lue à son propre seuil de validation, TF–IDF classe correctement 108 paires que la fusion manque, et l'inverse se produit pour 73 paires ; le test de McNemar donne $p \approx 0{,}011$. Ce protocole ne permet donc pas de revendiquer une supériorité mesurée de l'hybride : la différence observée va dans le sens inverse. Elle ne démontre pas pour autant une supériorité générale du lexical, car le générateur construit CV et offres à partir d'un vocabulaire contrôlé et partagé, ce qui favorise le recouvrement des termes. La fusion est retenue pour des raisons que ce benchmark ne mesure pas : la justification par compétences restituée au recruteur et une moindre sensibilité à la reformulation, observée sur l'expérience contrôlée du Tableau 4.5. Le temps moyen mesuré est 55,85 ms par paire

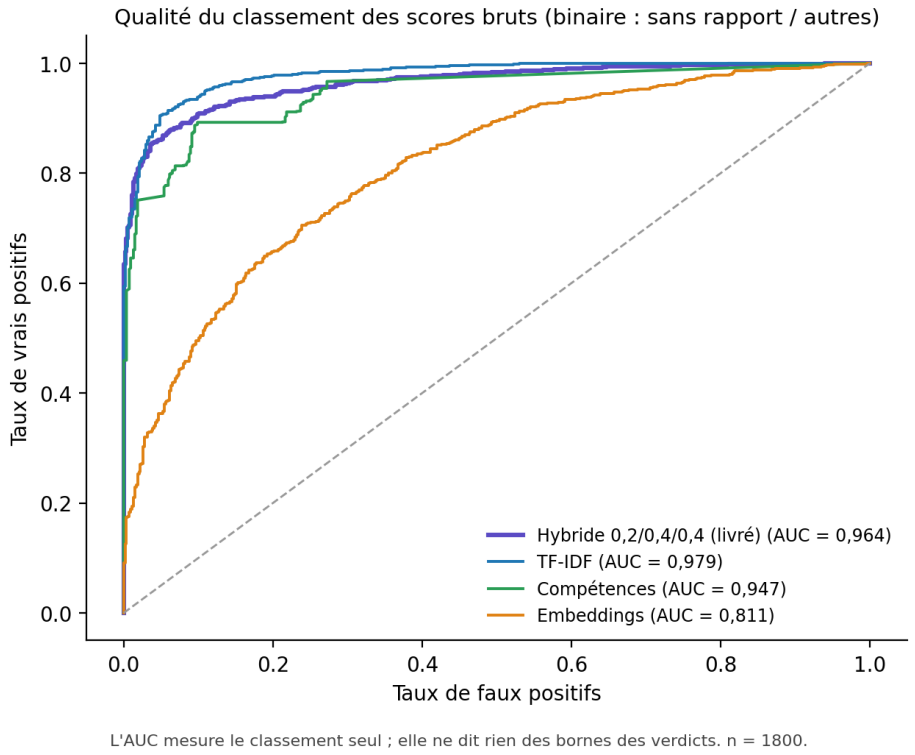


Figure 4.3: Courbes ROC des scores bruts

pour la fusion, contre 921,60 ms pour les embeddings seuls dans cet environnement.

Verdicts affichés par la version finale

Le Tableau 4.3 mesure la capacité de discrimination du score brut. Le Tableau 4.4 évalue ce que voit le recruteur, c'est-à-dire les verdicts `REJECT`, `CONSIDER` et `HIRE`, sur les mêmes 1 800 paires de test. Celles-ci comptent 536 paires sans rapport, 545 profils transférables et 719 adéquations fortes.

Tableau 4.4: Qualité des verdicts affichés

Configuration	Macro-F1 3 classes [IC 95 %]	F1 `HIRE`	F1 binaire `CONSIDER` ou mieux	Kappa q.	Verdicts R / C / H
Hybride calibré (version finale)	**0,740** [0,720–0,760]	0,782	0,934	0,799	548 / 557 / 695
TF–IDF seul, calibré	0,746 [0,726–0,765]	0,758	0,950	0,797	471 / 641 / 688
Fusion apprise 0,75 / 0,05 / 0,20, calibrée (non retenue)	0,780 [0,761–0,799]	0,792	0,957	0,833	552 / 560 / 688

La figure 4.5 compare la répartition des verdicts à celle des classes synthétiques. La version finale produit une répartition proche de celle des classes, sans que cette proximité suffise à garantir que chaque paire reçoive le bon verdict.

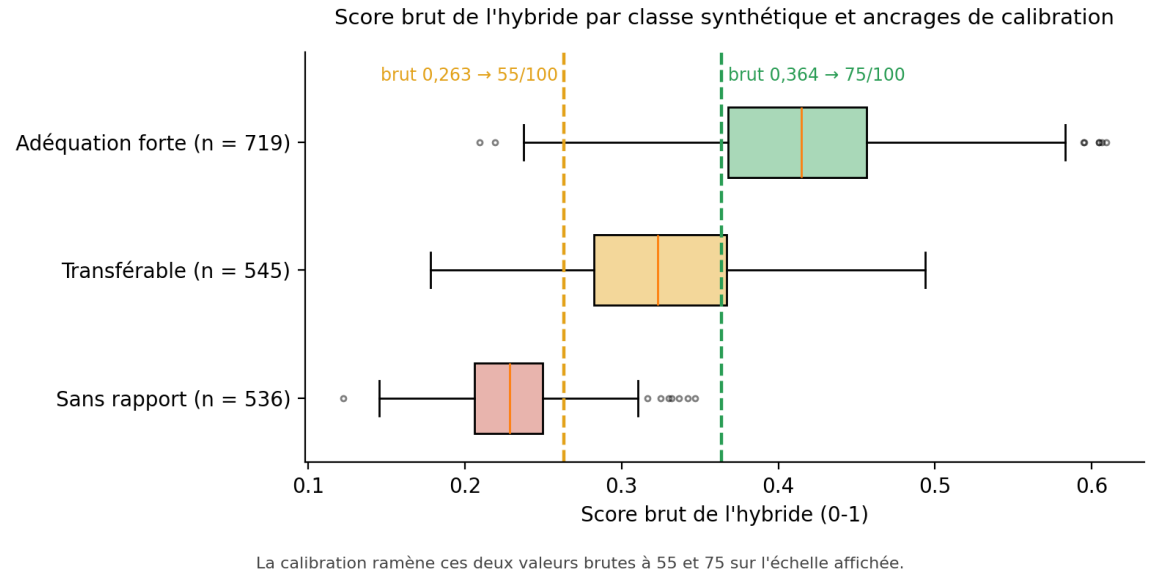


Figure 4.4: Score brut de la fusion par classe

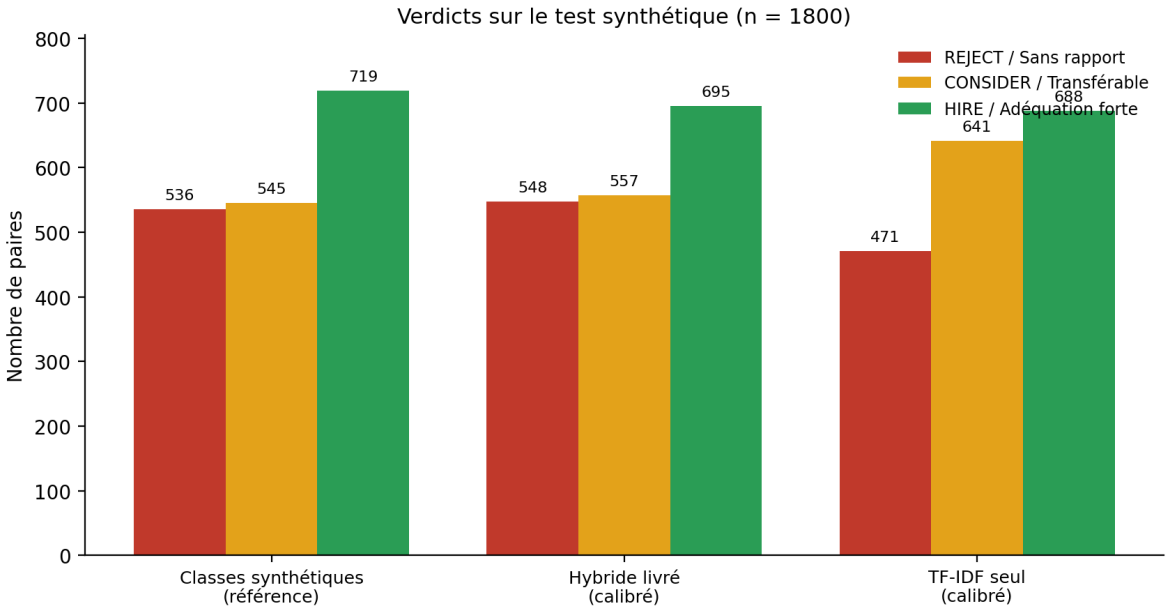


Figure 4.5: Répartition des verdicts

La matrice de confusion de la version finale (figure 4.6) montre que les erreurs se concentrent entre classes voisines. Les paires sans rapport sont bien identifiées (459 `REJECT` sur 536, aucune `HIRE`), et 553 des 719 adéquations fortes reçoivent `HIRE`. La classe intermédiaire reste la plus difficile (F1 de 0,59) : 325 profils transférables sur 545 reçoivent `CONSIDER`, 142 `HIRE` et 78 `REJECT`. Une comparaison appariée par bootstrap sur les mêmes paires ne montre pas de différence entre l'hybride calibré et TF–IDF seul (écart de −0,006, IC à 95 % de −0,028 à 0,016). Les paires multilingues (CV anglais, offre française, 120 paires) sont moins bien traitées que les paires françaises (exactitude de 0,65 contre 0,75).

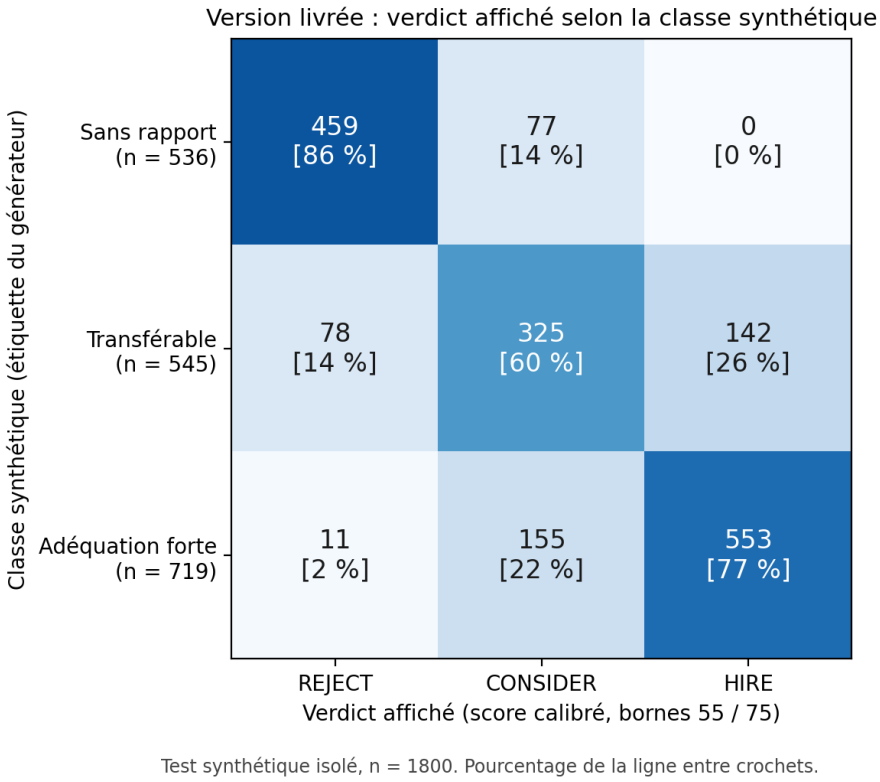


Figure 4.6: Matrice de confusion des verdicts

Choix de la pondération

Une fusion dont les poids sont appris sur l'apprentissage seul (0,75 / 0,05 / 0,20) obtient le meilleur macro-F1 en validation comme en test, et la règle de sélection fixée avant la lecture du test la désignait. Elle n'est pas retenue pour le produit, pour deux raisons. D'abord, son avantage provient d'un corpus dont les étiquettes sont construites à partir du recouvrement des compétences recopiées dans l'offre, ce qui favorise mécaniquement le signal lexical. Ensuite, une pondération dominée par TF–IDF récompense la simple reprise du vocabulaire de l'annonce. Sur une paire contrôlée de CV de même substance, l'un calqué sur l'offre et l'autre reformulé, une pondération 0,9 / 0,0 / 0,1 donnait un écart de 25,6 points, contre 9,6 points avec la pondération documentée (Tableau 4.5). La version finale conserve donc la pondération documentée 0,2 / 0,4 / 0,4 ; la calibration de la fusion apprise est archivée avec le code.

Tableau 4.5: Effet de la reformulation selon la pondération

Pondération du moteur	CV calqué sur l'offre	CV reformulé	Écart
0,9 / 0,0 / 0,1	99,2	73,6	25,6 points
0,2 / 0,4 / 0,4 (documentée)	94,4	84,8	9,6 points

Robustesse face aux CV manipulés

Le Tableau 4.6 mesure l'effet du contrôle des preuves sur des CV de la partition de validation modifiés de façon contrôlée, à raison de 200 paires par scénario. Les cinq premiers scénarios partent de profils sans rapport avec l'offre ; les deux derniers partent d'adéquations fortes. Les deux colonnes utilisent la même calibration ; seule change la présence du contrôle des preuves. Je n'ai ajusté aucun paramètre sur ces scénarios.

Tableau 4.6: Effet du contrôle des preuves

Scénario	Sans contrôle	Version finale
Référence : profils sans rapport non modifiés	174 / 25 / 1	
Profil sans rapport + liste d'exigences de l'offre recopiée	0 / 0 / 200	133 / 67 / 0
Profil sans rapport + mêmes compétences, réordonnées, dans une section Compétences	0 / 0 / 200	0 / 200 / 0
Profil sans rapport + offre entière collée	0 / 0 / 200	174 / 26 / 0
Profil sans rapport + mention niée des compétences	0 / 0 / 200	175 / 24 / 1
Profil sans rapport + compétences insérées dans une phrase d'expérience	0 / 0 / 200	0 / 0 / 200
Référence : adéquations fortes non modifiées	1 / 33 / 166	
Adéquation forte privée de la moitié de ses compétences	15 / 129 / 56	15 / 129 / 56
Adéquation forte allongée par des expériences sans rapport	0 / 16 / 184	0 / 16 / 184

Le contrôle ramène les listes recopiées, l'offre collée et les mentions niées au niveau de la référence non modifiée : aucun CV à liste recopiée ou à offre collée n'obtient `HIRE`, et le seul `HIRE` restant parmi les mentions niées correspond à un profil déjà mal classé avant modification. Une liste réordonnée placée dans une section Compétences n'est pas reconnue comme copie, mais la règle de prudence ramène le verdict à `CONSIDER`. Les CV partiels et les CV longs ne sont pas affectés, et aucun verdict des 1 800 paires authentiques de validation ne change. En revanche, un CV qui affirme une expérience dans une phrase rédigée reste classé `HIRE` : l'analyse du texte ne peut pas distinguer une déclaration mensongère d'une expérience réelle. Cette limite justifie la lecture humaine du dossier et les questions d'entretien proposées par la couche 2.

Sur le déploiement Azure CPU, un essai de couche 2 a produit les cinq sections attendues en 54 290,6 ms. Cette mesure ponctuelle décrit la démonstration, non une garantie de latence ou une évaluation de qualité générative.

4.10 Limites, risques et perspectives

Les résultats quantitatifs sont internes à un corpus synthétique et à des étiquettes générées par règles ; les ancrages de calibration sont appris sur ces étiquettes. Les essais que j'ai menés avec mes propres CV face à des offres réelles, présentés au chapitre 6, sont fonctionnels : faute de jugements de référence

établis par des recruteurs, ils ne permettent de calculer ni précision, ni rappel, ni F1. Les mesures ne couvrent ni la qualité d'une embauche, ni les effets sur les candidats, ni l'équité entre groupes protégés. Un CV qui décrit faussement une expérience n'est pas détecté. Le dictionnaire peut omettre une technologie ou mal qualifier un intitulé ambigu ; les embeddings peuvent rapprocher des contenus trop généraux ; le modèle local peut produire une justification plausible mais erronée. JobTest doit donc rester une aide à la priorisation, avec révision humaine, traçabilité des critères et possibilité de correction.

Les suites nécessaires sont une validation avec des recruteurs sur des données autorisées et minimisées, la mesure de l'accord inter-évaluateurs, des tests de biais et de robustesse par sous-groupes, ainsi qu'un réapprentissage des ancrages de calibration sur ces données annotées. L'évaluation du modèle de langage doit être séparée de celle du score : fidélité aux faits, utilité des questions et taux d'incohérences sont des axes distincts.

Le tableau 4.7 récapitule le rôle de chaque couche et le garde-fou qui lui est associé.

Tableau 4.7: Synthèse des couches

Couche	Rôle	Statut et garde-fou
Couche 1	score hybride calibré, contrôle des preuves, décision et explications	implémentée ; décision déterministe ; `HIRE` écarté si les preuves sont insuffisantes
Couche 2	analyse locale et questions d'entretien	optionnelle ; ne change jamais la couche 1
Évaluation	benchmark train/validation/test synthétique	disponible ; ancrages appris sur étiquettes synthétiques ; non représentatif d'une embauche réelle

Conclusion

Le moteur combine signaux lexicaux, sémantiques et structurés, contrôle les preuves avant le calcul et calibre son score avant d'appliquer les bandes de décision. Sur le test synthétique, les verdicts affichés atteignent un macro-F1 de 0,740 sur trois classes, sans dépasser TF–IDF seul, et les CV qui recopient l'offre n'obtiennent plus `HIRE`. La calibration devra être réapprise sur des données réelles annotées avant tout usage opérationnel. Le chapitre suivant présente la réalisation de la plateforme.

Chapitre 5

Réalisation et présentation de la plateforme

Plan

1 Environnement matériel et logiciel 58

2 Choix des technologies 59

3 Organisation du code source 61

4 Réalisation du backend 62

5 Réalisation du frontend 62

6 Intégration frontend–backend 63

7 Présentation des interfaces 64

8 Parcours fonctionnel de démonstration 80

9 Difficultés de réalisation et solutions apportées 80

Introduction

Ce chapitre décrit le passage de la conception à une plateforme utilisable par un recruteur. L'enjeu n'est pas de reprendre l'architecture logique ni les calculs présentés précédemment, mais de montrer comment ils sont rendus accessibles par une application Web cohérente : saisie des données, exécution d'une évaluation, restitution expliquée et conservation des résultats. Les éléments décrits ci-après proviennent de la version courante du dépôt JobTest ; l'ancienne interface Streamlit est donc considérée comme un prototype et non comme l'interface finale.

5.1 Environnement matériel et logiciel

5.1.1 Environnement matériel

J'ai réalisé le développement, les validations locales et la rédaction de ce mémoire sur mon ordinateur personnel, décrit ci-dessous. Cet environnement local est distinct de la machine virtuelle Azure décrite au chapitre 6 ; aucune mesure de performance matérielle n'est déduite de ces caractéristiques.

Le tableau 5.1 en donne les caractéristiques.

Tableau 5.1: Environnement matériel

Composant	**Caractéristique**
Ordinateur	HP
Processeur	Intel Core i3, 11e génération
Mémoire vive	8 Go de RAM
Processeur graphique	Intel UHD Graphics
Stockage total	Environ 1 300 Go

5.1.2 Environnement logiciel

Le développement associe un service Python et une application Web Next.js. L'environnement local s'appuie sur Windows, Python, Node.js et `npm`. L'API est exécutée avec Uvicorn ; le navigateur sert à vérifier les parcours de l'interface. Git est utilisé pour le suivi des sources. Les versions expressément verrouillées par le projet sont indiquées ; aucune version non vérifiable de système, d'éditeur ou de navigateur n'est inférée.

Le tableau 5.2 récapitule ces outils et le rôle tenu par chacun.

Tableau 5.2: Environnement de réalisation

Élément	**Outil ou technologie**	**Rôle dans le projet**
Système local	Windows	Développement et exécution locale des services.
Backend	Python, FastAPI et Uvicorn	Exposition de l'API, services métier et exécution ASGI.
Frontend	Next.js 14.2.35, React 18, TypeScript	Pages App Router, composants et interactions du recruteur.
Styles et animation	Tailwind CSS, Framer Motion	Système visuel, états de l'interface et transitions.
Données locales	SQLite et SQLAlchemy	Persistance locale des comptes, offres, candidats, CV et évaluations.
Outils IA	scikit-learn, Sentence-Transformers, spaCy, Ollama	Moteur de rapprochement et analyse qualitative optionnelle.
Documents	ReportLab	Construction du rapport PDF d'évaluation.
Conteneurs	Docker Compose, Caddy, PostgreSQL	Environnement de déploiement ; détaillé au chapitre 6.

5.2 Choix des technologies

Les choix présentés dans cette section sont justifiés par les responsabilités effectivement portées par chaque partie de JobTest.

5.2.1 Technologies frontend

Le répertoire `web/` contient le frontend final. Next.js organise les écrans dans `app/`, selon l'App Router, et React porte les composants réutilisables. TypeScript définit les contrats manipulés par le client API et évite que les données de réponse soient traitées comme des objets non structurés. Les routes principales correspondent notamment à la connexion, à la vue d'ensemble, aux offres, aux candidats, à l'analyse, à l'historique, au classement et au centre de qualité.

Tailwind CSS est employé pour composer les styles de l'interface et les variantes des composants communs, tels que boutons, cartes, tableaux, badges, champs et états vides. Framer Motion est utilisé pour les transitions de pages, l'apparition du résultat et les animations de score. Le projet référence également Three.js, `@react-three/fiber` et `@react-three/drei` ; leur emploi est concret dans les éléments graphiques `AmbientBackground`, `ScoreTorus` et `NeuralField`. Ces éléments complètent la

lecture visuelle du score sans porter la décision elle-même.

5.2.2 Technologies backend

FastAPI regroupe l'API sous le préfixe `/api/v1`. L'application installe les routeurs d'authentification et le routeur `/api/v1/ui`, destiné aux besoins du frontend actuel. Les modèles Pydantic définissent et valident les corps des requêtes, par exemple une offre, une évaluation ou une demande de rapport. La séparation rend explicites les entrées HTTP avant l'appel aux services de rapprochement.

SQLAlchemy représente les entités *User*, *Candidate*, *CV*, *JobOffer*, *MatchResult* et *Skill*. Les relations lient le candidat à ses CV et ses résultats, et le recruteur à ses propres candidats et offres. Les contrôles de propriété appliqués par l'API empêchent qu'un identifiant d'offre ou de candidat appartenant à un autre recruteur soit utilisé dans une évaluation.

Le parsing des CV repose sur `pdfplumber` avec repli `PyPDF2` pour les PDF, et sur `python-docx` avec repli XML pour les documents Word. ReportLab génère le PDF à partir des données déjà calculées : en-tête, identité affichée, score, radar, compétences, écarts, recommandations et aide à la décision. La génération reste ainsi découplée de l'affichage Next.js.

L'authentification s'appuie sur des jetons JWT transmis dans l'en-tête `Authorization: Bearer`. Le client conserve le jeton dans le stockage local du navigateur et l'ajoute aux appels protégés. Les mots de passe ne sont pas retournés par les endpoints ; seul le hachage est conservé dans le modèle utilisateur.

5.2.3 Technologies d'intelligence artificielle

Le premier niveau d'analyse mobilise les composants décrits au chapitre 4 : TF–IDF de scikit-learn, similarité sémantique avec Sentence-Transformers, extraction et rapprochement de compétences, puis fusion hybride. spaCy intervient lorsqu'un modèle linguistique est disponible, avec une stratégie de repli dans le prétraitement. Le chargement des services est différé jusqu'à la première requête nécessitant le moteur, ce qui permet à l'API de démarrer sans calculer immédiatement les modèles.

La seconde couche qualitative est appelée après l'affichage du résultat principal. Le runtime local Ollama est configuré avec `qwen2.5:1.5b`. Son indisponibilité n'empêche pas l'évaluation statistique : la réponse expose alors un état contrôlé tel que *unavailable*, *disabled* ou *error*. Cette distinction est importante pour l'utilisateur : l'analyse approfondie enrichit la lecture, mais ne modifie ni le score ni le verdict du premier niveau.

5.2.4 Technologies de données et d'infrastructure

SQLite constitue le choix par défaut en développement local grâce à une base fichier et à l'intégration SQLAlchemy. Pour l'environnement conteneurisé, le fichier `docker-compose.yml` définit PostgreSQL

16 comme service de données. Docker Compose réunit également l'API, l'application Web, Ollama et le proxy inverse. Dans la version déployée, ce proxy est Caddy, point d'entrée HTTPS ; il relaie les chemins `/api/v1/` vers FastAPI et les autres chemins vers Next.js. La procédure complète de sécurisation, de conteneurisation et de déploiement sur machine virtuelle Azure est réservée au chapitre 6.

Le tableau 5.3 synthétise ces choix et l'usage qui en est réellement fait.

Tableau 5.3: Choix techniques

Domaine	**Technologie**	**Utilisation réelle dans JobTest**
Interface	Next.js, React, TypeScript	Écrans App Router, composants et client HTTP.
Présentation	Tailwind CSS, Framer Motion, Three.js	Styles, transitions et représentations visuelles du score.
API	FastAPI, Pydantic	Endpoints, validation et sérialisation JSON.
Persistance	SQLAlchemy, SQLite / PostgreSQL	Entités métier et stockage selon l'environnement.
Rapprochement	scikit-learn, Sentence-Transformers, spaCy	Signaux lexical, sémantique et compétences.
Analyse qualitative	Ollama, Qwen	Analyse approfondie non bloquante.
Export	ReportLab	Rapport PDF téléchargeable.

5.3 Organisation du code source

L'organisation privilégie la séparation entre interface, API, règles de rapprochement et accès aux données. L'arborescence simplifiée suivante ne montre que les dossiers participant directement au fonctionnement de la version étudiée.

```
web/                pages Next.js, composants, client API et styles
src/api/            application FastAPI et routeurs auth / UI
src/matching/       service d'évaluation et persistance du résultat
src/models/         TF-IDF, embeddings, compétences et chargement
src/fusion/         score hybride
src/decisioning/    règles de verdict
src/explainability/ explication et recommandations structurées
src/ai/             prompts et analyse qualitative Ollama
src/parsing/        extraction PDF/DOCX et nettoyage
src/reporting/      génération ReportLab
```

`src/models_db.py`	modèles SQLAlchemy
`deploy/Caddyfile`	proxy inverse HTTPS

Le frontend ne recopie pas les règles du moteur. Son fichier `lib/api/client.ts` centralise les appels HTTP et les types associés. Côté API, le routeur de parité UI traduit les requêtes de l'interface vers le même service métier que les endpoints techniques. Cette organisation limite les divergences entre un calcul réalisé depuis un écran et un calcul réalisé par API.

5.4 Réalisation du backend

L'initialisation FastAPI crée les tables lorsqu'elles sont absentes et applique une vérification additive de schéma. Les routeurs couvrent l'inscription, la connexion, la lecture de l'utilisateur courant, les offres, les candidats, les résultats, l'évaluation, l'analyse approfondie, la comparaison des modèles, les indicateurs et le rapport PDF. Un middleware ajoute un identifiant de requête, mesure le temps de traitement et applique une limitation de fréquence en mémoire.

Le routeur UI fournit les opérations nécessaires aux pages : lecture et création d'offres, modification de leur statut, consultation et modification des candidats, lecture d'historique, évaluation textuelle ou par fichier, et lecture des indicateurs. Lors d'une évaluation, l'API vérifie l'offre sélectionnée, le contenu minimal du CV et l'appartenance des données au recruteur connecté. Le service `evaluate_and_persist` calcule le résultat, crée ou retrouve le candidat selon les données fournies, conserve le CV et enregistre les sous-scores, le verdict, les seuils actifs et l'explication JSON.

La route de rapport reçoit les objets déjà calculés et renvoie un flux `application/pdf` avec un en-tête de téléchargement. Ce choix évite de recalculer le matching lors de l'export. De manière analogue, la route d'analyse approfondie reçoit l'identifiant d'un résultat persistant, reconstruit son contexte et renvoie une enveloppe d'état sans faire échouer le parcours lorsque le LLM n'est pas accessible.

5.5 Réalisation du frontend

La zone authentifiée est enveloppée par un layout commun : barre latérale, barre supérieure, transition de page et zone centrale. Au chargement, le layout vérifie la présence du jeton puis appelle `/auth/me`. En cas d'échec, il redirige vers la connexion. Cette vérification protège l'expérience utilisateur au niveau de l'interface, tandis que l'API vérifie de nouveau le jeton pour chaque ressource protégée.

Les pages utilisent des composants génériques pour maintenir une restitution homogène. `Table` porte les listes, `Modal` les opérations de modification ou de confirmation, `Field` les saisies, et `DataStates` les états de chargement ou d'absence de données. Les pages d'offres et de candidats affichent leurs données sous forme de tableau et appellent le client centralisé pour actualiser leur état

après une modification. Les erreurs réseau ou fonctionnelles sont converties en notifications visibles ; le bouton de l'évaluation affiche également un état de travail pendant le calcul.

La page d'analyse regroupe la sélection d'une offre ouverte, le choix de méthode, l'identité facultative du candidat et deux modes d'entrée du CV : collage de texte ou téléversement PDF/DOCX. Après la réponse JSON, elle révèle le score, le verdict, les composantes, le radar de compétences, les forces, lacunes et recommandations. L'export PDF transmet les données du résultat vers le backend. Le composant `DeepAnalysis` déclenche séparément la couche qualitative et présente son état sans masquer la restitution du premier niveau.

5.6 Intégration frontend–backend

Le parcours d'une évaluation relie deux responsabilités complémentaires. Le navigateur assure la collecte et la restitution ; le serveur possède les règles métier et les données persistantes.

1. Le recruteur choisit une offre et saisit ou téléverse le CV dans l'écran *Candidate Analysis*.
2. Le frontend contrôle les informations requises et construit une requête vers `/api/v1/ui/evaluate` ou `/api/v1/ui/evaluate/file`.
3. Le client API ajoute le jeton bearer, puis FastAPI valide le schéma Pydantic, l'utilisateur et la propriété de l'offre.
4. Le service métier extrait éventuellement le document, exécute le moteur de niveau 1 et produit explication, recommandations et décision.
5. SQLAlchemy conserve le candidat, le CV et le résultat lorsque le parcours est persistant.
6. La réponse JSON est affichée par les composants de score ; l'analyse approfondie est demandée indépendamment après cette réponse.
7. Sur demande, le frontend envoie les éléments nécessaires à `/api/v1/report/pdf` et télécharge le document retourné.

En développement, le client pointe vers l'API locale configurée par `NEXT_PUBLIC_API_BASE`. Dans le déploiement, le même chemin relatif `/api/v1` est servi derrière le proxy inverse, ce qui évite d'exposer au navigateur les adresses internes des conteneurs. Les réponses d'erreur HTTP sont transformées en messages d'interface ; l'état particulier de la Layer 2 est, lui, affiché comme une disponibilité de service et non comme un échec de l'évaluation principale.

5.7 Présentation des interfaces

J'ai réalisé les captures de cette section sur une instance locale exécutant la version finale du code (modèle `hybrid-checkpoint-v4` et calibration versionnée livrée). Je les ai produites avec le navigateur Chrome, piloté par un script du dépôt (`scripts/capture_screens.py`), dans une fenêtre de 1 440 × 900 pixels, sans autre retouche que le masquage des dates. Pour la lisibilité, la barre de navigation latérale, identique sur toutes les pages et visible sur la figure 5.4, est recadrée sur les autres écrans de l'application. Le jeu de démonstration est entièrement fictif : compte recruteur de test, offres rédigées pour l'occasion, candidats dotés d'adresses `example.com` et dix-huit CV fictifs au format PDF. Aucun CV personnel n'y figure.

5.7.1 Connexion et identité visuelle

La page de connexion comporte les champs courriel et mot de passe, un lien de création de compte et une notification d'erreur en cas d'échec. Les champs sont vides à l'ouverture. La partie visuelle présente l'identité JobTest et le rôle d'aide au recrutement. Après authentification, l'utilisateur est dirigé vers la vue d'ensemble (figure 5.1).

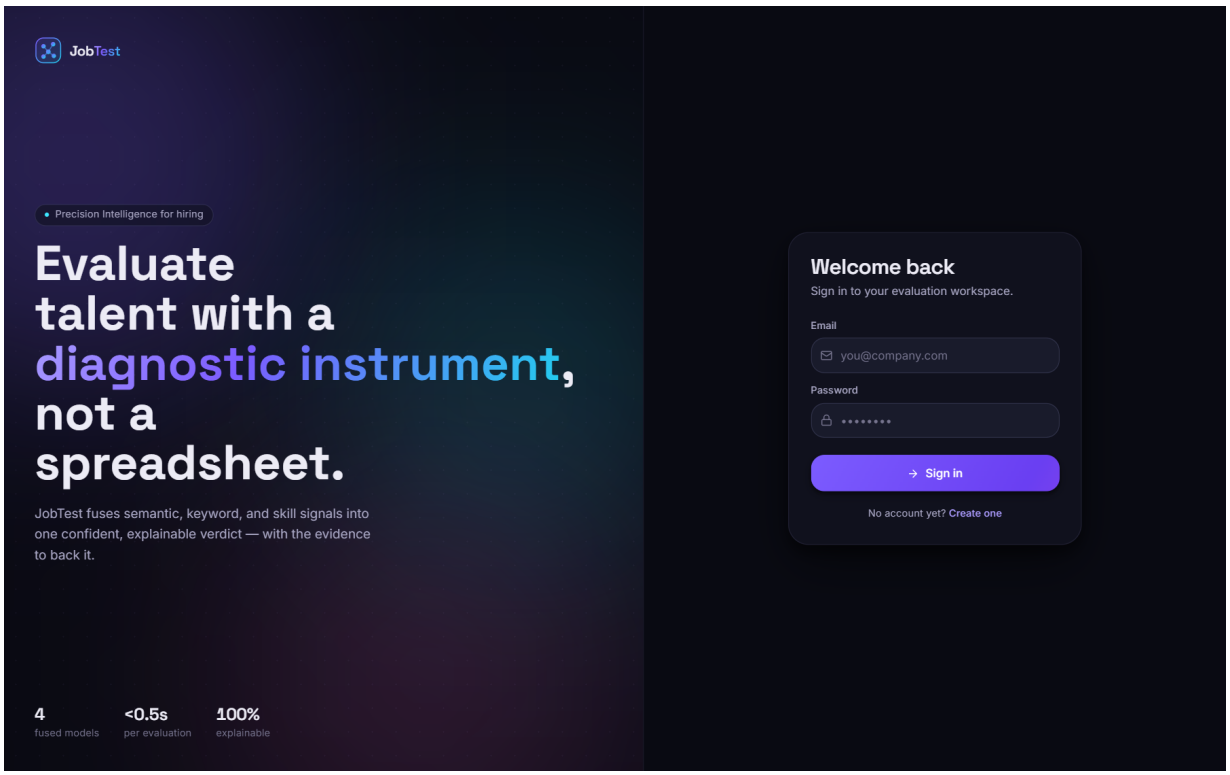


Figure 5.1: Écran de connexion

L'inscription est proposée sur ce même écran ; elle demande le nom, le courriel, le mot de passe et sa confirmation (figure 5.2).

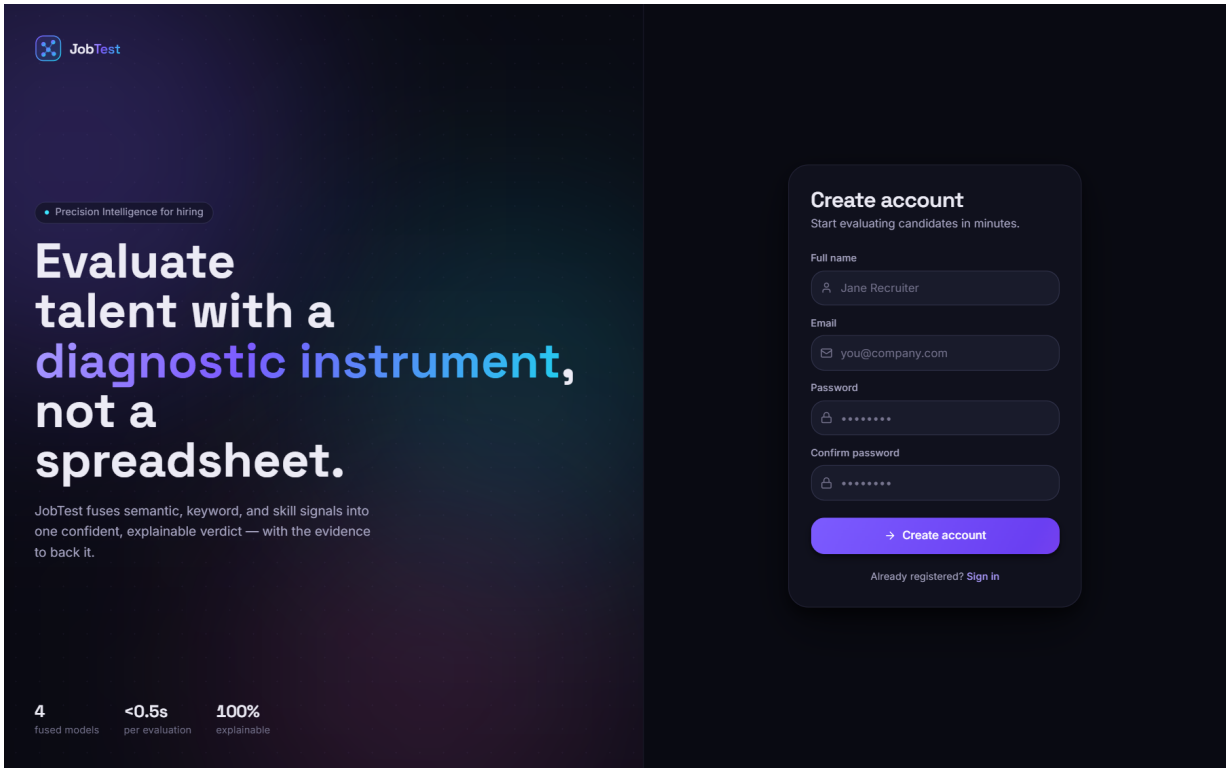


Figure 5.2: Écran d'inscription

5.7.2 Consultation sur mobile

La mise en page s'adapte aux écrans étroits : la barre latérale laisse place à une navigation horizontale et les grilles passent sur une colonne, sans modifier les règles métier ni les résultats fournis par l'API. La figure 5.3 montre la vue d'ensemble et l'historique à la largeur d'un téléphone, obtenus par l'émulation mobile du navigateur (390 pixels de large).

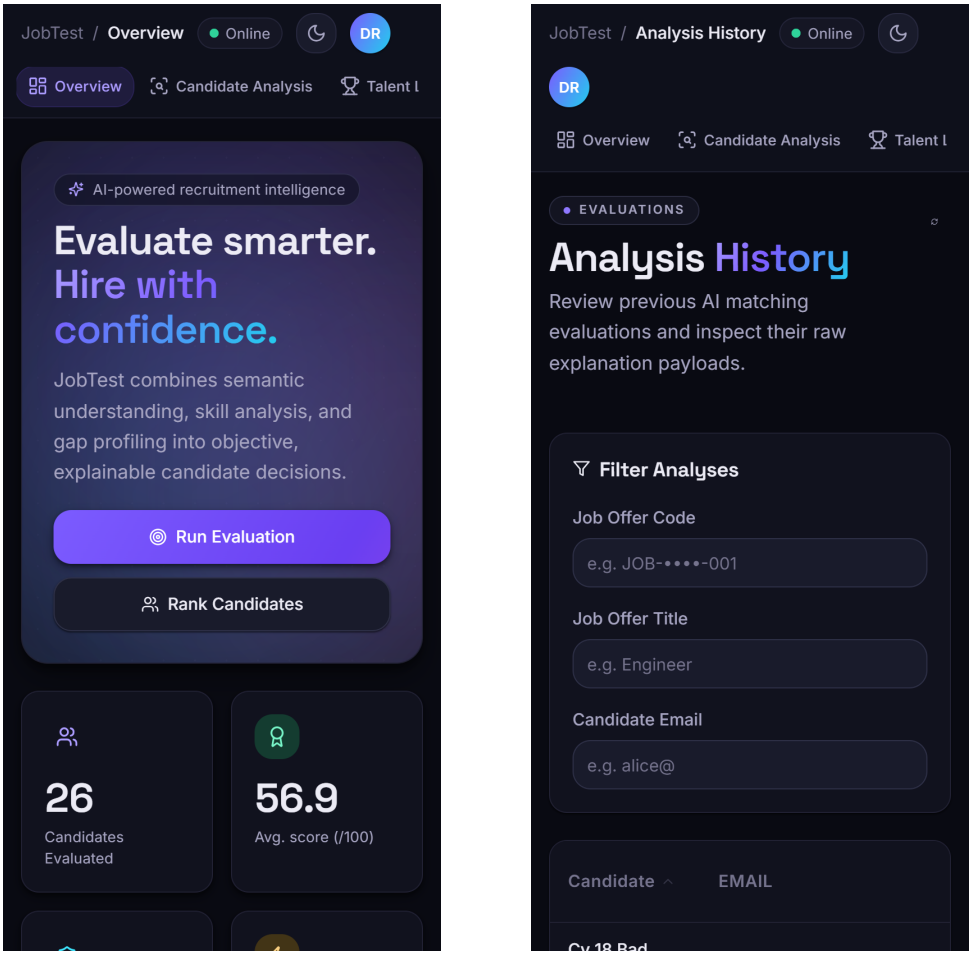


Figure 5.3: Affichage en largeur mobile

5.7.3 Vue d'ensemble, offres et candidats

La vue *Overview* synthétise le nombre de candidats, le nombre d'offres, le nombre d'évaluations, le score calibré moyen sur 100 ainsi que la part des verdicts `HIRE`, puis les résultats récents et la répartition des verdicts. Ces valeurs sont calculées côté API à partir des objets appartenant au recruteur connecté, et seuls les résultats calculés sur l'échelle calibrée entrent dans les moyennes (figure 5.4).



Figure 5.4: Vue d'ensemble

La figure 5.5 illustre la recherche transversale accessible depuis la barre supérieure (raccourci `Ctrl+K`).

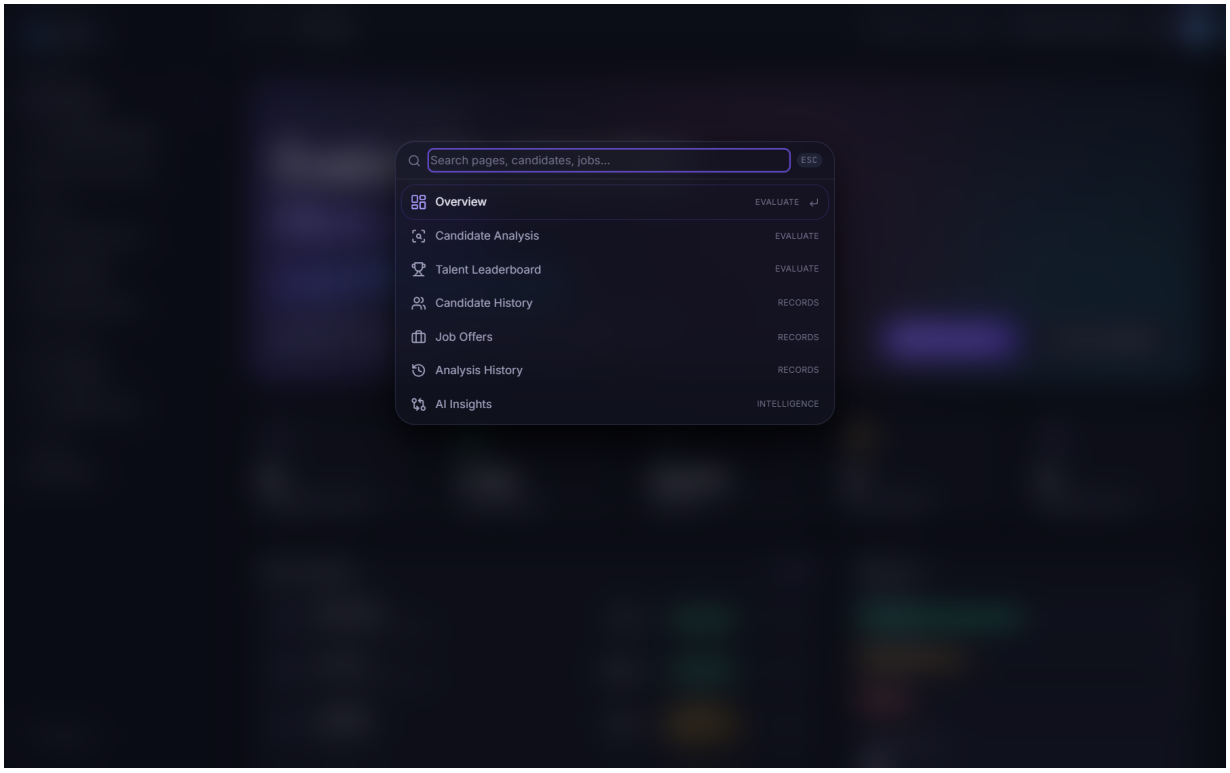


Figure 5.5: Recherche transversale

La page *Job Offers* permet de créer, consulter, modifier, fermer ou rouvrir une offre. Chaque offre possède un code, un titre, une description, des compétences demandées et un statut parmi *OPEN*,

ON_HOLD et *CLOSED* (figures 5.6 et 5.7).

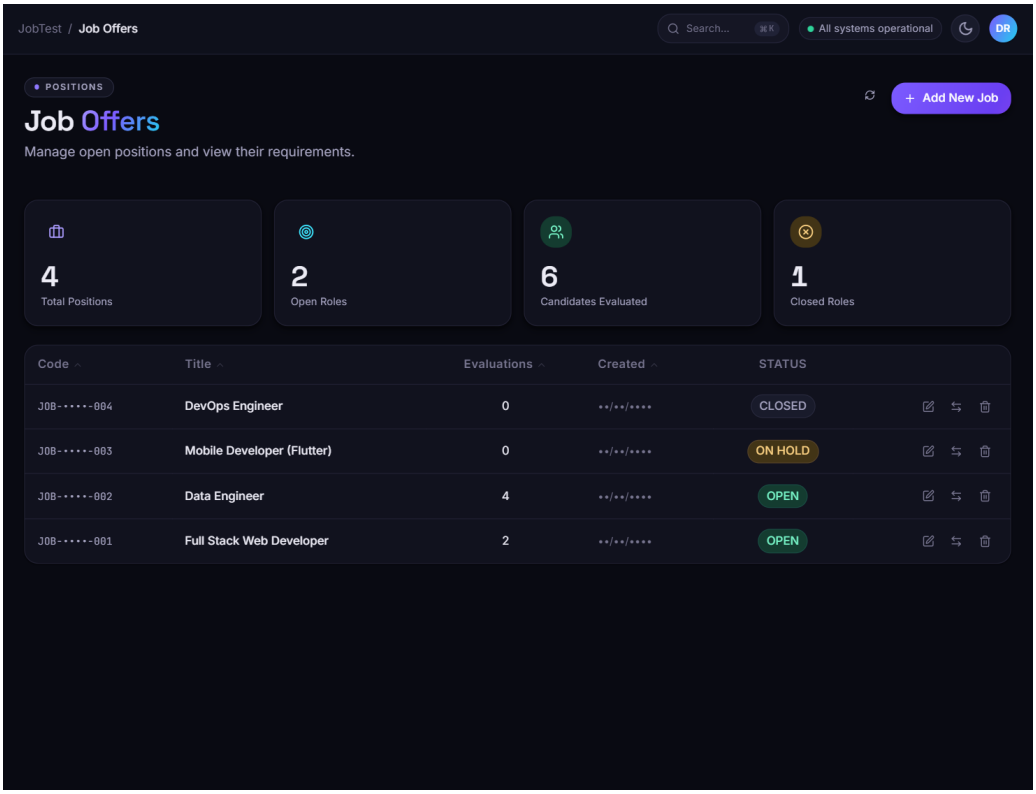


Figure 5.6: Liste des offres d'emploi

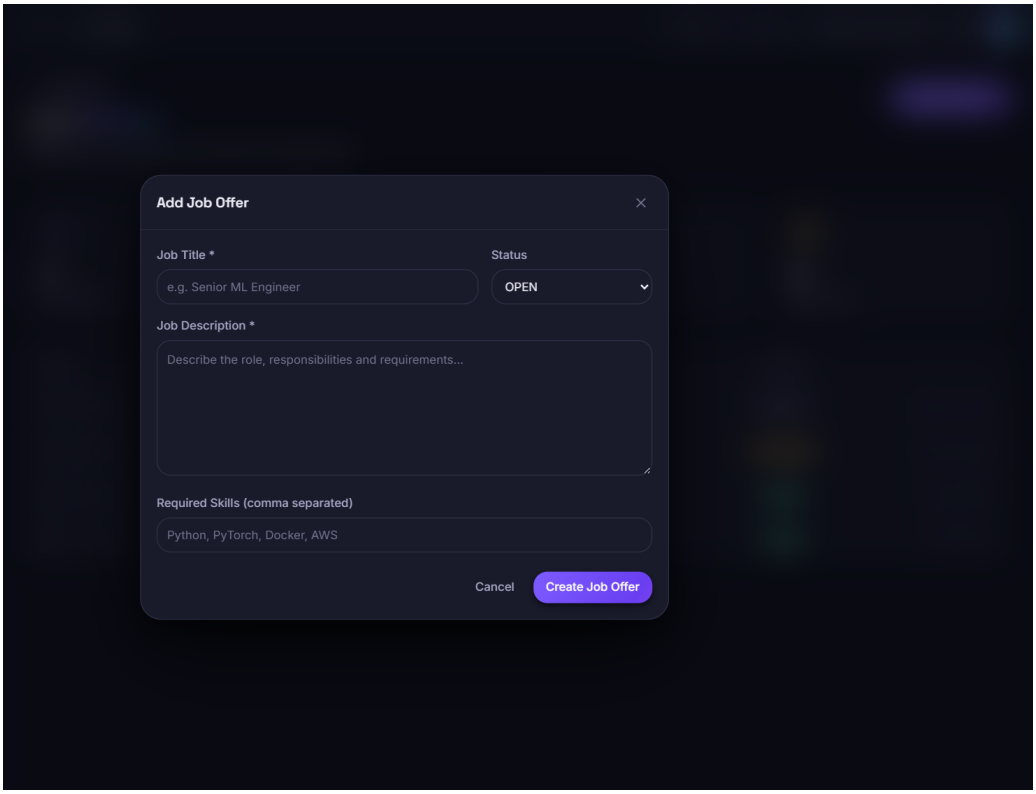


Figure 5.7: Création d'une offre d'emploi

La page *Candidate History* distingue les candidats des comptes recruteurs. Elle affiche les informations disponibles, l'état du parcours, le nombre d'évaluations et le meilleur score calibré de chaque candidat ; la modification et la suppression sont des actions explicites (figure 5.8).

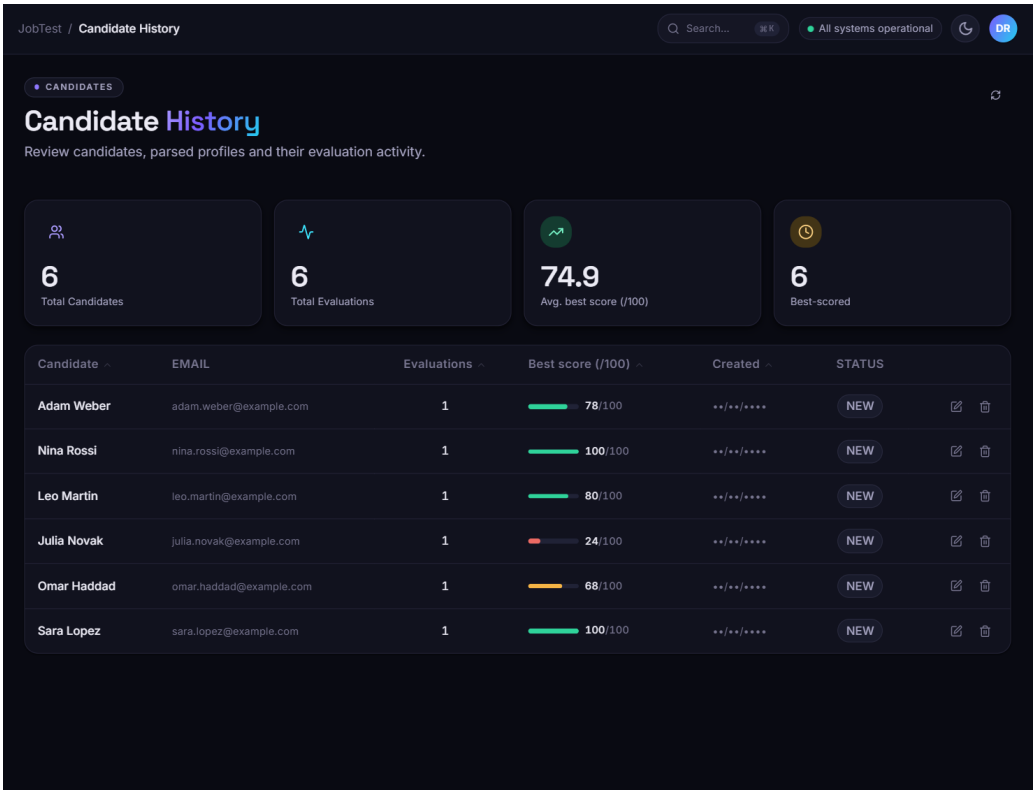


Figure 5.8: Suivi des candidats

L'historique des évaluations liste chaque résultat persisté avec son offre, son score sur 100 et son verdict. Le panneau de détail rappelle la version de calibration, le score brut de la méthode et les seuils appliqués lors du calcul ; un résultat calculé sur l'ancienne échelle y serait signalé comme tel (figure 5.9).

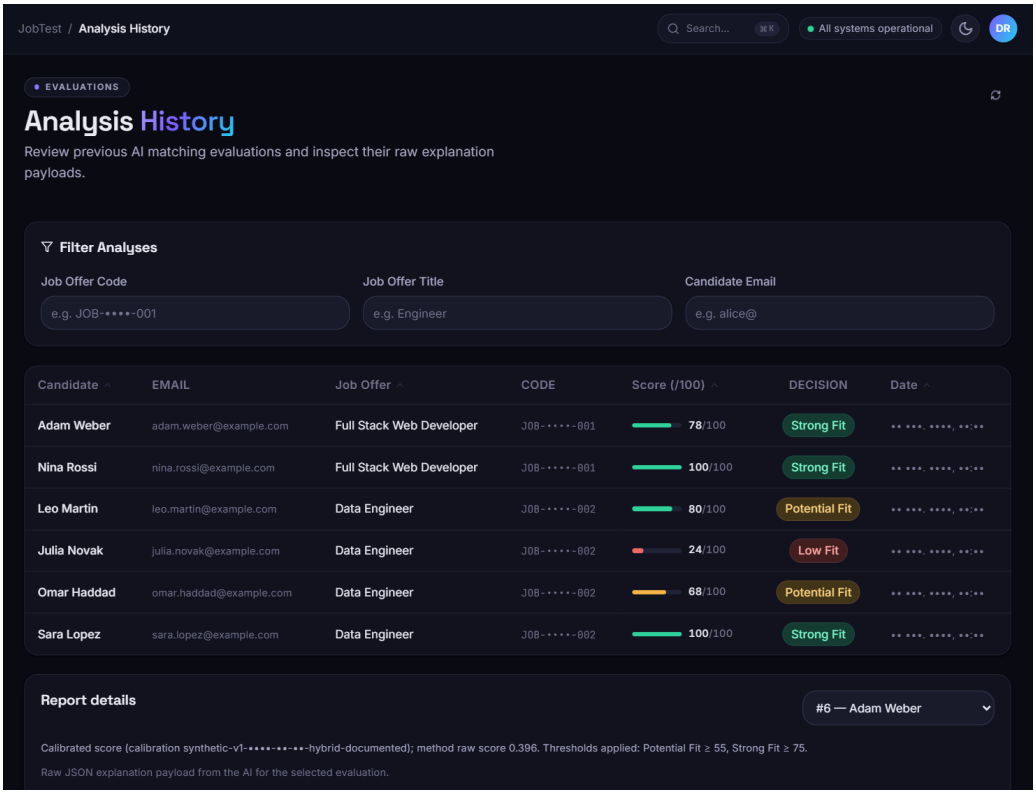


Figure 5.9: Historique des évaluations

5.7.4 Évaluation, résultat et analyse approfondie

L'écran *Candidate Analysis* constitue le point de départ du parcours principal. Il présente les compétences demandées par l'offre choisie, la politique de décision active et le bouton de lancement (figure 5.10). L'exemple suivant évalue un CV fictif de développeur web face à l'offre *Full Stack Web Developer*.

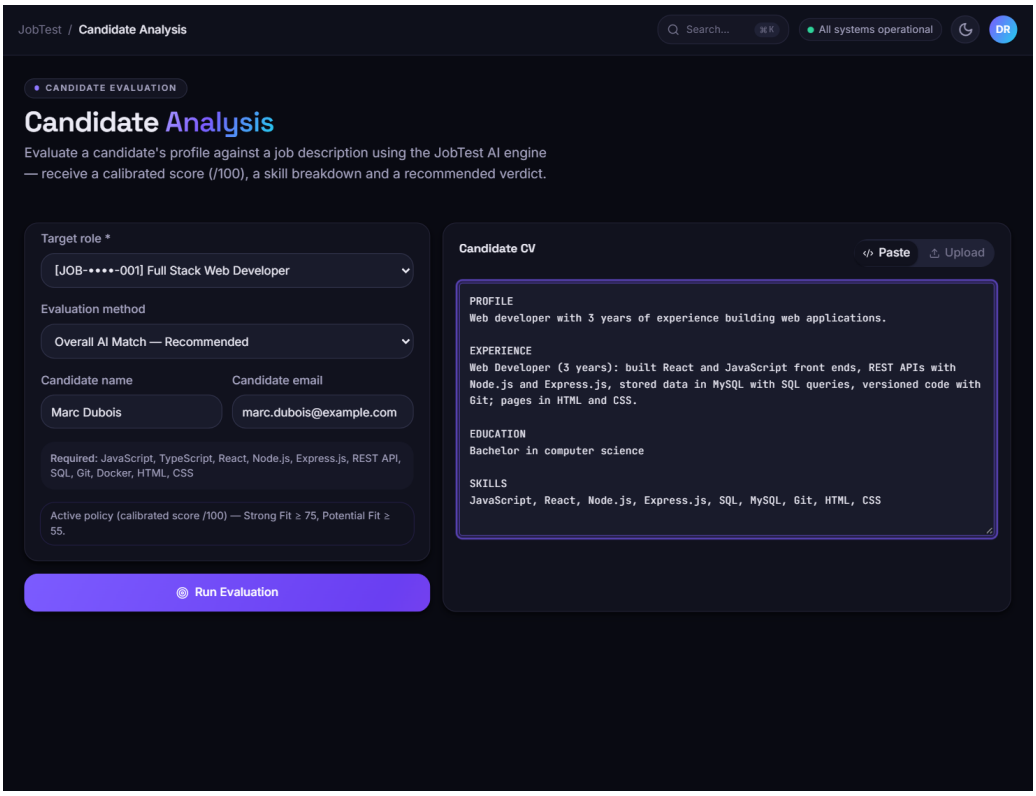


Figure 5.10: Saisie d'une évaluation

Le résultat affiche le score calibré sur 100, le verdict recommandé, la distance au seuil le plus proche et la justification. Sous le score, une mention rappelle la version de calibration, le score brut de la méthode et les bandes appliquées. Dans cet exemple, le score brut de 0,651 dépasse la borne haute de la calibration (0,613) : le score affiché atteint donc 100, alors que 8 des 11 compétences demandées sont reconnues. La justification signale aussi qu'une compétence exigée, TypeScript, reste à vérifier. La distance au seuil n'est pas une probabilité (figure 5.11).

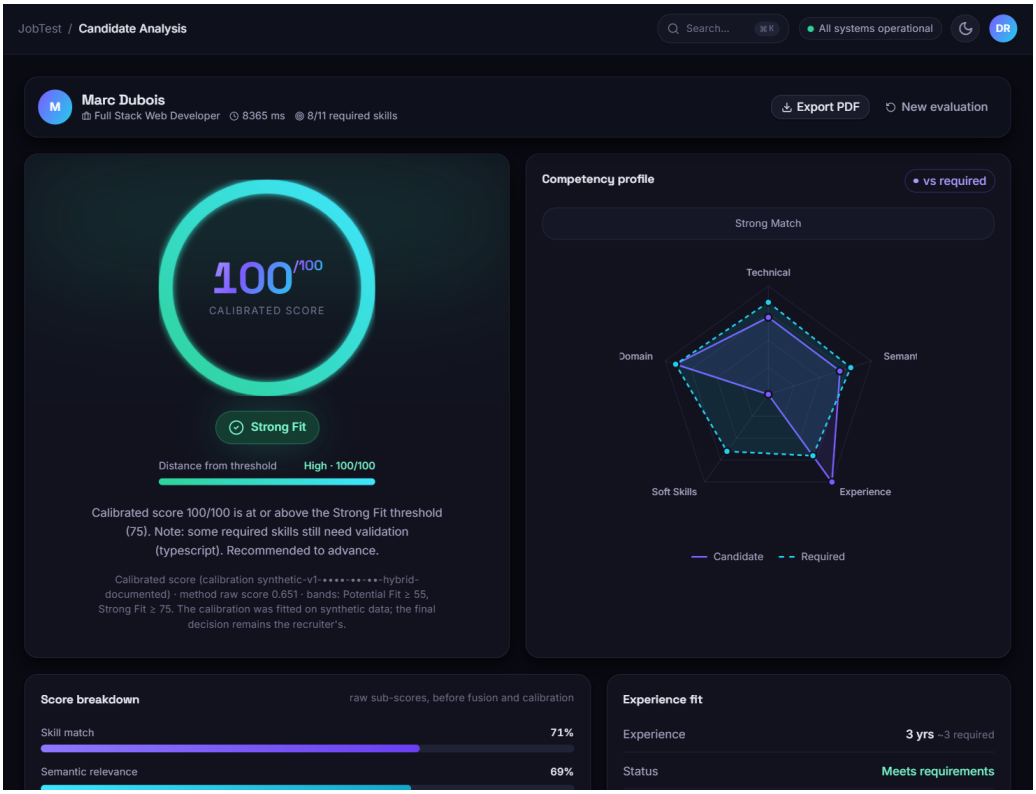


Figure 5.11: Score, verdict et profil de compétences

Les sous-scores sont présentés comme des valeurs brutes, avant fusion et calibration, à côté de l'adéquation d'expérience (figure 5.12). Les onglets donnent accès aux compétences, aux forces et écarts, aux actions recommandées et aux données brutes.

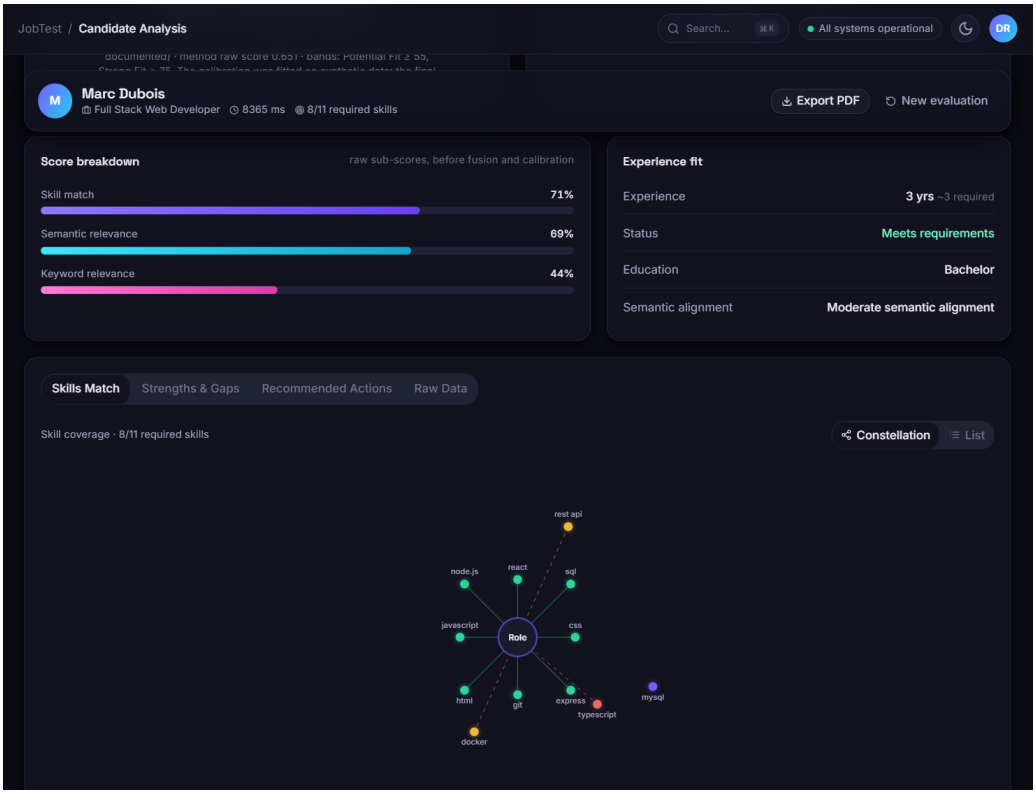


Figure 5.12: Sous-scores et adéquation d'expérience

La constellation des compétences distingue les compétences reconnues, les lacunes prioritaires,

les compétences manquantes et les compétences supplémentaires du candidat (figure 5.13).

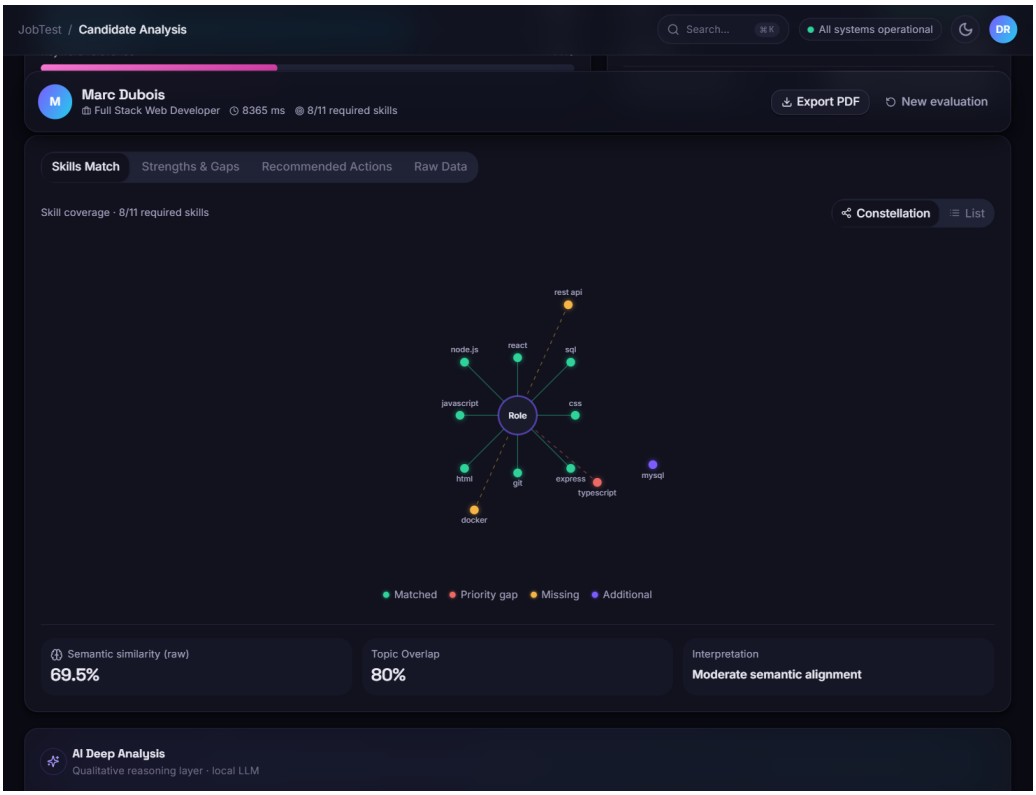


Figure 5.13: Compétences reconnues et manquantes

L'analyse approfondie est demandée automatiquement après l'affichage du résultat. Exécutée localement par `qwen2.5:1.5b` via Ollama, elle propose une synthèse, une trajectoire, des signaux à examiner, des recommandations et des questions d'entretien. Une mention rappelle qu'elle ne modifie ni le score ni le verdict (figure 5.14). Si le runtime ne répond pas, son indisponibilité est affichée sans altérer le résultat principal.

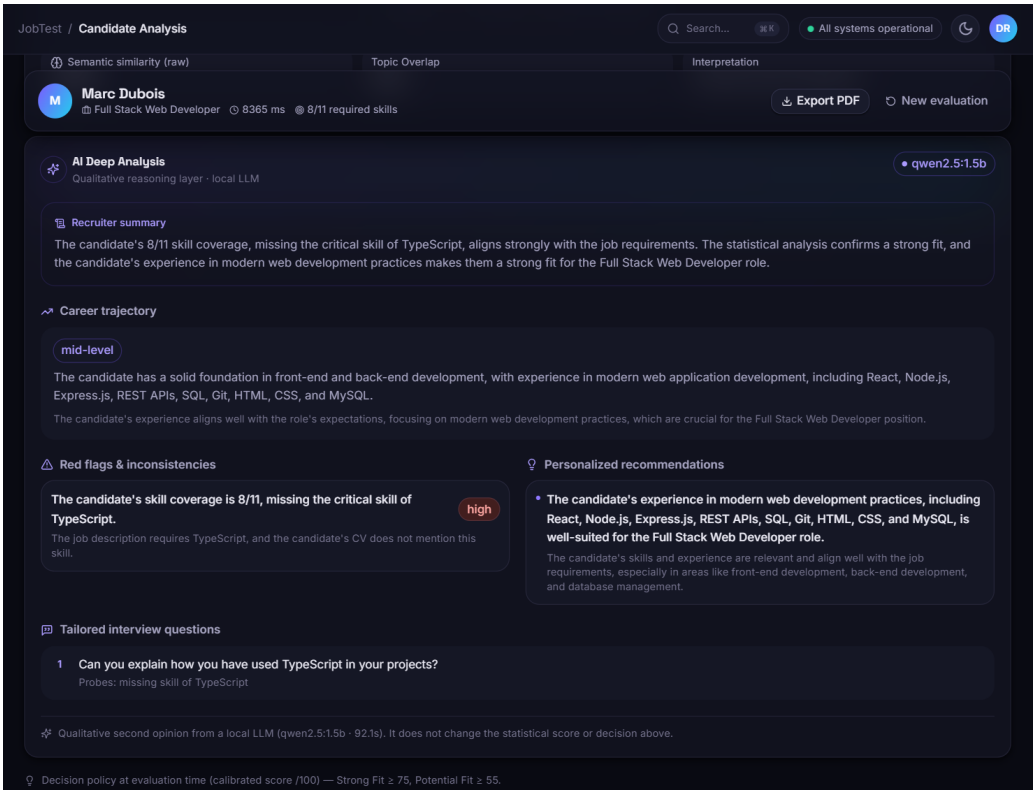


Figure 5.14: Analyse approfondie locale

Lorsque les compétences correspondantes reposent surtout sur des déclarations ou sur du texte recopié de l'offre, un encadré *Evidence check* le signale. La figure 5.15 montre un CV fictif de graphiste complété par la liste des compétences de l'offre *Data Engineer* : son score calibré atteint 80,6, mais aucune des huit compétences correspondantes n'apparaît dans l'expérience décrite. Le verdict est donc limité à *Potential Fit* (`CONSIDER`), avec la liste des compétences seulement déclarées.

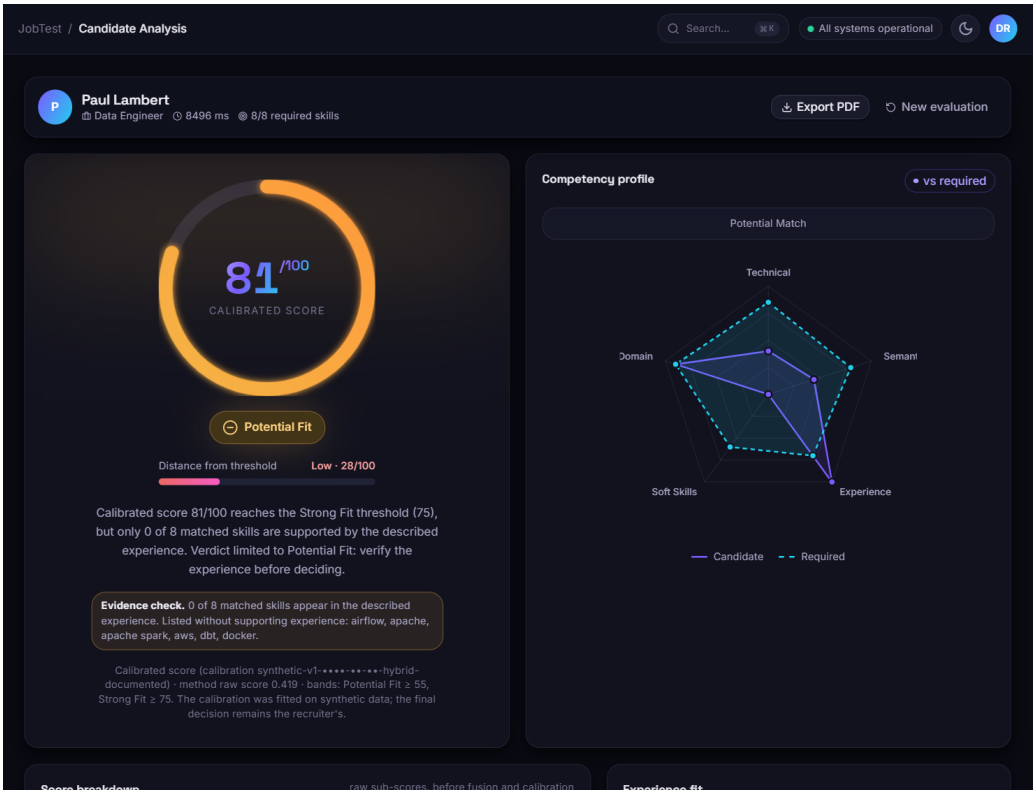


Figure 5.15: Verdict limité par le contrôle des preuves

5.7.5 Classement, comparaison, qualité et paramètres

Le *Talent Leaderboard* accepte plusieurs CV pour une offre et restitue un classement. Les figures 5.16 à 5.19 rejouent avec la version finale l'essai décrit au chapitre 6 : dix-huit CV fictifs au format PDF face à l'offre *Full Stack Web Developer*. L'export CSV du classement inclut le score calibré, le score brut, les seuils et la version de calibration.

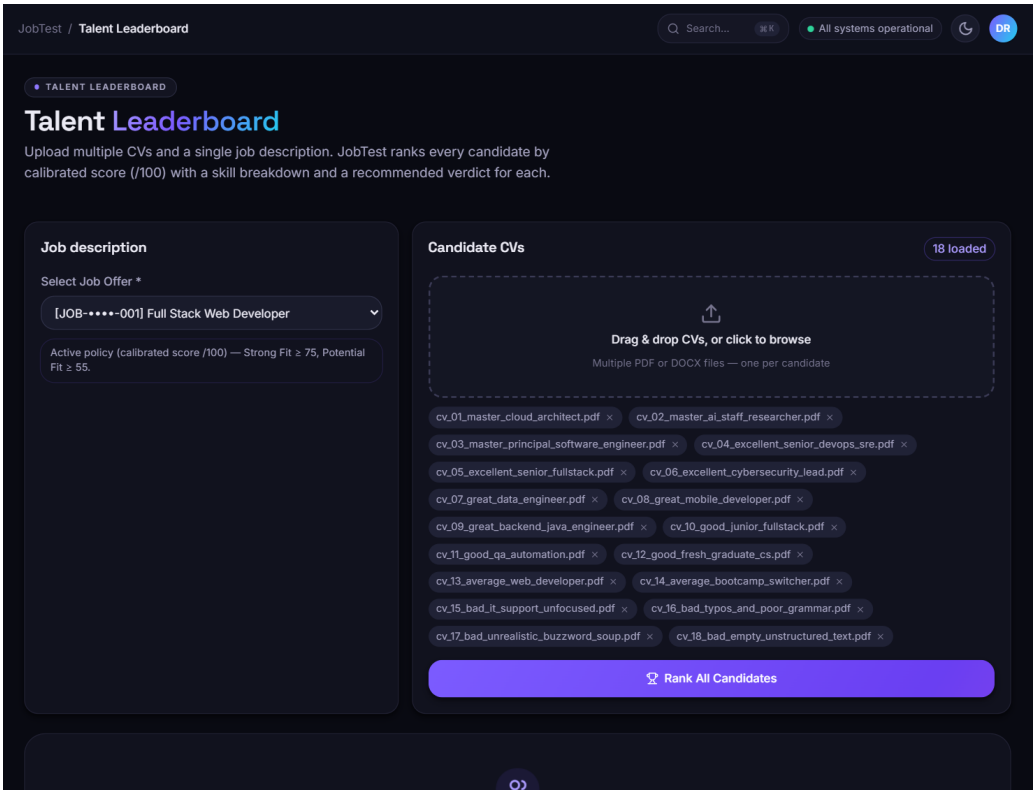


Figure 5.16: Sélection de l'offre et des CV



Figure 5.17: Distribution des scores du classement

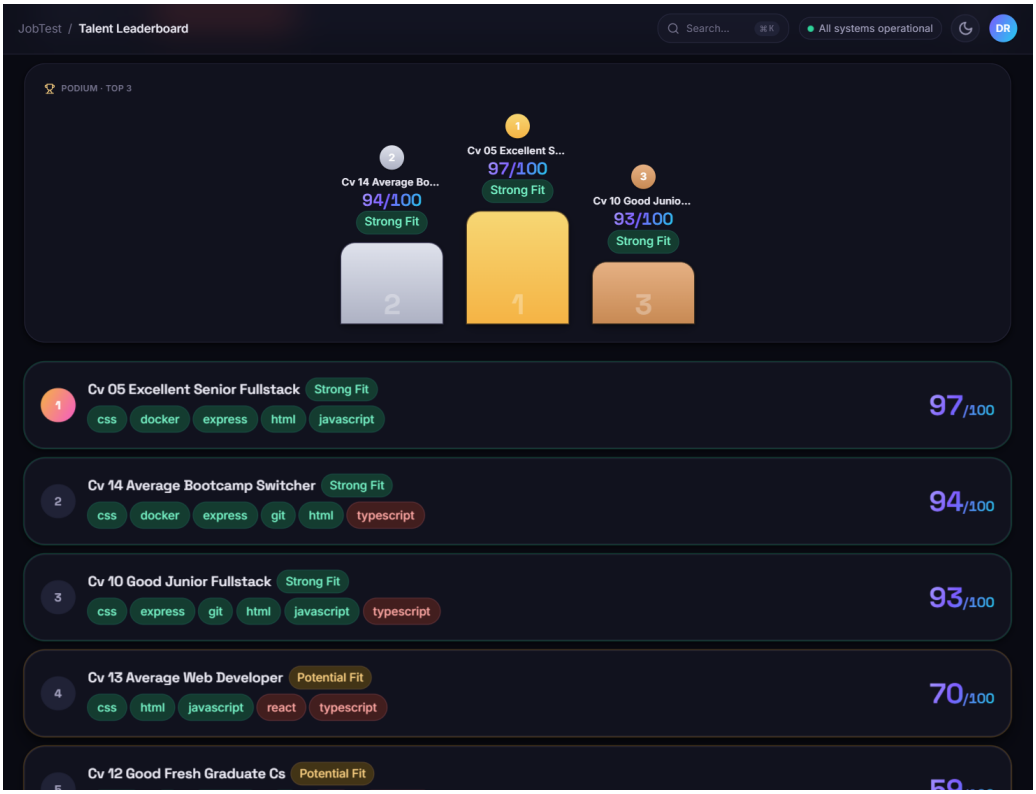


Figure 5.18: Podium du classement

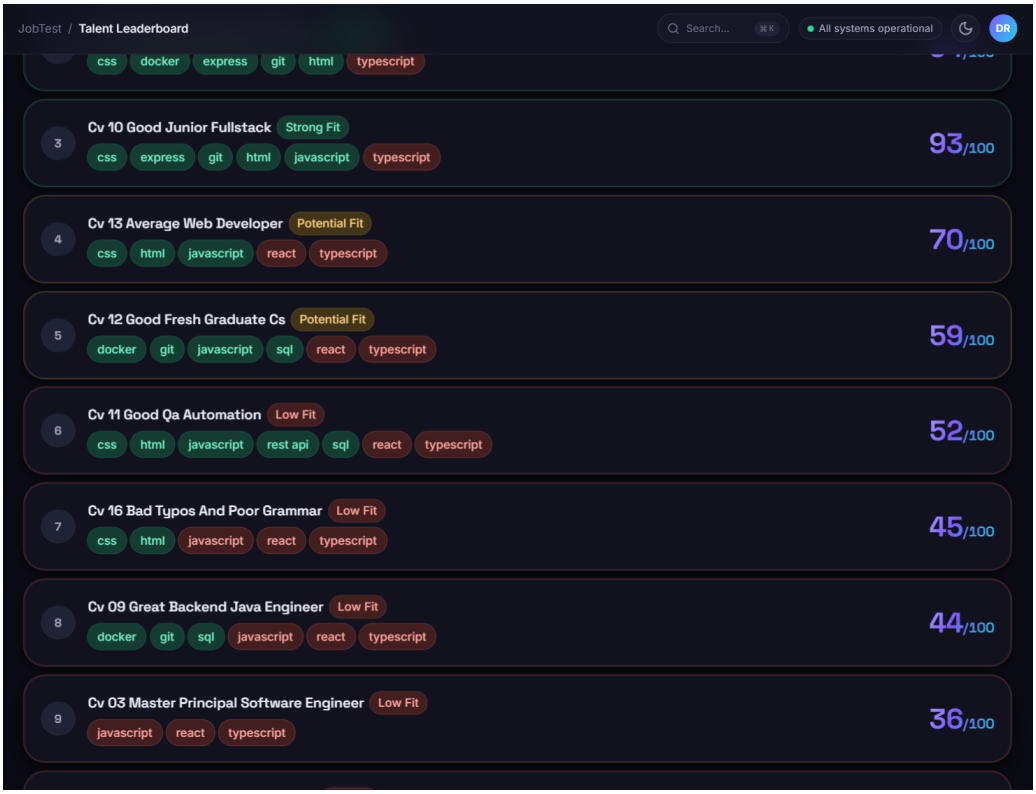


Figure 5.19: Suite du classement

AI Insights compare les quatre méthodes sur un même couple CV–offre sans créer de résultat persistant. Chaque score brut est ramené à l'échelle sur 100 par sa propre calibration, et les barres sont tracées sur cette échelle commune avec les bornes 55 et 75 (figure 5.20).

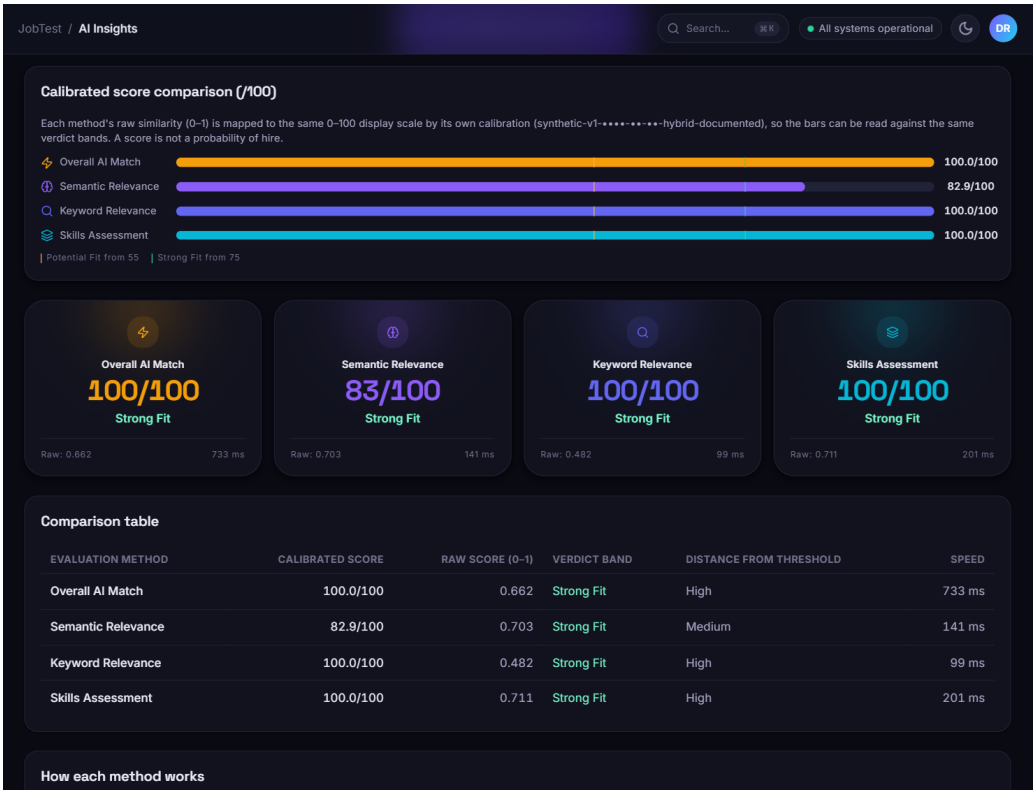


Figure 5.20: Comparaison des quatre méthodes

L'écran de paramètres règle les seuils *Strong Fit* et *Potential Fit*, exprimés sur l'échelle calibrée, et présente l'état des composants de la plateforme (figure 5.21).

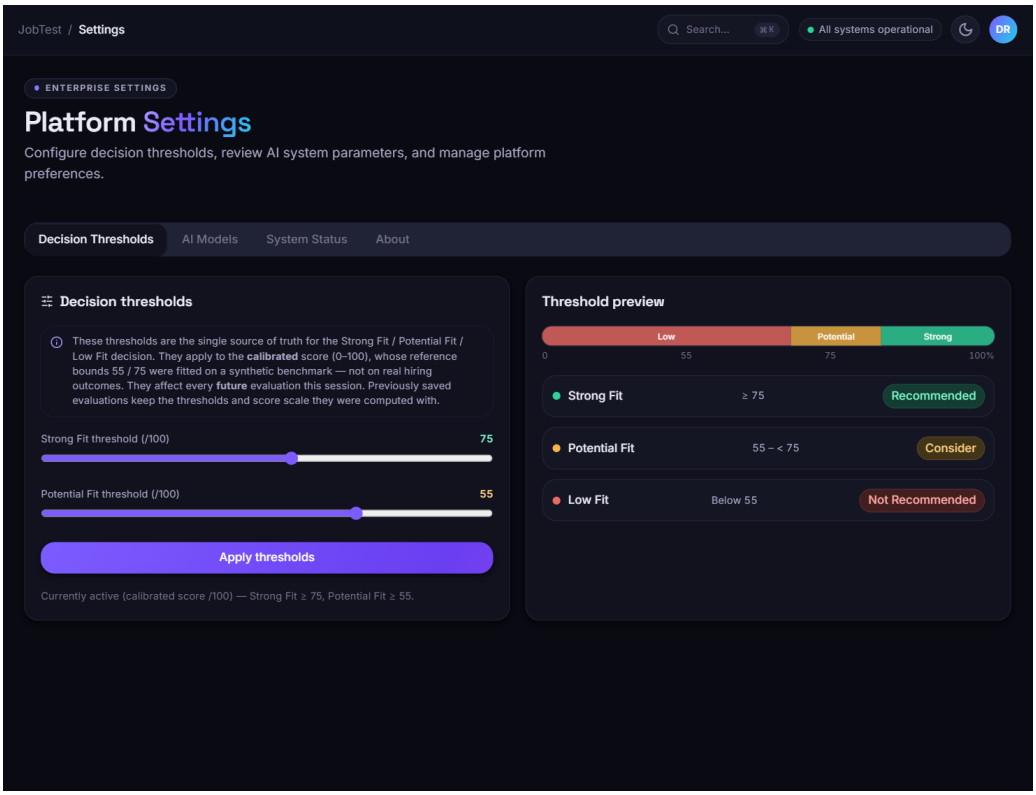


Figure 5.21: Réglage des seuils de décision

L'écran *AI Quality Center* présente les résultats d'évaluation hors ligne en indiquant pour chacun le fichier source, la taille de l'échantillon et sa concordance avec la calibration livrée. Il distingue

trois catégories : le banc d'essai synthétique, seul porteur de métriques ; les essais fonctionnels sur CV fictifs ; et les essais menés avec mes propres CV, pour lesquels les métriques sont affichées comme non mesurées (figure 5.22).

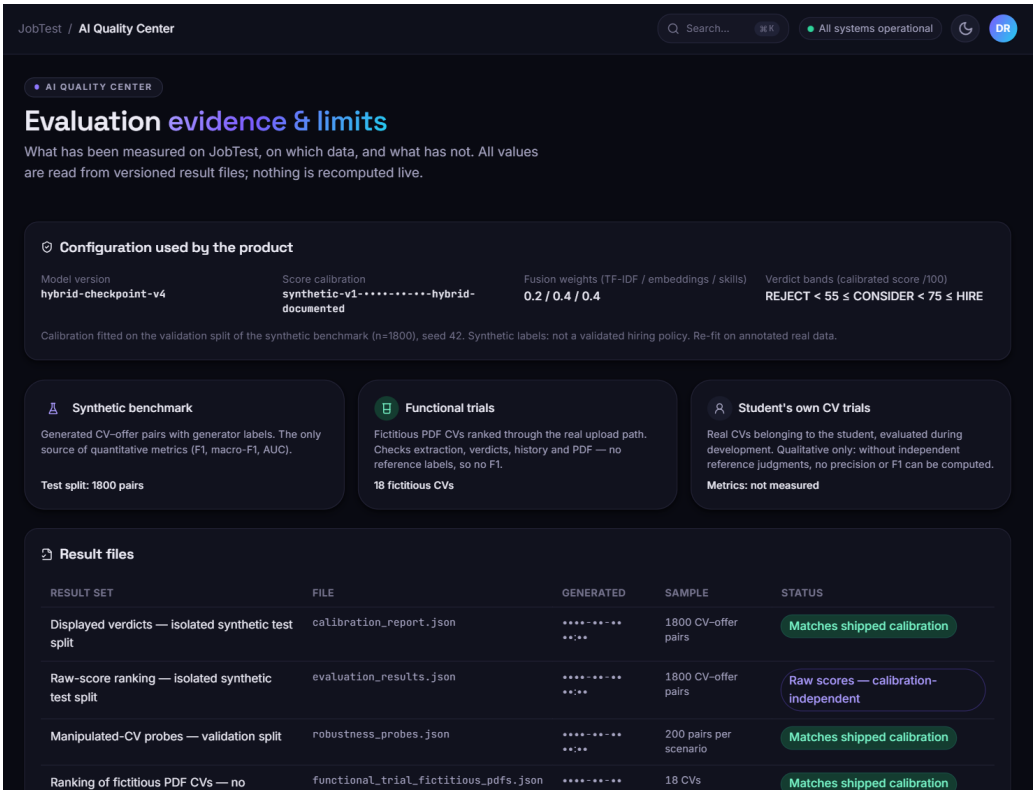


Figure 5.22: *AI Quality Center* : configuration

La qualité des verdicts affichés (macro-F1 à trois classes) et la qualité du classement du score brut (F1 binaire à un seuil expérimental) y sont présentées dans deux tableaux distincts, avec leurs définitions et la matrice de confusion de la version livrée (figure 5.23). Une dernière partie reprend les essais de robustesse face aux CV manipulés (figure 5.24). Toutes ces valeurs sont lues dans les fichiers versionnés du dossier `evaluation/results` ; une métrique absente est affichée comme non mesurée, jamais comme nulle.

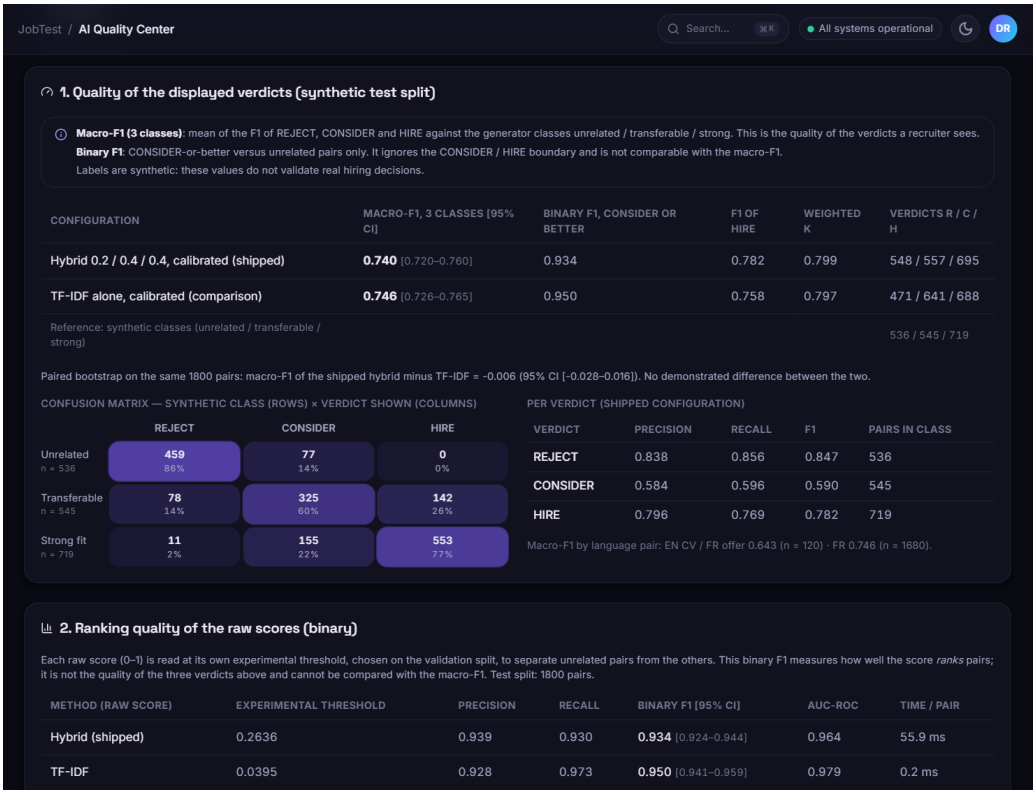


Figure 5.23: *AI Quality Center* : qualité des verdicts

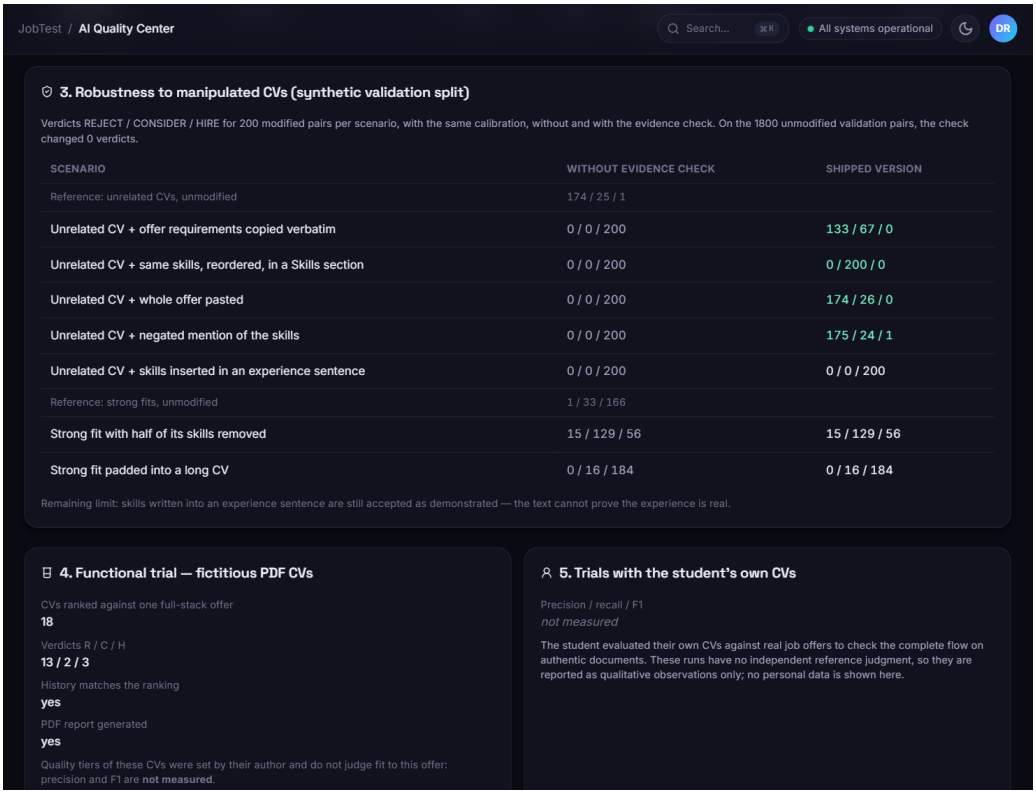


Figure 5.24: *AI Quality Center* : robustesse

Enfin, le bouton *Export PDF* de l'écran de résultat déclenche la génération ReportLab et le téléchargement du rapport. Le document reprend les éléments de décision déjà visibles, au lieu de fournir une décision autonome supplémentaire.

5.8 Parcours fonctionnel de démonstration

Un scénario de démonstration se déroule de façon continue : le recruteur se connecte, crée ou sélectionne une offre ouverte, fournit le texte ou le fichier d'un CV, puis lance l'évaluation. Il lit ensuite le score et les raisons associées, consulte si besoin l'analyse approfondie, puis exporte le rapport PDF. L'historique permet enfin de retrouver une évaluation persistée. Ce parcours maintient une séparation claire entre les données métier (offre, candidat et CV), le calcul de correspondance et l'aide à l'interprétation.

5.9 Difficultés de réalisation et solutions apportées

Les difficultés retenues ci-dessous sont celles qui apparaissent dans le code ou dans l'historique technique du projet. Elles concernent la réalisation de la plateforme, non les incidents de déploiement qui seront détaillés au chapitre suivant.

Le tableau 5.4 les présente avec la réponse apportée à chacune.

Tableau 5.4: Difficultés et solutions

Difficulté	**Conséquence**	**Solution retenue**
Prototype Streamlit et interface finale distincts	Risque que le moteur ne soit pas accessible par l'interface Web actuelle.	Création du routeur `/api/v1/ui`, réutilisant les services métier existants depuis Next.js.
CV PDF et DOCX hétérogènes	Extraction incomplète possible selon le document.	Parsers dédiés avec replis PDF et DOCX avant le prétraitement commun.
Modèles d'embeddings indisponibles ou contraints	Le composant sémantique ne doit pas empêcher le service de répondre.	Chargement différé et stratégie de repli locale documentée dans le moteur.
Layer 2 potentiellement lente ou indisponible	Une réponse LLM ne doit pas bloquer le verdict calculé.	Appel séparé, enveloppe d'état contrôlée et conservation du niveau 1 comme source du score.
Évolution SQLite vers PostgreSQL	Les entités et relations doivent rester cohérentes entre environnements.	Modèles SQLAlchemy communs et configuration par URL de base de données.
Export exploitable hors application	Le recruteur doit partager une synthèse lisible.	Génération serveur ReportLab à partir du résultat déjà obtenu.

Conclusion

JobTest est réalisé comme une plateforme Web modulaire dans laquelle Next.js porte le parcours du recruteur et FastAPI centralise les règles métier, les calculs et la persistance. Les écrans transforment un

résultat de matching en éléments interprétables : score, compétences, explications, recommandations, analyse qualitative conditionnelle et rapport PDF. Le système soutient ainsi l'évaluation du recruteur sans se substituer à sa décision. Le chapitre suivant présentera les tests, les mesures de sécurité, la conteneurisation et le déploiement Azure.

Chapitre 6

Tests, sécurité, DevOps et déploiement Azure

Plan

1 Stratégie de validation 83
2 Environnement et données de test 85
3 Tests fonctionnels 85
4 Validation du moteur intelligent 86
5 Tests de l'API et de la persistance 87
6 Validation du rapport PDF 87
7 Tests de santé et stabilité des services 87
8 Résultats de validation de bout en bout 88
9 Validation sur données réelles 88
10 Classement de dix-huit CV fictifs 90
11 Sécurité applicative 90
12 Sécurité réseau et infrastructure 91
13 Analyse honnête des limites de sécurité 92
14 Traitement des données à caractère personnel 93
15 Conteneurisation 94
16 Déploiement sur Microsoft Azure 95
17 Architecture de déploiement 98
18 Exploitation et maîtrise des coûts 99
19 Procédures d'exploitation 99
20 Bilan du déploiement 99

Introduction

Après la réalisation de la plateforme, sa valeur dépend autant de la fiabilité de la chaîne complète que de la cohérence du moteur de rapprochement. Ce chapitre présente les contrôles effectivement disponibles dans JobTest, les vérifications effectuées lors du déploiement et les protections visibles dans le code et la configuration. Il distingue les tests automatisés, les vérifications manuelles et les observations ponctuelles faites sur la machine de démonstration. L'objectif n'est pas de certifier une disponibilité de production, mais de documenter de manière traçable le périmètre validé, les choix d'exploitation et les limites restantes.

6.1 Stratégie de validation

La validation combine une suite `pytest`, des contrôles d'API et une vérification de bout en bout sur l'environnement conteneurisé. Les essais de benchmark du chapitre 4 restent séparés : ils étudient un corpus synthétique et ne constituent pas une suite de tests fonctionnels. Le tableau 6.1 synthétise le statut des éléments examinés.

Tableau 6.1: Niveaux de validation

Niveau	Objectif	Outil ou méthode	Statut
Unitaires	Vérifier extraction, fusion, décision, PDF et règles de cohérence.	`pytest` ; modèles de langue et Ollama simulés lorsque nécessaire.	Réalisé : 174 réussites, 0 échec, 1 test ignoré.
API et intégration	Vérifier routes, statuts HTTP, JWT, persistance et contrats.	Tests FastAPI et contrôles d'API documentés.	Réalisé ; contrôle d'accès entre recruteurs testé.
Fonctionnel	Vérifier le parcours recruteur depuis l'inscription jusqu'au rapport.	Scénarios manuels sur la VM Azure et en local ; script de bout en bout.	Réalisé ; résultats qualitatifs.
Chaîne IA	Vérifier sous-scores, verdict, explication et statut de la couche 2.	Tests du moteur, réponses de l'API et contrôle du résultat persistant.	Réalisé ; benchmark distinct au chapitre 4.
Déploiement	Vérifier les conteneurs, le proxy, les health checks et les ports.	`docker compose`, `nginx -t`, requêtes HTTP et journaux.	Réalisé ponctuellement.

La suite `pytest` de la version finale comporte 175 tests : 174 réussis, aucun échec et 1 test ignoré, qui concerne l'ancien prototype Streamlit retiré du périmètre maintenu. Outre l'extraction, la fusion, le PDF et la couche 2, elle couvre :

- le contrôle d'accès entre recruteurs et le limiteur de débit ;
- l'échelle calibrée, les frontières exactes des verdicts, la version de calibration enregistrée et le traitement des résultats calculés sur l'ancienne échelle ;
- le contrôle des preuves : texte recopié de l'offre, négations, compétences déclarées ou démontrées, règle limitant `HIRE` ;
- la cohérence du verdict entre l'API, l'historique, l'explication et le PDF ;
- l'*AI Quality Center* : provenance et statut de chaque fichier de résultats, séparation du F1 binaire et du macro-F1, fichiers absents ou calculés pour une autre calibration ;

- les entrées vides, les seuils invalides et les documents illisibles.

Ces tests remplacent les modèles d'embeddings par un substitut déterministe : ils vérifient la logique applicative et non la qualité prédictive du moteur, traitée au chapitre 4. Un script complémentaire rejoue le parcours complet avec les modèles réels sur une base temporaire.

6.2 Environnement et données de test

J'ai conduit le développement localement sous Windows avec Python, Node.js et SQLite, puis vérifié dans une topologie Docker Compose utilisant PostgreSQL. La validation intégrée a ensuite été observée sur une machine virtuelle Azure. Trois catégories de données ont servi aux essais. Le corpus synthétique du chapitre 4 sert au banc d'essai quantitatif. Dix-huit CV fictifs au format PDF, que j'ai rédigés et générés pour couvrir plusieurs niveaux de qualité, servent aux essais de classement. Enfin, mes propres CV, confrontés à des offres réelles, ont servi à une validation sur données réelles (section 6.9) ; de portée qualitative, elle ne constitue pas un jeu de test annoté par des recruteurs. Les captures de ce mémoire utilisent un jeu de démonstration fictif. Aucun résultat présenté dans ce chapitre ne requiert l'affichage d'un mot de passe, d'un jeton, d'une clé SSH, d'une chaîne de connexion ou du contenu du fichier `.env`.

6.3 Tests fonctionnels

La matrice suivante reprend seulement les scénarios dont l'exécution ou le résultat est attesté par les tests du dépôt ou le rapport de déploiement. Les cas non exécutés ne sont pas convertis artificiellement en succès.

Le tableau 6.2 en dresse la liste.

Tableau 6.2: Contrôles fonctionnels

ID	Fonction	Précondition	Résultat attendu	Résultat obtenu	Statut
F01	Inscription et connexion	Compte de vérification.	Création puis obtention d'un jeton sans l'exposer.	Compte créé et JWT utilisé pendant la vérification Azure.	OK
F02	Refus d'identifiants invalides	Identifiants incorrects.	Réponse d'authentification refusée.	Couvert par les tests `test_auth`.	OK
F03	Consultation du profil	Jeton Bearer valide.	Retour du recruteur connecté.	Route protégée couverte par les tests d'authentification.	OK
F04	Création d'offre	Recruteur authentifié.	Offre enregistrée et rattachée au recruteur.	Offre de contrôle créée (identifiant 1 dans l'environnement observé).	OK

ID	Fonction	Précondition	Résultat attendu	Résultat obtenu	Statut
F05	Évaluation d'un CV	Offre et CV de démonstration.	Sous-scores, explication et recommandation générés.	Résultat `MatchResult` 1 créé, verdict `CONSIDER`.	OK
F06	Persistance	Évaluation réussie.	Résultat récupérable avec relations cohérentes.	Un résultat et une analyse LLM valide comptabilisés dans PostgreSQL.	OK
F07	Analyse approfondie	Résultat persistant, Ollama disponible.	Enveloppe d'analyse sans modification du score.	`status=ok` observé et analyse enregistrée.	OK
F08	Rapport PDF	Résultat calculé.	Flux PDF ouvrable avec informations d'évaluation.	PDF 1.4, deux pages et 112 039 octets dans ce cas.	OK
F09	Isolement entre recruteurs	Deux comptes recruteurs, offre du premier.	Refus (`404`) pour le second recruteur.	Évaluation, classement et liste refusés ou filtrés dans `test_api_extended`.	OK
F10	CV qui répète l'offre	CV sans rapport complété par les compétences exigées.	Pas de `HIRE` ; justification explicite.	`CONSIDER` identique dans l'API, l'historique et le PDF, compétences signalées comme seulement déclarées.	OK
F11	Classement de 18 CV au format PDF	Offre full-stack, CV fictifs.	Extraction, verdicts, historique et PDF cohérents.	Verdicts 3 / 2 / 13 (H / C / R), historique concordant, PDF produit.	OK
F12	Essais avec mes propres CV	Mes CV personnels, offres réelles.	Parcours complet sans erreur.	Verdicts observés ; aucun jugement de référence.	Qualitatif

6.4 Validation du moteur intelligent

Le contrôle fonctionnel du moteur concerne ici sa mise en œuvre, non la qualité prédictive discutée au chapitre 4. Une évaluation retournée par l'API contient les trois sous-scores utilisés dans la fusion, les paramètres de pondération, l'explication structurée et la recommandation algorithmique. Elle contient aussi le score brut, le score calibré, la version de calibration et le résultat du contrôle des preuves. Les tests couvrent la fusion hybride, la calibration, le contrôle des preuves, les frontières de décision, la cohérence entre moteur et module d'explication ainsi que les états `disabled`, `unavailable`, `error`, `ok` et `inconsistent` de la couche 2.

La couche générative reçoit le résultat déterministe mais ne calcule ni le score officiel ni le verdict `HIRE`, `CONSIDER` ou `REJECT`. Les tests de raisonnement emploient un Ollama simulé ; la disponibilité réelle de la couche 2 a, elle, été vérifiée une fois sur Azure. Les cas de CV vide, d'offre inexistante et

de ressources appartenant à un autre recruteur sont contrôlés par les routes concernées. Les frontières exactes (55 et 75 sur l'échelle calibrée) sont vérifiées par des tests dédiés, qui contrôlent que le verdict suit le score affiché arrondi au dixième.

6.5 Tests de l'API et de la persistance

Les schémas Pydantic valident les corps de requête et FastAPI retourne notamment des erreurs `401`, `404`, `422` ou `429` selon le contrat concerné. Chaque route de données passe par l'authentification Bearer. La fonction `_assert_owned` vérifie que l'offre et le candidat visés appartiennent au recruteur connecté ; une ressource d'un autre périmètre est traitée comme absente. Les requêtes courantes sont exprimées par SQLAlchemy plutôt que par concaténation de SQL ; cette pratique réduit l'exposition à des chaînes SQL construites manuellement, sans constituer à elle seule une garantie universelle.

La vérification Azure a confirmé l'écriture d'un `MatchResult` et d'une analyse LLM valide, puis leur comptage dans PostgreSQL. Les relations candidat, offre et résultat ont donc été parcourues dans un scénario réel de démonstration. Swagger était accessible derrière le proxy à `/api/v1/docs`; aucune capture n'est ajoutée afin d'éviter de répéter les écrans du chapitre 5.

6.6 Validation du rapport PDF

La génération est testée dans le dépôt par des contrôles de structure binaire et je l'ai également validée dans le parcours déployé. Pour un résultat précis, la réponse HTTP a été reçue sous forme de PDF 1.4, le fichier s'est ouvert sur deux pages et sa taille était de 112 039 octets. Ces valeurs caractérisent un exemple de démonstration : le nombre de pages et le poids peuvent évoluer avec les données du candidat et de l'offre. L'export utilise les informations déjà calculées ; il ne lance pas un nouveau rapprochement.

6.7 Tests de santé et stabilité des services

Lors du déploiement de démonstration, j'ai observé les cinq conteneurs à l'état *healthy*, avec une politique `unless-stopped` et sans redémarrage inattendu lors du contrôle. L'accueil et Swagger ont retourné HTTP 200 via le proxy. L'endpoint FastAPI de santé et son contrôle détaillé ont retourné `ok`; la couche 1 était disponible, Ollama répondait et le modèle `qwen2.5:1.5b` était installé. Les journaux examinés ne montraient ni OOM, ni erreur fatale, ni boucle de redémarrage. Il s'agit d'une observation à un instant donné et non d'un engagement de disponibilité permanente.

6.8 Résultats de validation de bout en bout

Le scénario intégré attesté suit les étapes suivantes : utilisation d'un compte recruteur, authentification, création d'une offre, évaluation d'un CV synthétique, enregistrement du résultat, lecture de l'explication, analyse approfondie puis génération du PDF servi par le proxy. Cette chaîne a produit les trois composantes de score, une recommandation `CONSIDER`, un `MatchResult` persistant, une analyse LLM `ok` et le rapport PDF décrit précédemment.

Le même parcours, rejoué localement avec la version finale du moteur et les modèles réels, conduit à des verdicts identiques dans l'API, l'explication, l'historique et le PDF. Un CV volontairement complété par les compétences exigées par l'offre y reçoit `CONSIDER`, assorti d'une justification indiquant qu'aucune de ces compétences n'est appuyée par l'expérience décrite.

La mesure de l'analyse approfondie est de 54 290,6 ms sur cette VM CPU. Elle doit être interprétée comme une mesure ponctuelle sur une infrastructure de démonstration et non comme un objectif de temps de réponse. La couche 1 demeurait opérationnelle pendant que la couche qualitative est, par conception, additive et consultative.

6.9 Validation sur données réelles

Les mesures chiffrées du chapitre 4 portent sur un corpus synthétique annoté. Pour vérifier que le moteur se comporte de manière cohérente hors de ce corpus, j'ai soumis à JobTest des documents authentiques : mes propres CV, confrontés à des offres réelles. L'essai le mieux tracé oppose deux de mes CV à une offre réelle et détaillée de développeur frontend, qui décrit le poste et liste sept compétences requises. J'ai d'abord réalisé ces évaluations dans l'application, puis je les ai reproduites avec la version finale du moteur (`hybrid-checkpoint-v4` et calibration versionnée livrée) à l'aide du script `scripts/real_cv_trials.py`. Ce script lit les documents dans la base locale de développement, exécute le moteur sur la machine, sans appel au LLM ni à un service externe, et n'enregistre que des agrégats dans `evaluation/results/real_cv_trials.json`. Aucun contenu de CV ni aucune coordonnée n'est reproduit ici ; les deux documents sont désignés par « CV réel A » et « CV réel B ».

Tableau 6.3: Mes CV face à une offre réelle

Indicateur	**CV réel A**	**CV réel B**
Score affiché (sur 100)	**62,5**	**63,6**
Verdict	`CONSIDER`	`CONSIDER`
Score brut de l'hybride	0,301	0,306
Sous-score TF–IDF	0,108	0,064
Sous-score embeddings	0,342	0,305
Sous-score compétences	0,357	0,429
Compétences requises retrouvées	4 sur 7	3 sur 7
Compétence critique manquante	aucune	aucune
Contrôle des preuves	1 compétence sur 4 appuyée par l'expérience : signalé	aucun signalement
Domaine détecté (CV / offre)	données et IA / frontend	données et IA / frontend
Recommandations générées	5	5
Deux exécutions successives	scores identiques	scores identiques

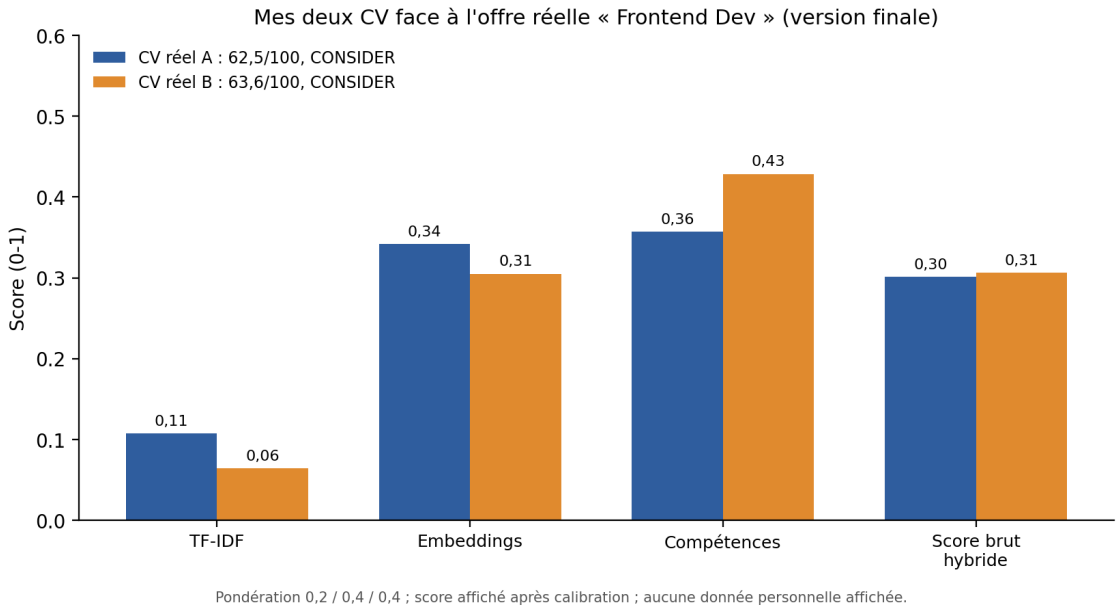


Figure 6.1: Sous-scores de mes CV réels

Le tableau 6.3 et la figure 6.1 montrent un comportement cohérent avec le contenu des documents. Les deux CV reçoivent `CONSIDER`, un verdict intermédiaire qui correspond à la situation réelle : le moteur classe mes CV dans le domaine des données et de l'intelligence artificielle, alors que l'offre relève du développement frontend, et il retrouve un peu plus de la moitié des compétences

requises sans qu'aucune compétence critique ne manque. Les composantes apportent des signaux différents : le CV A est plus proche de l'offre par le vocabulaire et par le sens, le CV B couvre davantage de compétences explicites. La fusion les ramène à des scores voisins (0,301 et 0,306), ce qui illustre qu'aucune composante ne décide seule.

Le contrôle des preuves joue également son rôle sur ces documents authentiques. Pour le CV A, une seule des quatre compétences retrouvées est appuyée par l'expérience décrite ; l'explication le signale, et un verdict `HIRE` aurait été ramené à `CONSIDER` si le score avait dépassé 75. Deux exécutions successives donnent des scores identiques, et chaque évaluation prend moins d'une seconde sur mon poste de travail une fois les modèles chargés. Ces essais confirment ainsi que la chaîne d'évaluation, du calcul des scores au verdict, à l'explication et aux recommandations, fonctionne sur des documents réels.

Leur portée reste qualitative. Ils portent sur un petit nombre de documents, dont je suis l'auteur, et aucun recruteur indépendant n'a établi de jugement de référence pour ces offres : aucune précision ni aucun F1 n'en est tiré. Ils complètent la mesure synthétique du chapitre 4 sans la remplacer. Une validation sur un échantillon de CV réels annotés par des recruteurs constitue la prochaine étape, présentée dans les perspectives.

6.10 Classement de dix-huit CV fictifs

Ce second essai vérifie le classement multi-CV sur des documents au format PDF. Il ne constitue pas une évaluation statistique : aucun jugement de référence établi par un recruteur n'existe pour l'offre considérée.

J'ai exercé le classement multi-CV avec dix-huit CV fictifs au format PDF face à une offre de développeur full-stack. Les captures du classement présentées au chapitre 5 montrent cet essai avec la version finale. Ces CV sont associés à des niveaux de qualité générale que j'ai fixés, et non à une appréciation de leur adéquation à cette offre ; ces niveaux ne peuvent donc pas servir à calculer une précision ou un F1. Avec la version finale, l'essai donne 3 verdicts `HIRE`, 2 `CONSIDER` et 13 `REJECT`. Les profils de développement web et full-stack occupent les premières places et les profils sans lien avec l'offre les dernières, quel que soit leur niveau général. Un profil de reconversion que j'avais jugé moyen, mais dont les projets décrivent les technologies demandées, reçoit `HIRE`. Ce cas illustre la limite déjà relevée : une expérience décrite dans le texte ne peut pas être vérifiée par le moteur.

6.11 Sécurité applicative

Les mots de passe sont traités par `passlib` avec le schéma `bcrypt` visible dans le code. Les jetons JWT sont signés en HS256, portent une expiration et sont vérifiés à chaque dépendance `get_current_user`.

La clé provient de l'environnement ; en son absence, l'application fabrique une valeur aléatoire au démarrage, ce qui invalide les sessions au redémarrage et produit un avertissement. En déploiement, une valeur durable est donc fournie hors dépôt.

La séparation par `recruiter_id`, les contrôles de propriété et les schémas Pydantic constituent les protections principales des opérations métier. Les fichiers sont acceptés selon l'extension PDF/DOCX et passent par un fichier temporaire ; le proxy limite le corps des requêtes à 15 Mo. Cette dernière limite ne remplace pas une validation de signature de fichier : celle-ci n'est pas implémentée. Un limiteur en mémoire applique par défaut 60 requêtes par minute et par adresse ; il ne remplace ni une politique de verrouillage de compte ni une protection distribuée.

Les secrets réels sont exclus du dépôt et des contextes Docker. La configuration de la VM place `/opt/jobtest/.env` avec le mode 0600 observé. Les erreurs de la couche LLM sont ramenées à une enveloppe d'état contrôlée : la chaîne statistique reste utilisable si Ollama est indisponible. Ces mécanismes sont des mesures concrètes, mais ne justifient aucune revendication de conformité ou de sécurité totale.

6.12 Sécurité réseau et infrastructure

Un groupe de sécurité réseau Azure (NSG) filtre le trafic entrant et sortant au niveau des ressources virtuelles ; ses règles peuvent notamment spécifier le protocole, le port, l'action et la priorité [20]. Pour le redéploiement HTTPS, les règles entrantes relevées dans le portail autorisent SSH (TCP 22, priorité 300), HTTP (TCP 80, priorité 320, redirigé vers HTTPS par Caddy) et HTTPS (TCP 443, priorité 330), depuis toute source ; tout autre trafic entrant est refusé par la règle par défaut `DenyAllInBound`. L'ouverture de SSH à toute source reste un point à durcir.

Le proxy inverse est le seul conteneur publiant des ports hôtes. Dans la version déployée, il s'agit de Caddy, qui publie les ports 80 et 443 ; il relaie `/api/v1/` vers FastAPI et les autres chemins vers Next.js. Compose attache les services au réseau `jobtest` de type bridge ; ils sont joignables par leurs noms de service, tandis qu'un port hôte n'est nécessaire que pour l'accès extérieur [21]. Ainsi, 3000, 8000, 5432 et 11434 ne sont pas publiquement exposés. J'ai constaté que UFW était inactif sur la VM : l'isolation actuelle repose donc sur le NSG, l'absence de mappings de ports et le proxy, plutôt que sur une seconde couche active de filtrage local.

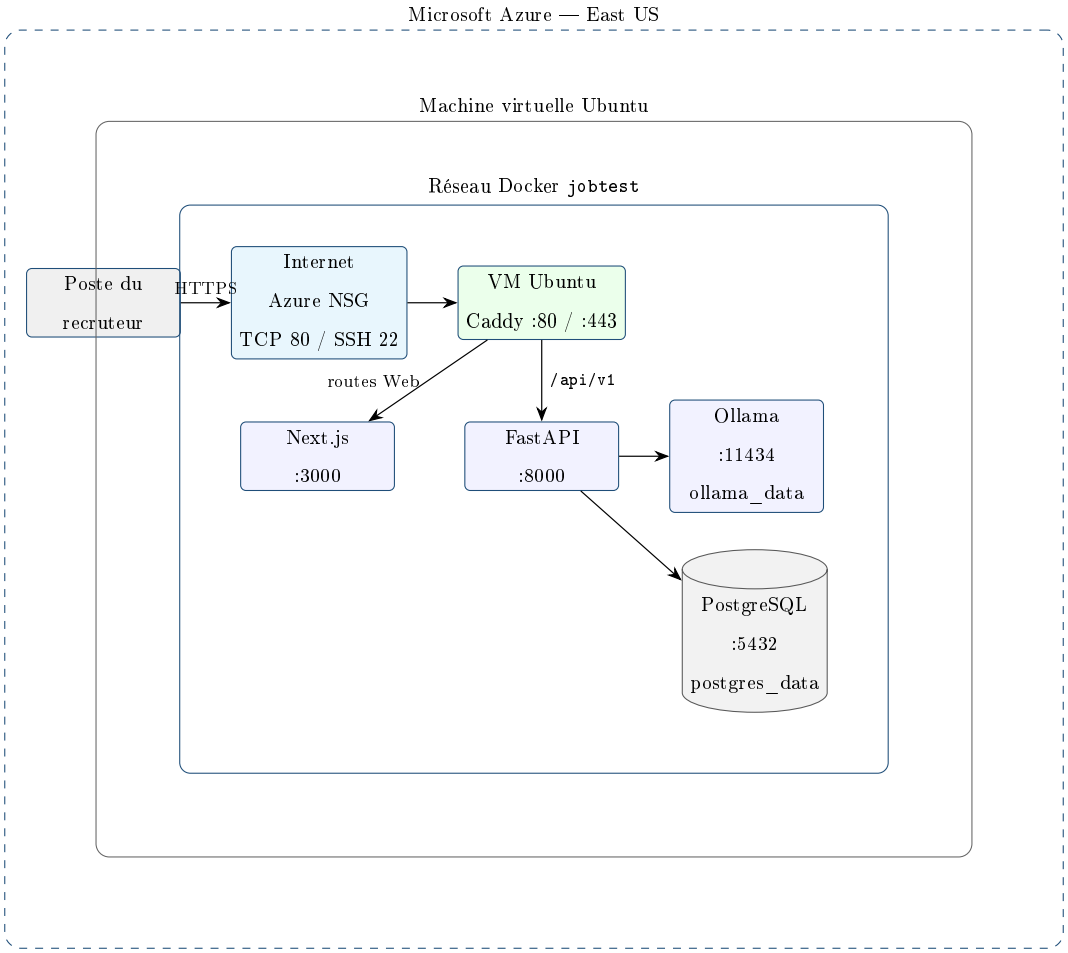


Figure 6.2: Architecture de déploiement

6.13 Analyse honnête des limites de sécurité

Le démonstrateur est servi en HTTPS sous un nom DNS Azure, avec un certificat Let's Encrypt obtenu et renouvelé automatiquement par Caddy ; les requêtes HTTP sont redirigées vers HTTPS. Le jeton Bearer est conservé côté frontend dans `localStorage`; une injection de script réussie pourrait donc le lire. OWASP recommande de ne pas placer de données sensibles dans ce stockage lorsque des attaques XSS sont à considérer [22]. Un passage vers des cookies `HttpOnly`, `Secure` et `SameSite` est une amélioration souhaitable, associée à une protection XSS approfondie.

Le port SSH exposé doit être limité à une adresse d'administration connue ; `fail2ban` peut compléter cette mesure selon le contexte. Aucune rotation automatisée des secrets, aucun WAF, aucune supervision centralisée ni sauvegarde PostgreSQL planifiée ne sont attestés. L'infrastructure n'a qu'une VM, une base et un point d'entrée : elle ne fournit ni haute disponibilité ni reprise automatisée. Les téléversements restent perfectibles (contrôle de type fondé surtout sur l'extension, pas de validation par signature), et les textes envoyés à la couche LLM doivent être considérés comme non fiables. Une politique de conservation, d'anonymisation et de suppression contrôlée des CV est également nécessaire avant un emploi sur des candidatures réelles.

6.14 Traitement des données à caractère personnel

Un CV n'est pas un document technique ordinaire : il identifie une personne et décrit son parcours. Toute plateforme qui l'ingère traite donc des données à caractère personnel, et il serait malhonnête de discuter la sécurité de JobTest sans nommer cette dimension. Les catégories manipulées par la plateforme sont l'identité déclarée du candidat, son adresse électronique et son téléphone lorsqu'ils figurent dans le dossier, le texte intégral du CV avec les employeurs, établissements et dates qu'il contient, puis les données produites par le système lui-même : score, verdict, compétences détectées ou absentes et analyse qualitative éventuelle. Cette dernière catégorie mérite attention, car une appréciation calculée sur une personne devient à son tour une donnée la concernant.

Le cadre applicable en Tunisie est la loi organique n° 2004-63 du 27 juillet 2004 relative à la protection des données à caractère personnel [23]. Sans en reprendre le détail, son économie générale repose sur quelques principes : le traitement suppose en règle générale le consentement de la personne concernée, il doit poursuivre une finalité déterminée et légitime, les données collectées doivent rester en rapport avec cette finalité, et la personne conserve des droits d'accès, de rectification et de suppression sur ce qui la concerne. La loi institue également une autorité de contrôle, l'Instance nationale de protection des données à caractère personnel, chargée de recevoir les déclarations de traitement, de délivrer les autorisations prévues par le texte et d'instruire les réclamations [24]. Un usage réel de JobTest sur des candidatures authentiques relèverait de ce cadre et supposerait les démarches correspondantes auprès de cette instance ; le présent projet, conduit dans un contexte académique sur des CV synthétiques ou fictifs et sur mes propres CV, ne les a pas engagées.

Plusieurs choix d'architecture décrits dans ce mémoire ont néanmoins un effet direct sur la protection des candidats, et il est utile de les relire sous cet angle plutôt que de les présenter comme de simples mesures techniques. Le cloisonnement par `recruiter_id` fait qu'un dossier n'est lisible que dans l'espace du recruteur qui l'a créé : une offre ou un candidat appartenant à un autre compte est traité comme inexistant, ce qui limite la circulation interne des données. L'exécution locale du modèle de langage est le second point notable : la couche 2 s'adresse à un conteneur Ollama joint par le réseau interne, de sorte que le contenu d'un CV n'est jamais transmis à un fournisseur d'intelligence artificielle externe, contrairement à ce qu'impliquerait l'appel d'une API tierce. Enfin, la suppression d'un dossier candidat est une action exposée par l'interface, ce qui fournit un premier support à une demande d'effacement, même si sa portée reste bornée par la règle qui protège l'historique des évaluations.

Ces éléments sont des dispositions de conception, non une mise en conformité. Plusieurs exigences que la loi rend centrales ne sont pas couvertes par la version étudiée : aucun mécanisme de recueil ni de preuve du consentement n'existe, puisque le candidat n'interagit jamais avec la plateforme ;

aucune durée de conservation n'est définie ni appliquée automatiquement aux CV et aux résultats ; aucune information n'est délivrée à la personne évaluée sur l'existence du traitement ou sur la logique du score qui la concerne. À cela s'ajoute que le texte d'un CV est conservé intégralement dans la base, alors qu'une partie de son contenu n'est pas nécessaire au calcul de correspondance.

Les travaux à mener avant tout usage sur des candidatures réelles se déduisent de cette liste : définir et appliquer une durée de conservation avec purge automatique, informer les candidats et organiser le recueil de leur consentement par le recruteur, réduire les données stockées à ce que le rapprochement exige réellement, ouvrir une voie d'exercice des droits d'accès et de suppression, et effectuer les formalités requises auprès de l'autorité de contrôle. Ces points ne sont pas des perspectives lointaines mais des conditions d'entrée en service ; les mentionner explicitement vaut mieux que de laisser croire qu'un prototype validé techniquement serait, pour cette raison, utilisable sur des personnes.

6.15 Conteneurisation

Docker Compose décrit cinq services et les volumes nommés qui conservent les données hors du cycle de vie d'un conteneur. Les Dockerfiles de l'API et de Next.js construisent respectivement les images applicatives ; les images Caddy, PostgreSQL et Ollama sont référencées par Compose. Les dépendances conditionnent le démarrage de l'API à la santé de PostgreSQL, et celui du proxy à la santé de l'API et du frontend. Les health checks contrôlent l'API, la page web, `pg_isready` et `ollama list`. Caddy sert de proxy inverse et assure l'accès HTTPS à la plateforme.

Le tableau 6.4 précise le rôle, le port et la persistance de chacun d'eux.

Tableau 6.4: Services conteneurisés

Conteneur	**Image ou build**	**Rôle**	**Port**	**Persistance**
Caddy	`caddy:2-alpine`	Entrée HTTPS, certificat TLS et proxy inverse.	80 et 443 publics	`caddy_data`, `caddy_config`.
Web	Build `web/Dockerfile`	Interface Next.js.	3000 interne	Aucune donnée métier.
API	Build `Dockerfile.api`	API, moteur et génération PDF.	8000 interne	`embedding_cache`.
PostgreSQL	`postgres:16-alpine`	Base relationnelle.	5432 interne	`postgres_data`.
Ollama	`ollama/ollama:0.5.13`	Analyse qualitative locale.	11434 interne	`ollama_data`.

6.16 Déploiement sur Microsoft Azure

J'ai suivi un enchaînement simple pour le déploiement : création du groupe de ressources, création de la VM Ubuntu, accès SSH, préparation de Docker, transfert de la version retenue, création locale du fichier `.env`, construction des images, téléchargement du modèle Ollama, démarrage Compose puis vérification des services et du proxy. J'ai ouvert les ports HTTP et HTTPS de façon contrôlée pour la démonstration. La VM `vm-jobtest-pfe`, dans le groupe `rg-jobtest-pfe`, est localisée en East US.

J'ai provisionné la machine virtuelle `vm-jobtest-pfe` (Standard D2as_v7) avec le script `deploy/provision_vm.sh`, puis j'y ai déployé la version finale du dépôt, avec Caddy comme proxy inverse. La plateforme est accessible à l'adresse `https://jobtest-pfe-iheb.eastus.cloudapp.azure.com/login`. Lors de ma vérification, le certificat Let's Encrypt était valide, les requêtes HTTP étaient redirigées vers HTTPS (code 308) et le contrôle de santé de l'API répondait `ok`. Cet état ponctuel ne garantit aucune disponibilité future : l'instance peut cesser de répondre en cas d'arrêt, de désallocation ou de suppression de la VM, et seules des données fictives doivent y être utilisées.

QR code d'accès à la démonstration de JobTest

`https://jobtest-pfe-iheb.eastus.cloudapp.azure.com/login`

La figure 6.3 montre la machine virtuelle recréée pour ce redéploiement, en cours d'exécution, avec le nom DNS qui porte l'adresse HTTPS de la plateforme.

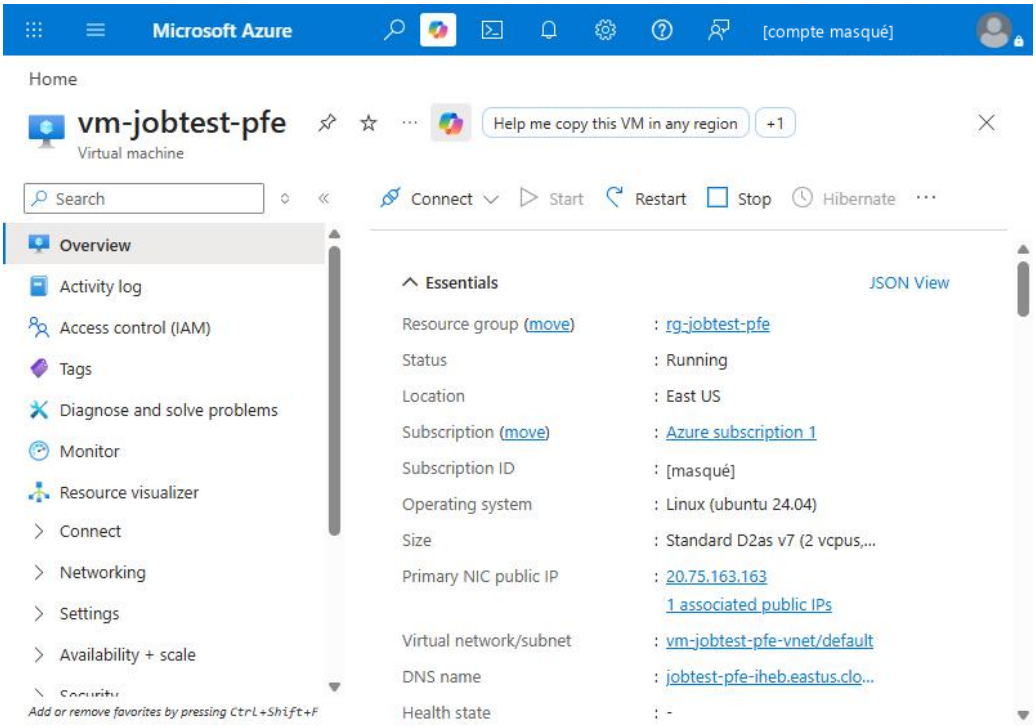


Figure 6.3: Machine virtuelle Azure

Tableau 6.5: Configuration de la machine virtuelle

Paramètre	**Valeur observée ou déclarée dans le rapport de déploiement**
Fournisseur et région	Microsoft Azure, East US.
Machine et système	`vm-jobtest-pfe`, Ubuntu Server 24.04 LTS.
Taille	Standard D2as_v7 ; 2 vCPU et 8 Gio de mémoire.
Stockage et mémoire virtuelle	Disque Standard SSD ; espace d'échange persistant de 4 Gio.
Accès	HTTPS TCP 443 ; HTTP TCP 80 redirigé vers HTTPS ; SSH TCP 22 pour l'administration.
Arrêt automatique	Configuré pour un arrêt quotidien en dehors des démonstrations.

J'ai relevé les captures du portail Azure ci-dessous sur la première instance de démonstration ; elles ne décrivent pas la machine du déploiement HTTPS. Elles sont recadrées et masquées dans les pixels afin de ne pas afficher l'adresse e-mail de connexion, l'identifiant d'abonnement, l'adresse IP publique de cette première instance ni les dates. Elles constituent des constats visuels de l'environnement de démonstration et complètent les valeurs synthétisées dans le tableau 6.5.

La figure 6.4 présente l'abonnement utilisé pour la démonstration.

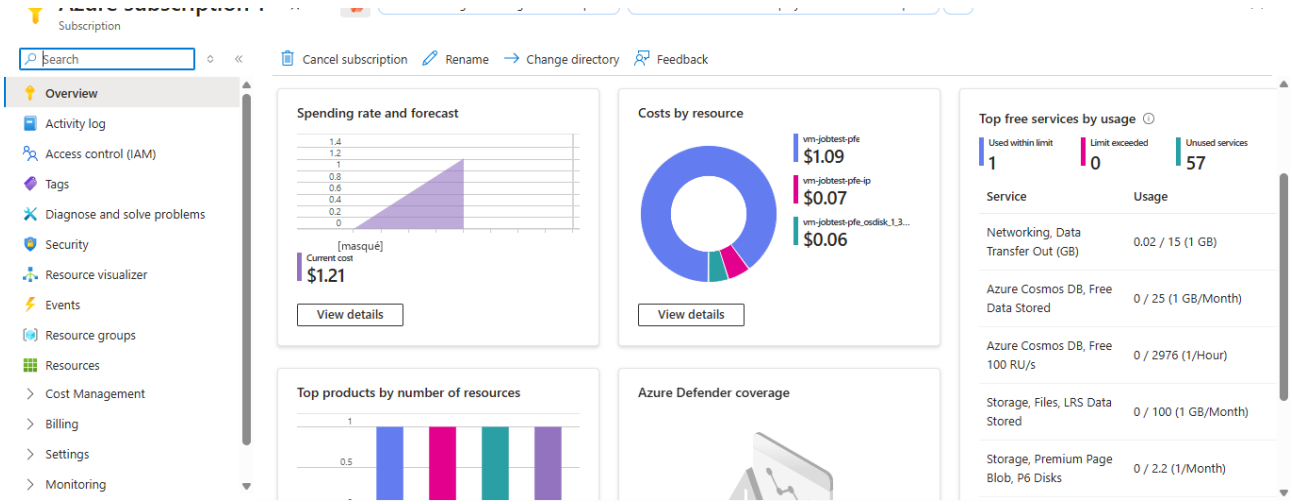


Figure 6.4: Abonnement Azure

La figure 6.5 donne l'aperçu de consommation associé.

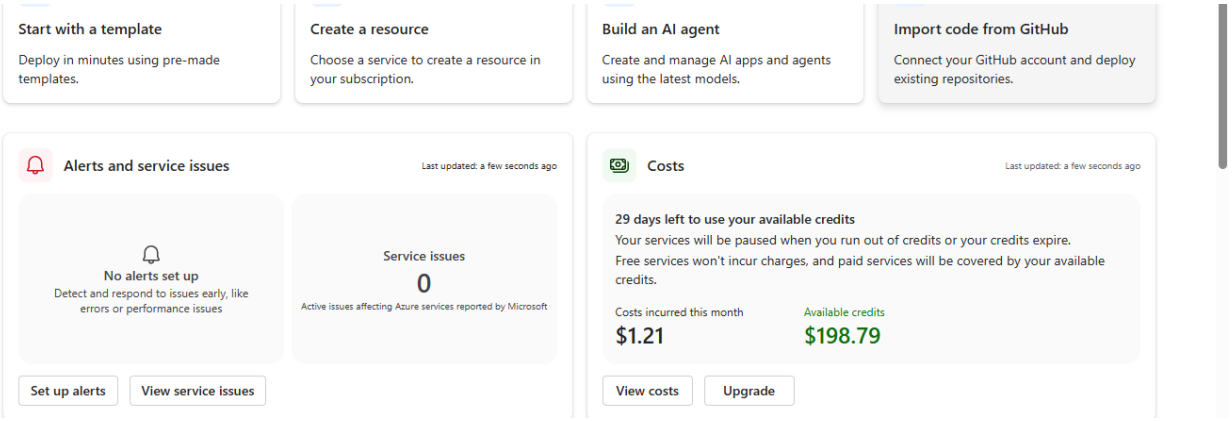


Figure 6.5: Consommation Azure

Les ressources créées pour le projet sont visibles sur la figure 6.6.

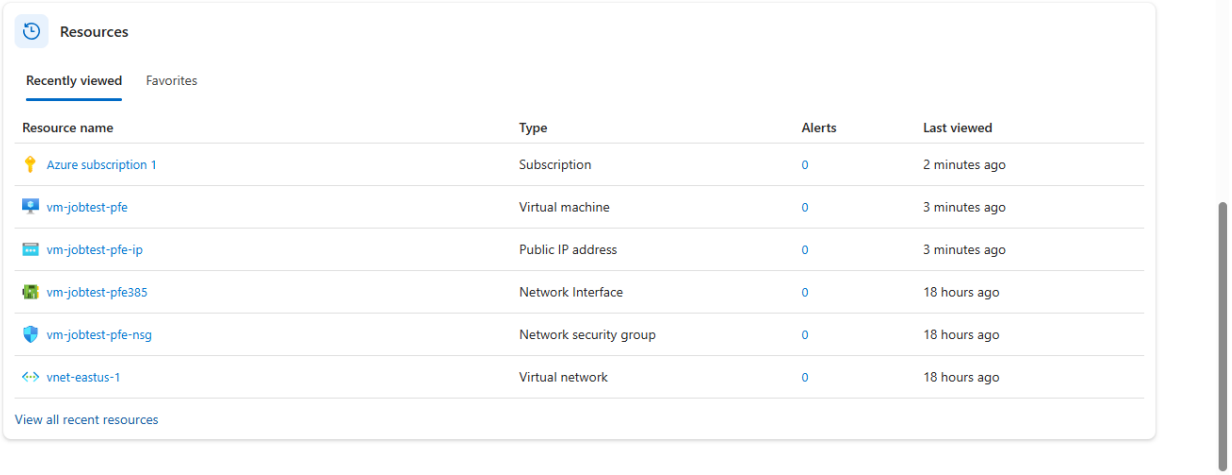


Figure 6.6: Ressources Azure

La figure 6.7 reprend les propriétés de la machine virtuelle.

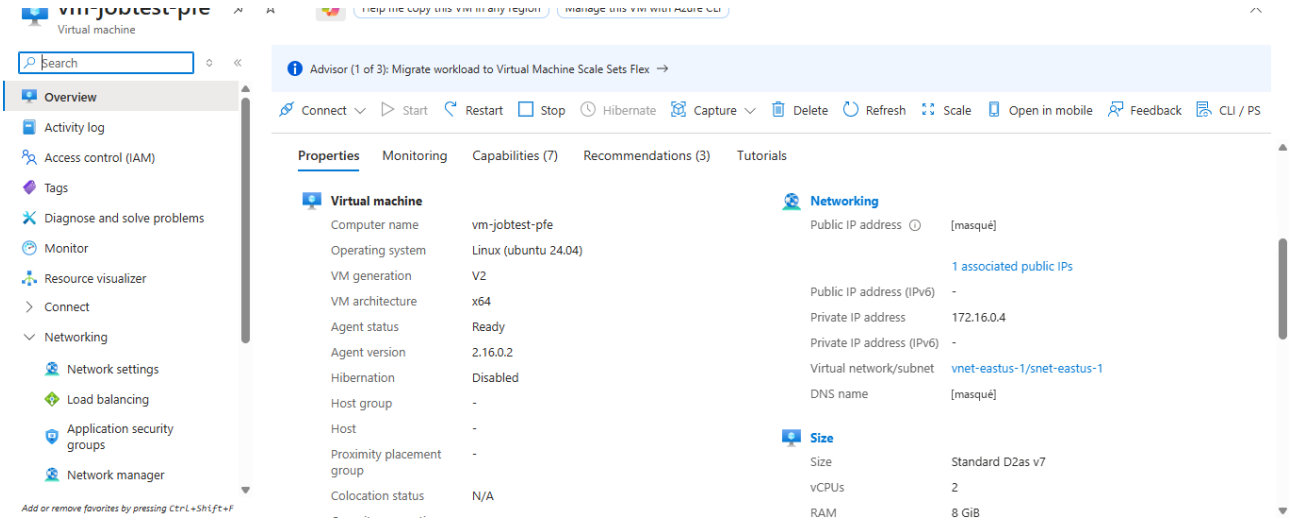


Figure 6.7: Propriétés de la machine virtuelle

Son état d'exécution apparaît sur la figure 6.8.

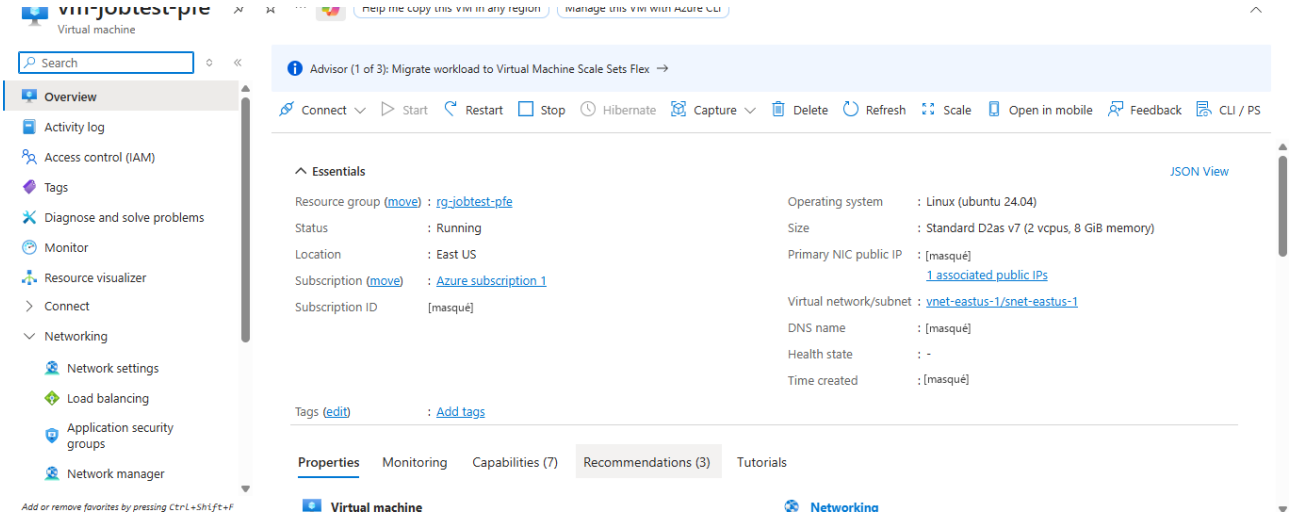


Figure 6.8: État de la machine virtuelle

J'ai retenu le format D2as_v7 comme compromis pour une démonstration de PFE et pour l'exécution locale de `qwen2.5:1.5b`. Il n'est pas dimensionné pour une charge réelle soutenue : la latence de la couche 2 sur CPU illustre cette contrainte. Les règles NSG du déploiement HTTPS sont décrites dans la section consacrée à la sécurité réseau.

6.17 Architecture de déploiement

La figure 6.2 matérialise les trois frontières utiles à l'exploitation : Azure filtre l'entrée publique, la VM héberge le proxy et le réseau Docker porte les communications de service à service. Les volumes `postgres_data`, `ollama_data` et `embedding_cache` séparent les données persistantes des conteneurs recréables. Le navigateur ne communique ni avec PostgreSQL, ni avec Ollama, ni directement avec FastAPI ou Next.js : il atteint le proxy inverse, qui distribue ensuite les routes internes.

6.18 Exploitation et maîtrise des coûts

Le contexte de démonstration mentionne un crédit Azure Free Trial de 200 USD, sans calcul mensuel figé : le coût affiché dépend de la région, du temps d'allocation et des ressources associées. L'arrêt automatique est activé mais ne doit pas être assimilé sans vérification à une désallocation. Microsoft indique que la fonctionnalité d'auto-arrêt est configurable selon un horaire et un fuseau horaire [25]. Après une démonstration, il faut contrôler l'état réel de la VM et privilégier *Stop (deallocate)* pour libérer le calcul. Le disque et, selon sa configuration, l'adresse IP peuvent continuer à générer des coûts ; la persistance de ces ressources explique qu'une désallocation ne soit pas une suppression.

6.19 Procédures d'exploitation

Les commandes suivantes correspondent aux opérations prévues par la procédure de déploiement. Elles ne révèlent pas le contenu de l'environnement ni les secrets.

```
cd /opt/jobtest
docker compose ps
docker compose logs --tail=200 caddy api web ollama db
docker compose restart
docker compose up -d --build
docker compose down
docker compose exec -T db \
  sh -c 'pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB"' \
  | gzip > jobtest-backup.sql.gz
```

La dernière commande est la forme documentée de sauvegarde PostgreSQL. La commande `docker compose down` arrête les conteneurs tout en conservant les volumes nommés ; l'option `-v` n'est donc pas utilisée, car elle supprimerait les données persistantes et les caches.

6.20 Bilan du déploiement

Le tableau 6.6 met en regard chaque résultat observé et la limite qui l'accompagne.

Tableau 6.6: Synthèse des résultats et limites

Élément	**Résultat observé**	**Limite**
Interface Next.js	Accessible derrière le proxy, en HTTPS.	Instance unique, sans haute disponibilité.
API FastAPI	Santé et routes publiques de démonstration disponibles.	Ressources CPU/RAM limitées.
PostgreSQL	Persistance d'un résultat et d'une analyse validée.	Sauvegarde à automatiser.
Couche 1	Évaluation et explication fonctionnelles.	Validation réelle avec recruteurs encore nécessaire.
Couche 2	Analyse `ok` disponible.	Latence CPU élevée ; contenu consultatif.
PDF	Export valide sur un exemple de deux pages.	Test ponctuel, taille variable.
Réseau	Ports applicatifs internes non publiés.	SSH à restreindre ; UFW inactif observé.
Disponibilité	Démonstration réussie avec cinq services sains.	VM unique, sans haute disponibilité.

Conclusion

Les preuves réunies confirment le fonctionnement d'une chaîne complète de démonstration : l'évaluation, la persistance, l'analyse qualitative et l'export PDF sont servis par une pile conteneurisée, avec un proxy inverse comme point d'entrée, l'accès en HTTPS et des services métier non publiés directement. Le déploiement Azure est donc opérationnel dans son périmètre de PFE. Il ne constitue toutefois ni une certification de sécurité ni une infrastructure de production : le durcissement de SSH, les sauvegardes, la supervision et la tolérance aux pannes restent à mettre en œuvre. Ces résultats permettent de replacer, dans la conclusion générale, les apports du prototype et les conditions de son évolution.

Conclusion générale

Le recrutement confronte les organisations à un volume important de CV, à des offres exprimées de manière hétérogène et à la nécessité de justifier un premier tri. Dans ce contexte, rapprocher automatiquement un CV et une offre ne doit pas réduire une candidature à un classement opaque. Le besoin auquel répond JobTest est celui d'une aide structurée : mettre en évidence les éléments de compatibilité, les écarts et les raisons d'une recommandation, tout en laissant au recruteur la décision finale.

Le projet avait pour objectif de concevoir et réaliser une plateforme Web d'aide au recrutement capable de gérer les offres, les candidats et leurs CV, de produire une évaluation hybride, d'en restituer une explication et d'en préserver la traçabilité. La solution livrée propose un frontend Next.js, une API FastAPI, une persistance SQLAlchemy utilisant SQLite en développement et PostgreSQL dans l'environnement conteneurisé, ainsi qu'un export PDF. Le recruteur peut créer ses offres, gérer ses candidats, lancer une évaluation, consulter son historique et obtenir un rapport. Les contrôles d'appartenance des données maintiennent la séparation entre les espaces des recruteurs.

La démarche a suivi les étapes de l'étude du contexte, de l'analyse des besoins, de la conception UML et architecturale, du développement, de la conception du moteur de rapprochement, puis de la validation et du déploiement. Elle s'inscrit dans le cycle en cascade présenté au début du rapport, tout en intégrant les retours techniques nécessaires au fil des vérifications. L'industrialisation a complété le développement par Docker Compose, un proxy inverse comme point d'entrée et un déploiement de démonstration sur une VM Microsoft Azure, servi en HTTPS.

La contribution centrale est un moteur hybride qui combine les signaux TF–IDF, sémantique et compétences, contrôle les preuves présentes dans le CV, calibre son score puis applique des règles de décision explicites. Le score est reproductible, versionné et s'accompagne de sous-scores, de forces, de lacunes et de recommandations structurées. Un CV qui recopie les exigences de l'offre ou qui ne fait que les lister n'obtient pas le verdict `HIRE`. Une seconde couche locale, exécutée avec Ollama, produit une analyse qualitative séparée : elle enrichit la lecture du recruteur, mais ne peut ni modifier le score officiel ni déterminer seule la recommandation algorithmique `HIRE`, `CONSIDER` ou `REJECT`. Cette séparation est un résultat important du projet, car elle maintient une distinction claire entre calcul déterministe et contenu génératif.

Les validations réunies montrent que le parcours complet fonctionne : authentification, création d'offre, évaluation d'un texte ou d'un document, persistance, analyse approfondie et génération PDF. Je l'ai exercé sur des données synthétiques, sur des CV fictifs au format PDF et sur mes propres CV face à des offres réelles. Le déploiement a également confirmé les cinq services conteneurisés, le proxy inverse,

l'accès HTTPS et l'isolation des ports métier du réseau public. Sur le test synthétique isolé, les verdicts affichés par la version finale atteignent un macro-F1 de 0,740 sur trois classes, sans dépasser TF–IDF seul calibré. Ces mesures restent internes au corpus synthétique ; les essais menés avec mes propres CV, faute de jugements de référence indépendants, restent qualitatifs. La valeur démontrée est celle d'un prototype fonctionnel et traçable, non celle d'un système déjà validé en situation opérationnelle.

Cette prudence est essentielle. Le jeu d'évaluation quantitative est synthétique, les ancrages de calibration sont appris sur ses étiquettes, les poids restent fixés empiriquement, le dictionnaire de compétences est nécessairement incomplet et aucune annotation par des recruteurs n'a été réalisée. Un CV qui décrit faussement une expérience n'est pas détecté par l'analyse du texte. Les biais possibles dans les données, les libellés ou les règles doivent être étudiés avant tout usage réel. La couche 2 présente une latence notable sur CPU et son texte, bien que contrôlé par des statuts, reste consultatif. Du point de vue de l'infrastructure, la démonstration est hébergée sur une VM unique, sans haute disponibilité, sans sauvegarde automatisée ni supervision avancée. Une politique explicite de protection, de conservation, d'anonymisation et de suppression des CV demeure également indispensable.

Les évolutions prioritaires à court terme sont la sécurisation des sessions par cookies adaptés, la restriction de SSH, les sauvegardes PostgreSQL, le monitoring et la correction des limites de téléversement. L'amélioration de l'interface et l'optimisation de la couche 2 peuvent progresser en parallèle. À moyen terme, il sera nécessaire de confronter le système à un corpus réel anonymisé et autorisé, de travailler avec des recruteurs, d'enrichir le dictionnaire de compétences, de réapprendre la calibration du score sur ces données annotées, ce qui constitue la priorité, d'évaluer les biais et de formaliser l'évaluation des explications génératives, y compris dans plusieurs langues.

À plus long terme, une charge réelle justifierait des traitements asynchrones, une file de tâches, une meilleure observabilité et, si nécessaire, la séparation de certains services sur une infrastructure évolutive. Des intégrations avec des systèmes de gestion de recrutement pourraient être envisagées, accompagnées de mécanismes de gouvernance et d'audit des décisions assistées. Ces perspectives ne sont pas des fonctions déjà réalisées ; elles dessinent les conditions d'une évolution responsable.

JobTest constitue ainsi un prototype fonctionnel et une base technique cohérente pour l'aide à l'analyse des candidatures. Il organise des signaux et des explications utiles au recruteur, sans prétendre remplacer son jugement, l'appréciation du contexte ou sa responsabilité dans la décision finale.

Bibliographie

[1] CyberCloud Solutions. "CyberCloud Solutions : page entreprise," LinkedIn. adresse : https://www.linkedin.com/company/cybercloudsolutions/

[2] J. Berg. "The Messy Business of Managing People at Work: Is AI the Solution?" International Labour Organization. adresse : https://www.ilo.org/resource/article/messy-business-managing-people-work-ai-solution

[3] National Institute of Standards and Technology. "Artificial Intelligence — Glossary," NIST. adresse : https://csrc.nist.gov/glossary/term/artificial_intelligence

[4] A. Koechling et M. C. Wehner, "Discriminated by an Algorithm: A Systematic Review of Discrimination and Fairness by Algorithmic Decision-Making in the Context of HR Recruitment and HR Development," *Business Research*, t. 13, p. 795-848, 2020. doi : 10.1007/s40685-020-00134-w adresse : https://doi.org/10.1007/s40685-020-00134-w

[5] J. Devlin, M.-W. Chang, K. Lee et K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, p. 4171-4186. doi : 10.18653/v1/N19-1423 adresse : https://aclanthology.org/N19-1423/

[6] N. Reimers et I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, p. 3982-3992. doi : 10.18653/v1/D19-1410 adresse : https://aclanthology.org/D19-1410/

[7] LinkedIn. "Search with AI-Assisted Search in Recruiter," LinkedIn Recruiter Help. adresse : https://www.linkedin.com/help/recruiter/answer/a6509735

[8] LinkedIn. "AI Features in LinkedIn Recruiter," LinkedIn Recruiter Help. adresse : https://www.linkedin.com/help/recruiter/answer/a7784112

[9] Workable. "Candidate Profile in Pipeline View Overview," Workable Help. adresse : https://help.workable.com/hc/en-us/articles/115012857047-Candidate-profile-in-pipeline-view-overview

[10] Workable. "Workable Security and Data Privacy," Workable Help. adresse : https://help.workable.com/hc/en-us/articles/115011942227-Workable-security-and-data-privacy

[11] Greenhouse. "Greenhouse AI Features," Greenhouse Support. adresse : https://support.greenhouse.io/hc/en-us/articles/33043749845403-Greenhouse-AI-features

[12] Greenhouse. "Scorecard Overview," Greenhouse Support. adresse : https://support.greenhouse.io/hc/en-us/articles/4414777492891-Scorecard-overview

[13] Greenhouse. "Essential Candidate Report," Greenhouse Support. adresse : https://support.greenhouse.io/hc/en-us/articles/360042740232-Essential-candidate-report

[14] W. W. Royce, "Managing the Development of Large Software Systems," in *Proceedings of IEEE WESCON*, 1970, p. 1-9. adresse : https://www.praxisframework.org/files/royce1970.pdf

[15] ISO/IEC. "ISO/IEC 25010:2023 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model," International Organization for Standardization. adresse : https://www.iso.org/standard/78176.html

[16] Object Management Group. "Unified Modeling Language, Version 2.5.1," OMG. adresse : https://www.omg.org/spec/UML/2.5.1/PDF

[17] scikit-learn developers. "TfidfVectorizer," scikit-learn. adresse : https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[18] scikit-learn developers. "cosine_similarity," scikit-learn. adresse : https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html

[19] scikit-learn developers. "Metrics and scoring: quantifying the quality of predictions," scikit-learn. adresse : https://scikit-learn.org/stable/modules/model_evaluation.html

[20] Microsoft. "Azure Network Security Groups Overview," Microsoft Learn. adresse : https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview

[21] Docker Inc. "Networking in Compose," Docker Docs. adresse : https://docs.docker.com/compose/how-tos/networking/

[22] OWASP Foundation. "HTML5 Security Cheat Sheet," OWASP Cheat Sheet Series. adresse : https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

[23] République Tunisienne, *Loi organique 2004-63 du 27 juillet 2004, portant sur la protection des données à caractère personnel*, Journal Officiel de la République Tunisienne, Texte instituant le régime tunisien de protection des données à caractère personnel et l'autorité de contrôle correspondante, juill. 2004.

[24] Instance Nationale de Protection des Données à Caractère Personnel. "Instance Nationale de Protection des Données à Caractère Personnel (INPDP)," INPDP. adresse : https://www.inpdp.nat.tn/

[25] Microsoft. "Auto-shutdown a Virtual Machine," Microsoft Learn. adresse : https://learn.microsoft.com/en-us/azure/virtual-machines/auto-shutdown-vm

Résumé

Ce mémoire présente JobTest, une plateforme Web d'aide à la présélection qui rapproche des CV et des offres d'emploi. La solution combine des signaux TF–IDF, sémantiques et de compétences, contrôle les preuves présentes dans le CV, calibre le score obtenu, applique une décision explicite et restitue ses composantes au recruteur. Elle intègre un frontend Next.js, une API FastAPI, une persistance relationnelle, un export PDF et une analyse générative locale optionnelle, séparée du verdict officiel. Les validations couvrent les parcours principaux, un banc d'essai quantitatif sur données synthétiques, des essais fonctionnels avec des CV fictifs et avec mes propres CV, ainsi qu'un déploiement de démonstration conteneurisé sur Azure. Sur le test synthétique, les verdicts affichés atteignent un macro-F1 de 0,74 sur trois classes ; la calibration doit encore être réapprise sur des données réelles annotées. JobTest assiste le recruteur sans remplacer son jugement ni une décision humaine.

Mots-clés : recrutement, appariement CV-offre, IA explicable, FastAPI, Next.js.

Abstract

This thesis presents JobTest, a web platform that assists recruiters in matching resumes with job offers. The solution combines TF–IDF, semantic and skill-based signals, screens the evidence contained in the resume, calibrates the resulting score, applies an explicit decision policy, and exposes the components of the result to the recruiter. It includes a Next.js frontend, a FastAPI API, relational persistence, PDF export, and an optional local generative analysis that is kept separate from the official verdict. The validation covers the main user flows, a quantitative benchmark on synthetic data, functional trials with fictitious resumes and with my own resumes, and a containerized demonstration deployment on Azure. On the synthetic test set, the displayed verdicts reach a three-class macro-F1 of 0.74; the calibration still has to be re-fitted on annotated real data. JobTest supports recruiters without replacing their judgement or human decision-making.

Keywords: recruitment, CV-job matching, explainable AI, FastAPI, Next.js.

Démonstration de JobTest

`https://jobtest-pfe-iheb.eastus.cloudapp.azure.com/login`

Instance de démonstration hébergée sur Microsoft Azure et servie en HTTPS, sans garantie de disponibilité permanente. N'y saisir que des données fictives.